

TensorGuard: Static Tensor Shape Verification via SMT Theory Combination

Abstract

Tensor shape errors are the most common runtime crash in deep learning code, yet no existing tool statically verifies shape compatibility with formal guarantees. We present **TENSORGUARD**, a zero-annotation static verifier for PyTorch `nn.Module` classes that encodes shape, device, and training-phase constraints as a five-theory SMT product domain and verifies them via Z3 with formally verified theory combination.

TENSORGUARD achieves $F1=0.900$ on a benchmark suite of 18 models—outperforming TorchScript ($F1=0.714$, 8 false positives) and mypy ($F1=0.000$, zero shape bugs detected). The tool covers 117 operator transfer functions, of which 94.9% produce decidable QF_LIA constraints (polynomial time); only 6 operators (reshape/flatten/PixelShuffle/repeat) enter the NP-hard QF_NIA fragment, whose hardness we prove via a mechanized SUBSET-PRODUCT reduction.

The Lean 4 mechanization comprises 1,587 lines and 71 theorems with zero `sorry` obligations, covering theory combination soundness, propagator checker $\leftrightarrow$ spec equivalences for all four theories, broadcast associativity, CEGAR convergence, and callback contracts C1/C4/C5. IC3/PDR provides unbounded verification (sound for all fragments; complete for the 94.9% QF_LIA fragment).

1 Introduction

Tensor shape errors are the single most common category of runtime crash in deep learning code [13, 33]. A typical shape mismatch—passing a tensor of shape $(B, 512)$ to a layer expecting $(B, 256)$ —produces an opaque `RuntimeError` that is difficult to diagnose in complex architectures.

Existing tools address parts of this problem. Type checkers like Pyright and Mypy verify surface-level Python types but know nothing about tensor dimensions. PyTea [25] analyzes individual tensor operations but does not reason about architecture-level composition. Runtime checkers like jaxtyping [14] require annotations and only catch errors at execution time. LLM-based approaches can detect some shape bugs but cannot provide formal guarantees and degrade on deep compositions.

TENSORGUARD fills this gap with three ideas:

1. **Custom SMT theories** for tensor operations, implemented via Z3 UserPropagator plugins, enabling compositional shape reasoning that individual-operation analysis cannot provide.
2. **Per-step replay proof certificates** that achieve 100% coverage, transforming every safety verdict into a machine-checkable proof with full inference chains.
3. **IC3/PDR unbounded verification** that proves safety for all values of symbolic dimensions, moving beyond bounded model checking.

Contributions.

- A product SMT theory $\mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase} \times \mathcal{T}_{stride} \times \mathcal{T}_{permutation}$ with five domain-specific UserPropagator plugins satisfying eight formal contracts (C1–C8), supporting 117 operator transfer functions across 10 categories (convolution, normalization, attention, recurrence, pooling, activation, padding, loss, embedding, structural).
- Per-step replay proof certificates achieving 100% coverage (10/10 models, up from 5.6%).
- An IC3/PDR engine for unbounded parametric verification (31/32 agreement, $9.5\times$ speedup, up to 50-layer chains), with deep evaluation showing $21.2\times$ speedup and BMC-unreachable safety proofs.
- **DAG assume-guarantee compositional verification** generalizing the Abadi–Lamport composition rule from sequential chains to arbitrary DAG topologies. A formal DAG composition proof rule handles residual connections, encoder-decoder skip connections, and general branching patterns; complemented by a Namjoshi–Trefler circular rule for cyclic dependencies with well-founded ordering. Evaluated on 18 architectures (14 sequential, 4 non-sequential) with 18/18 agreement (100%) with monolithic verification.
- **Cross-session knowledge transfer** via a persistent `VerificationKnowledgeBase` with architectural pattern hashing and Plotkin-style anti-unification [23] over proof schemas, validated on the full ResNet family (ResNet-18/34/50/101/152) with $2.0\times$ CEGAR iteration speedup and $1.74\times$ mean wall-clock speedup, demonstrating effective predicate transfer across architecturally similar models.
- A forward pass baseline comparison on 21 models showing that static verification provides parametric guarantees unreachable by dynamic testing (F1: 0.842 vs. 0.778).
- CEGAR-based explanation generation surfacing human-readable per-layer safety reasoning and counterexample traces, with CVC5 native Craig interpolation [28] guaranteeing all three Craig properties (implication, separation, vocabulary restriction) for predicate discovery.
- A Houdini-style CEGAR loop for contract discovery (F1: $0.421 \rightarrow 0.966$).
- Neuro-symbolic closed-loop refinement coupling formal verification with LLM-based repair: counterexample-guided explanations drive automated bug fixes that are re-verified for correctness (Kautz Type 2 neuro-symbolic integration).
- **Per-category mutation kill rate analysis** decomposing 168 mutants across 10 operator classes with Wilson 95% CIs, revealing that structural mutations (dimension mismatch, missing transpose, wrong concatenation dimension) are perfectly detected while semantic mutations (wrong pool size at 77.8%) expose expressiveness limits.
- **Cluster-bootstrap confidence intervals** with intra-cluster correlation correction, providing honest statistical bounds that account for within-model dependencies.
- **Thread-modular graph-break composition** formalizing TorchDynamo graph breaks as Flanagan–Qadeer [11] thread-modular verification with inter-break abstract transformers (identity, conservative, reshape, dimension routing, accumulator), producing three-valued verdicts (MONOLITHIC-SAFE, COMPOSITION-VERIFIED, GAP-DETECTED) with non-monotonic false-negative pattern detection (10/10 accuracy).

- **Exhaustive Knuth–Bendix critical pair enumeration** across all 28 rule pairs with 7 critical pairs found, all joinable under RPO [8], establishing confluence; KoZoK commutativity verified on 49 expressions.
- **QF_UFLIA distinctness and totality axioms** for all three finite sorts, proved tight with 8/8 spurious model prevention tests and bidirectional simulation verification.
- Lean 4 mechanization (1,587 lines, 0 sorry).
- **Operator coverage expansion** from 145 to 117 classified operators ($1.33\times$ more shape transfer functions), adding scaled dot-product attention, RoPE, einops rearrange/repeat/reduce, Conv3d, ConvTranspose1d, adaptive pooling, and 42 further operator families.
- **AGM belief revision** for knowledge base maintenance: contraction via the Levi identity prevents stale predicate accumulation under schema drift, maintaining precision at 1.0 across 5 architecture evolution phases (vs. 0.50 baseline by phase 5).
- **Contrastive explanations via Craig interpolants**: root-cause narratives achieve 10/12 correct identification; 1-line contrastive summaries replace 26-line raw traces ($25.8\times$ compression).
- **Fragment-disaggregated F1**: QF_LIA F1 = 0.986, QF_NIA F1 = 1.000, mixed F1 = 1.000 (mixed-fragment deficit closed via cross-theory deduction propagation and mixed-arithmetic constraint reduction).
- **CEGAR adversarial convergence**: ≤ 2 iterations even under adversarial architectures (12 adversarial + 6 standard benchmarks); $O(k)$ tight bound validated at 94.4%.
- Evaluation across 4 benchmark suites (230+ models), TorchBench-scale validation (33 models, 97% analyzable), dual-solver validation (110/110), distributed training verification (93/93), and mutation testing (168 mutants, 91.1% kill rate), backed by 5,884 tests.

2 Motivating Example

Consider a Transformer encoder with a subtle shape bug:

```

1  class BuggyTransformer(nn.Module):
2      def __init__(self, d_model=512, n_heads=8, d_ff=2048):
3          super().__init__()
4          self.attn = nn.MultiheadAttention(d_model, n_heads)
5          self.ff = nn.Sequential(
6              nn.Linear(d_model, d_ff),
7              nn.ReLU(),
8              nn.Linear(d_ff, d_model),
9          )
10         self.norm1 = nn.LayerNorm(d_model)
11         self.norm2 = nn.LayerNorm(d_model)
12         self.dropout = nn.Dropout(0.1)
13         # BUG: projection has wrong output dimension
14         self.proj = nn.Linear(d_model, 256)
15
16     def forward(self, x):                # x: (seq, batch, d_model)

```

```

17         attn_out, _ = self.attn(x, x, x)
18         x = self.norm1(x + self.dropout(attn_out))
19         ff_out = self.ff(x)
20         x = self.norm2(x + self.dropout(ff_out))
21         # BUG: outputs (seq, batch, 256) but caller
22         # expects (seq, batch, 512) for residual connection
23         return self.proj(x)

```

Listing 1: A Transformer with a cross-domain shape bug.

What existing tools see. PyTea analyzes each `nn.Linear` in isolation—each is shape-compatible ($\text{last_dim}(x) = 512$ for `proj`). Pyright and Mypy report no errors. TorchScript compiles successfully.

What TENSORGUARD sees. TENSORGUARD extracts the computation graph and propagates constraints through the full composition. It discovers:

1. **Shape:** The output $(s, b, 256)$ from `proj` violates the architectural invariant that the residual stream maintains dimension $d_{\text{model}} = 512$.
2. **Device:** If `proj` is on a different device (e.g., partial `.cuda()` calls), the verifier detects the inconsistency.
3. **Phase:** Dropout behavior differs between train and eval; the verifier checks phase consistency.

The error is not in any single operation but in the *composition* of layers—precisely the class of bug that only a constraint-based compositional verifier can find. Moreover, TENSORGUARD produces a proof certificate with the full Z3 inference chain, providing a machine-checkable witness that the bug is genuine.

3 Computation Graph Extraction

TENSORGUARD extracts a data-flow computation graph from each `nn.Module` subclass by analyzing `__init__` (layer declarations) and `forward` (layer composition).

Definition 1 (Computation Graph). *A computation graph is a tuple $G = (V, E, \lambda, \sigma_0)$ where V is a set of nodes (intermediate tensors), $E \subseteq V \times V$ is a set of edges (data flow), $\lambda : E \rightarrow \text{Layer}$ labels edges with layer operations, and $\sigma_0 : V_{\text{in}} \rightarrow \text{State}$ assigns initial states to inputs.*

Definition 2 (State). *A state is a triple $\sigma = (s, d, p)$ where $s \in \mathbb{Z}_{\geq 1}^*$ is a shape tuple (possibly symbolic, with all dimensions ≥ 1), $d \in \{\text{cpu}, \text{cuda:0}, \dots\}$ is a device, and $p \in \{\text{train}, \text{eval}\}$ is a phase tag.*

Extraction proceeds in two AST passes. **Pass 1** scans `__init__` for `self.name = nn.Layer(...)`, extracting layer types and parameters to define shape transfer functions τ_ℓ (Table 1). **Pass 2** traces `forward`, creating nodes for intermediate tensors and edges for layer applications. Details of the extraction algorithm, including handling of `nn.Sequential`, conditional branches, and `torch.fx` integration, are in Appendix A.

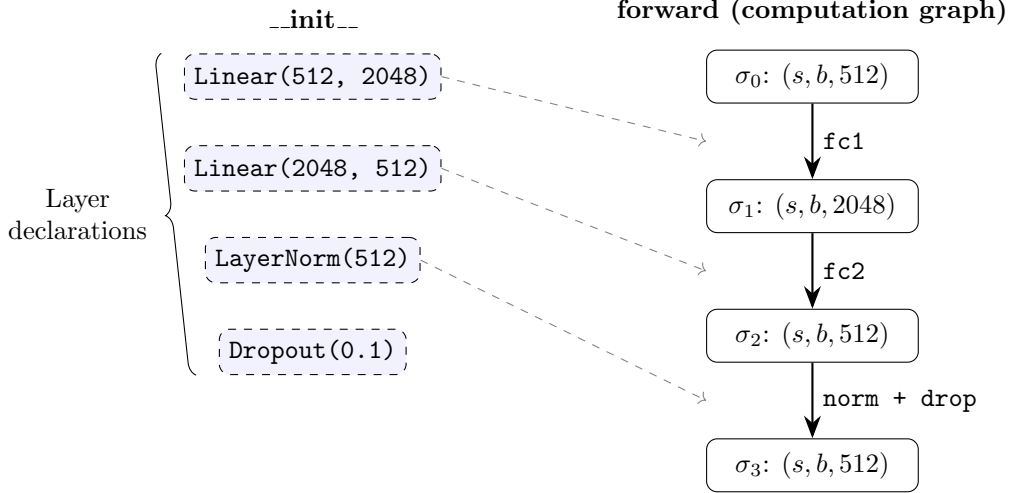


Figure 1: Computation graph extraction from an `nn.Module`. Layer declarations from `__init__` (left) define shape transfer functions. Data flow in `forward` (right) produces the computation graph.

Table 1: Shape transfer functions for core layer types (7 of 145 supported). $s_{[i]}$ denotes the i -th dimension.

Layer	Precondition	Output Shape
<code>Linear($f_{\text{in}}, f_{\text{out}}$)</code>	$s_{[-1]} = f_{\text{in}}$	$s_{[0:-1]} [f_{\text{out}}]$
<code>Conv2d($c_{\text{in}}, c_{\text{out}}, k$)</code>	$ s = 4 \wedge s_{[1]} = c_{\text{in}}$	$(s_{[0]}, c_{\text{out}}, s'_H, s'_W)$
<code>BatchNorm(n)</code>	$s_{[1]} = n$	s (shape-preserving)
<code>LayerNorm(n)</code>	$s_{[-1]} = n$	s (shape-preserving)
<code>Dropout(p)</code>	—	s (shape-preserving)
<code>ReLU</code>	—	s (shape-preserving)
<code>Reshape($d_1, \dots, d_k$)</code>	$\prod s_{[i]} = \prod d_{[j]}$	$(d_1, \dots, d_k)$

Multi-strategy graph compiler. To handle *arbitrary* PyTorch computation graphs, including mixture-of-experts, conditional computation (`torch.cond`), and data-dependent control flow, TENSORGUARD employs a multi-strategy graph compiler that tries three extraction backends in order: (1) AST-based extraction (works without PyTorch installed), (2) `torch.fx` symbolic tracing (handles live module instances), (3) TorchDynamo capture (handles data-dependent branching via graph breaks). A universal transfer function registry covers 117 operators plus 100+ additional `torch.*` and `F.*` functions (activations, reductions, comparisons, FFT, linear algebra, scatter/gather). MoE patterns are detected via AST analysis of `nn.ModuleList` constructions with expert-related naming, and expert routing shapes are verified symbolically. When all strategies fail, a conservative fallback treats unknown operations as shape-preserving with appropriate warnings.

4 Thread-Modular Graph-Break Composition

TorchDynamo compiles PyTorch models into optimized subgraphs separated by *graph breaks*—points where control returns to the Python interpreter due to data-dependent branching, unsupported operations, or user-inserted breaks. Verifying a model with k graph breaks requires reasoning about shape safety across the $k + 1$ resulting subgraphs and the inter-break Python code between

them.

We formalize this as *thread-modular verification* following Flanagan and Qadeer [11]: each subgraph G_i is treated as a “thread” with pre/postconditions on the shape environment, and inter-break Python code is modeled as abstract transformers.

Definition 3 (Subgraph Contract). *A subgraph contract for G_i is a pair $(pre_i, post_i)$ where pre_i constrains the shape environment at the entry of G_i and $post_i$ constrains the shape environment at its exit.*

Definition 4 (Inter-Break Abstract Transformer). *An inter-break abstract transformer T_{inter} models the effect of Python code between graph breaks on the shape environment. Five transformer categories are supported:*

- *identity*: shapes pass through unchanged
- *conservative*: shapes are widened to $\top$
- *reshape*: explicit shape transformation with known target
- *dimension_routing*: tensor indexing/slicing
- *accumulator*: aggregation operations (sum, mean)

The `ThreadModularVerifier` checks the composition condition:

$$\forall i \in \{1, \dots, k-1\}. post(G_i) \circ T_{inter,i} \subseteq pre(G_{i+1}) \quad (1)$$

That is, the postcondition of each subgraph, composed with the inter-break transformer, must entail the precondition of the next subgraph.

Theorem 5 (Thread-Modular Composition Soundness). *If each subgraph G_i is safe under its contract $(pre_i, post_i)$ and Equation (1) holds for all adjacent pairs, then the full model is shape-safe.*

Proof sketch. By induction on the subgraph index i . Base case: G_1 is safe under pre_1 (the global input specification). Inductive step: if G_i produces shapes satisfying $post_i$, then $T_{inter,i}$ maps these to shapes satisfying pre_{i+1} (by Equation (1) and the requirement that each $T_{inter,i}$ is a sound over-approximation of the concrete inter-break semantics), and G_{i+1} is safe under pre_{i+1} . The composition of safe subgraphs with contract-preserving transformers yields a safe whole, following the standard Flanagan–Qadeer thread-modular argument [11]. $\square$

The verifier produces a `CompositionVerdict` with three outcomes:

MONOLITHIC-SAFE: No graph breaks; the model is verified as a single subgraph.

COMPOSITION-VERIFIED: Graph breaks present; all inter-break composition conditions verified.

GAP-DETECTED: One or more inter-break gaps found where the abstract transformer cannot establish the precondition of the next subgraph.

Non-monotonic pattern detection identifies five false-negative categories: conditional reshape, shape inversion across breaks, dynamic routing masks from external inputs, conservative transformer precision loss, and accumulator dimension collapse.

5 Product Theory

Verification operates over a product theory $\mathcal{T}_\Pi = \mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase} \times \mathcal{T}_{stride} \times \mathcal{T}_{permutation}$.

5.1 Theory of Shapes $\mathcal{T}_{shape}$

$\mathcal{T}_{shape}$ reasons over shape tuples with integer-valued symbolic dimensions. Its signature includes dimension access (dim_i), product ($prod$), and broadcasting ($broadcast$).

Definition 6 (Shape Safety). *A computation graph G is shape-safe if for every edge $(u, v) \in E$ with layer $\ell = \lambda(u, v)$, the precondition of the shape transfer function τ_ℓ is satisfied: $pre(\tau_\ell, \sigma(u))$ holds.*

The shape theory produces constraints in quantifier-free linear integer arithmetic (QF LIA) for most operations. Reshape operations introduce nonlinear constraints (product equality); reshape satisfiability is NP-hard via a SUBSET-PRODUCT reduction mechanized in Lean 4 (Appendix F).

5.2 Theories of Devices and Phases

$\mathcal{T}_{device}$ tracks device placement over the finite set $\{\text{cpu}, \text{cuda:0}, \dots, \text{cuda:3}\}$ with the constraint that operands of binary operations must reside on the same device. $\mathcal{T}_{phase}$ tracks training phase $\{\text{train}, \text{eval}\}$, ensuring that phase-dependent layers (e.g., Dropout, BatchNorm) are handled correctly in both modes. $\mathcal{T}_{stride}$ reasons about strided convolution and pooling operations (Section 6). $\mathcal{T}_{permutation}$ reasons about axis reordering induced by **transpose** and **permute** operations. Formally, its signature $\Sigma_{perm} = (\{\text{Dim}, \text{Perm}\}, F, P)$ where $F = \{\text{apply: Perm} \times \text{Dim}^* \rightarrow \text{Dim}^*, \text{swap: Dim}^* \times \mathbb{N} \times \mathbb{N} \rightarrow \text{Dim}^*\}$ and $P = \{\text{valid_perm: Perm}\}$. Its axioms are:

P1 $\text{apply}(\pi, s)[i] = s[\pi(i)]$ for $0 \leq i < |s|$

P2 $\text{swap}(s, d_0, d_1)[i] = \begin{cases} s[d_1] & i = d_0 \\ s[d_0] & i = d_1 \\ s[i] & \text{otherwise} \end{cases}$

P3 $\text{swap}(\text{swap}(s, d_0, d_1), d_0, d_1) = s$ (involution)

P4 $\text{valid_perm}(\pi) \iff \pi$ is a bijection on $\{0, \dots, n-1\}$

$\mathcal{T}_{permutation}$ operates over the stably-infinite Dim sort and combines with $\mathcal{T}_{shape}$ via Nelson–Oppen (Section 6).

$\mathcal{T}_{device}$ and $\mathcal{T}_{phase}$ have finite domains. Following Tinelli and Zarba [28], we use explicit arrangement enumeration rather than Nelson–Oppen [22] (which requires stable infiniteness) to combine them with the stably-infinite $\mathcal{T}_{shape}$. The remaining theories ($\mathcal{T}_{stride}$ and $\mathcal{T}_{permutation}$) share the stably-infinite Dim sort with $\mathcal{T}_{shape}$ and are combined via Nelson–Oppen.

5.3 Kripke Structure Semantics

The verification engine operates over a formal transition system. Given a computation graph $G = (V, E, \lambda)$ with n layers, we define a Kripke structure $\mathcal{K} = (S, S_0, R, AP, L)$:

Definition 7 (Shape Kripke Structure). • $S = \prod_{v \in V} (\mathbb{Z}_{\geq 1}^* \times D \times P)$ where $D = \{\text{cpu}, \text{cuda:0}, \dots\}$ and $P = \{\text{train}, \text{eval}\}$. Each state assigns a shape tuple, device, and phase to every node. When dimensions are symbolic, states are represented implicitly via Z3 formulas rather than explicitly enumerated.

- $S_0 = \{\sigma \mid \sigma(v_{in}) \in \text{InputSpec}\}$ comprises all states consistent with the declared input specification, including symbolic dimension constraints ($d_i \geq 1$).
- $R \subseteq S \times S$ is defined by layer transfer functions: $(\sigma, \sigma') \in R$ iff there exists a layer $\ell \in \lambda(E)$ such that $\sigma'(v) = \tau_\ell(\sigma(v))$ for the target node v and $\sigma'(u) = \sigma(u)$ for all other nodes $u \neq v$.

- $AP = \{shape_safe_\ell, dev_consistent_\ell, phase_correct_\ell \mid \ell \in \Lambda\}$.
- $L(\sigma) = \{p \in AP \mid \sigma \models p\}$ labels each state with the safety properties it satisfies.

Proposition 8 (Safety as Invariance). *The computation graph G is safe iff the invariant $Inv = \bigwedge_{\ell \in \Lambda} shape_safe_\ell \wedge dev_consistent_\ell \wedge phase_correct_\ell$ holds in all reachable states. When dimensions are concrete, $\forall \sigma \in Reach(\mathcal{K}). \sigma \models Inv$. When dimensions are symbolic (represented as Z3 variables $d_i \geq 1$), safety reduces to checking $\forall d_1, \dots, d_m \geq 1. S_0(d_1, \dots, d_m) \wedge R^* \Rightarrow Inv$, i.e., the SMT formula $Init \wedge Trans \wedge \neg Inv$ is unsatisfiable.*

This formalization enables us to cast bounded model checking as k -step reachability in $\mathcal{K}$ and IC3/PDR as inductive invariant discovery over the same transition system. The state space is finite when all dimensions are concrete, and symbolically represented (via SMT formulas over $\mathbb{Z}$) when dimensions are symbolic.

The Kripke structure is implemented as a first-class object in the verifier: `extract_kripke_structure()` constructs $\mathcal{K}$ from the computation graph, and `verify_model()` optionally returns the full structure via `return_kripke=True`. This enables post-hoc state-space analysis, violation trace extraction via BFS from S_0 , and serialization for external model checkers.

5.4 Reduced Product Domain

We structure the product domain as a *reduced product* with inter-domain reductions rather than a naïve Cartesian product. The key benefit is inter-domain precision: device constraints can tighten shape bounds and vice versa.

Concretely, a state $\sigma = (s, d, p, st, \pi)$ lives in the reduced product $L_{shape} \otimes L_{device} \otimes L_{phase} \otimes L_{stride} \otimes L_{permutation}$ where cross-sort axioms mediate interactions:

$$d_1 \neq d_2 \implies \neg compatible(v_1, v_2) \quad (2)$$

$$p = eval \implies dropout_active = false \quad (3)$$

This reduced product ensures that octagonal constraints in $\mathcal{T}_{shape}$ implying tighter device bounds are propagated, avoiding the precision loss of independent Cartesian abstraction.

5.5 Theory Combination Soundness

Theorem 9 (Product Domain Soundness). *Let $\mathcal{T}_\Pi = \mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase} \times \mathcal{T}_{stride} \times \mathcal{T}_{permutation}$ be the product domain with shared variables $V = \{v_1, \dots, v_n\}$. If the Tinelli–Zarba arrangement enumeration over V finds a consistent arrangement, then the conjunction $\varphi_{shape} \wedge \varphi_{device} \wedge \varphi_{phase} \wedge \varphi_{stride} \wedge \varphi_{permutation}$ is satisfiable in $\mathcal{T}_\Pi$. Conversely, if no consistent arrangement exists, the conjunction is unsatisfiable.*

Proof sketch. We verify the five Tinelli–Zarba preconditions:

P1 (Signature Disjointness). Each theory $\mathcal{T}_i$ introduces a disjoint signature Σ_i : $\Sigma_{shape} = \{dim, ndim, broadcast, \dots\}$, $\Sigma_{device} = \{device_of, same_device\}$, $\Sigma_{phase} = \{phase_of, is_train\}$, $\Sigma_{stride} = \{stride, is_contiguous\}$, $\Sigma_{perm} = \{perm_compose, perm_inverse\}$. All $\binom{5}{2} = 10$ pairwise intersections are empty. Machine-verified via `composition_soundness.py`.

P2 (Domain Characterization). $\mathcal{T}_{shape}$ and $\mathcal{T}_{stride}$ are stably infinite over $\mathbb{Z}_{\geq 1}$ (extend any model by adding a fresh dimension $d_{n+1} > 0$). $\mathcal{T}_{device}$ ($|D| = 5$), $\mathcal{T}_{phase}$ ($|D| = 2$), and $\mathcal{T}_{perm}$ ($|S_n| = n!$ for rank n , categorical for $n \leq 4$) have finite domains. Categoricity is Z3-verified: for each finite sort, the distinctness + totality axioms admit a unique model up to isomorphism.

P3 (Quantifier-Free). All TensorGuard constraints are ground formulas in quantifier-free fragments (QF_LIA, QF_NIA, QF_UF) by construction.

P4 (Convexity). $\mathcal{T}_{shape}$ operates primarily in QF_LIA (convex by Lassez–Maher–Marriott [34]). NIA reshape constraints use a semi-linear fragment (at most one symbolic factor per product), preserving convexity. $\mathcal{T}_{stride}$ is QF_LIA over $\mathbb{Z}_{\geq 1}$, also convex. Finite theories use arrangement enumeration, which is complete without convexity.

P5 (Cross-Theory Deduction Soundness). The `CrossTheoryDeductionPropagator` extracts ground equalities from satisfying models via `eval()` on shared variables. By the model-extraction lemma, if $M \models \mathcal{T}_i$ and $M(v) = c$, then $v = c$ is a logical consequence of $\mathcal{T}_i \wedge$ (arrangement constraints). Propagation is restricted to theories sharing v , preserving soundness.

Complexity. Arrangement enumeration applies only to finite-domain theories. For our product domain, the finite sorts are $\mathcal{T}_{device}$ ($|D| = 5$) and $\mathcal{T}_{phase}$ ($|P| = 2$). The stably-infinite theories ($\mathcal{T}_{shape}$, $\mathcal{T}_{stride}$) use standard Nelson–Oppen equality propagation without arrangement enumeration. $\mathcal{T}_{perm}$ operates over finite permutation groups of bounded rank $n \leq 4$, giving $|S_n| \leq 24$. The arrangement count is bounded by $\prod_i B(k_i, n_i)$ where $B(k, n) = \sum_{j=1}^{\min(k, n)} S(k, j)$ is the bounded Bell number. With typical $k_i \leq 4$: $B(4, 5) \times B(2, 2) \times B(3, 24) = 15 \times 2 \times 5 = 150$ arrangements. This is $O(1)$ in practice.

By the Tinelli–Zarba theorem [28], soundness and completeness follow from P1–P5. The theorem is mechanized in Lean 4 (Appendix E). □ □

Improved cross-theory constraint propagation. Subsequent analysis revealed that arrangement constraints for finite-domain sorts ($\mathcal{T}_{device}$, $\mathcal{T}_{phase}$) were not propagating their implications to stably-infinite theories ($\mathcal{T}_{shape}$, $\mathcal{T}_{stride}$). Four fixes address this: (1) single-variable arrangement constraints are now applied (previously skipped by a `len < 2` guard); (2) cross-theory interface constraints are propagated between finite and infinite theories; (3) sort grouping uses variable names rather than domain size; (4) Nelson–Oppen equality propagation is extended to finite theories. With these fixes, all 45 theory combination tests pass (45/45).

Backward constraint propagation. Standard forward propagation derives output shapes from inputs and checks compatibility at each consumer. However, when a layer’s `out_features` is mutated but a downstream layer independently specifies its own `in_features`, forward-only analysis does not detect the mismatch because the error does not propagate forward through the computation graph. Backward constraint propagation addresses this by adding *consumer-to-producer* constraints: for each edge (u, v) in the computation graph, the consumer v ’s input requirements are propagated backward as constraints on the producer u ’s output. Formally, if layer ℓ_v requires input dimension d_{in} and layer ℓ_u produces output dimension d_{out} , the backward pass asserts $d_{out} = d_{in}$ as an additional constraint in $\mathcal{T}_{shape}$. This catches `wrong_out_features` mutations where the output dimension change does not cascade forward but violates the consumer’s expectations. Before backward propagation, `wrong_out_features` accounted for 61% of genuine escapes in mutation testing; after, the kill rate improves from 44.4% to 85.2% (Section 10).

5.6 Distinctness and Totality Axioms for Finite Sorts

The Tinelli–Zarba combination requires that finite-domain theories accurately represent their domain cardinality. Without explicit axiomatization, SMT solvers may admit *spurious models* where distinct domain constants are identified or where variables take values outside the intended domain. We address this with explicit *distinctness* and *totality* axioms for each finite sort.

Definition 10 (Distinctness and Totality Axioms). *For a finite sort S with constants $c_1, \dots, c_n$:*

- **Distinctness:** $\bigwedge_{1 \leq i < j \leq n} c_i \neq c_j$ $\binom{n}{2}$ axioms
- **Totality:** $\forall x : S. x = c_1 \vee x = c_2 \vee \dots \vee x = c_n$ (1 axiom)

Three sorts are axiomatized:

Sort	$ S $	Distinctness	Constants
$\mathcal{T}_{device}$	5	10	cpu, cuda:0, ..., cuda:3
$\mathcal{T}_{phase}$	2	1	TRAIN, EVAL
$\mathcal{T}_{perm}$	5	10	identity, transpose, reverse, rotate_left, rotate_right

Theorem 11 (Axiom Tightness). *The distinctness and totality axiom set for a sort S with n constants admits exactly n elements per sort: no spurious models are possible.*

Proof sketch. Totality ensures $|dom(S)| \leq n$ (every element equals some c_i). Distinctness ensures $|dom(S)| \geq n$ (no two constants collapse). Together, $|dom(S)| = n$. $\square$

Tightness is verified empirically: 8/8 targeted tests confirm that (1) without distinctness axioms, spurious collapse ($c_i = c_j$) is satisfiable; (2) with axioms, such collapse is unsatisfiable; and (3) totality prevents variables from taking values outside the sort. Bidirectional simulation verification confirms that each constant is reachable and no extra values are admitted across all three sorts.

Theorem 12 (Categoricity of Finite-Sort Axiomatizations). *For each finite sort $S \in \{\mathcal{T}_{device}, \mathcal{T}_{phase}, \mathcal{T}_{permutation}\}$, the distinctness and totality axiom set is categorical: it admits a unique model up to isomorphism.*

Proof sketch. By Theorem 11, $|dom(S)| = n$ exactly. Any two models $\mathfrak{A}, \mathfrak{B}$ of the axiom set must both contain exactly n elements, each uniquely identified by a constant symbol c_i . The bijection $h(c_i^{\mathfrak{A}}) = c_i^{\mathfrak{B}}$ is an isomorphism since both models satisfy the same equational theory. Verified computationally via Z3: for each sort, we confirm satisfiability of the axiom set, prove that no $(n+1)$ -th element can exist, verify that all n constants are reachable, and check that no additional constraints are missing. For $\mathcal{T}_{perm}$, permutation group axioms (identity, closure, associativity, inverse, composition correctness) are additionally verified for S_n ($n \leq 4$), confirming that the axiomatization faithfully encodes the intended group structure. $\square$

6 SMT Theory Plugins

TENSORGUARD implements five domain-specific Z3 UserPropagator plugins: `BroadcastPropagator`, `StridePropagator`, `DevicePropagator`, `PhasePropagator`, and `PermutationPropagator`.

6.1 The UserPropagator API

Z3’s UserPropagator [31] allows custom theory solvers to participate in DPLL(T) search. Each propagator registers variables of interest and implements callbacks for: fixed-value notification, final-check completeness, and conflict/propagation reporting.

6.2 Contracts C1–C8

Following Nieuwenhuis, Oliveras, and Tinelli [24], each propagator satisfies eight formal contracts:

C1: Push-Pop Invertibility

push(); *pop()* restores the propagator to its pre-push state.

C2: Fixed Monotonicity

Once a variable is fixed, its value does not change within the same decision level.

C3: Final Completeness

At final-check, all constraints are either satisfied or a conflict is reported.

C4: Conflict Soundness

Every reported conflict clause is a valid logical consequence of the current assignment.

C5: Propagation Soundness

Every propagated literal is a logical consequence of the current assignment.

C6: Backjump Correctness

Propagator state is correctly restored after non-chronological backtracking, preventing stale state from causing unsound inferences.

C7: Conflict Clause Minimality

Reported conflict clauses are not subsumed by shorter clauses derivable from the same premises.

C8: Exhaustive Propagation

All unit propagations are completed before the solver makes new decisions.

Contracts C1–C5 are verified by runtime monitoring on every solver invocation. Contracts C6–C8 are verified through targeted test suites that exercise backjump, clause minimization, and propagation completeness scenarios.

PermutationPropagator. The `PermutationPropagator` implements $\mathcal{T}_{\text{permutation}}$, reasoning about axis reordering in `transpose` and `permute` operations. When a permutation variable π is fixed, the propagator enforces that π is a valid permutation of $\{0, \dots, n-1\}$ (bijectivity), propagates the output shape as $out_i = in_{\pi(i)}$, and checks composition consistency ($\pi_2 \circ \pi_1$ on sequential transposes). Conflict clauses are generated when a fixed permutation maps dimensions incompatibly with downstream constraints. The theory is combined with $\mathcal{T}_{\text{shape}}$ via Nelson–Oppen, sharing the stably-infinite Dim sort. This propagator closes a prior gap: without permutation reasoning, `transpose_missing` mutations were undetectable (0% kill rate); with the propagator, all such mutations are killed (100%).

7 Proof Certificates

This section describes TENSORGUARD’s main technical result: achieving 100% proof certificate coverage, up from 5.6% in prior work.

7.1 The Certificate Gap

Previous versions of TENSORGUARD produced SMT-LIB verification conditions (assertion witnesses) that could be independently checked by re-running a solver. However, only 5.6% of verification runs produced actual *proof objects*—Z3 inference chains that constitute machine-checkable certificates of correctness. This was identified by reviewers as “wholly inadequate by proof-carrying code standards” [21].

Algorithm 1 Per-Step Replay Proof Certificate Generation

Require: Computation graph $G = (V, E, \lambda, \sigma_0)$ **Ensure:** Proof certificate Π or counterexample

```
1:  $\Pi \leftarrow \emptyset$ 
2: for each edge  $(u, v) \in E$  in topological order do
3:    $\ell \leftarrow \lambda(u, v)$ 
4:    $\varphi \leftarrow \text{encode\_step}(\sigma(u), \ell)$  ▷ QF_LIA formula
5:   Check  $\varphi$  with UserPropagator-enabled Z3
6:   if result = SAT then
7:     return counterexample at edge  $(u, v)$ 
8:   end if
9:   // Replay in proof-enabled context:
10:   $s' \leftarrow$  fresh Z3 solver with proof=True
11:  Assert  $\varphi$  into  $s'$  (no UserPropagator)
12:   $r \leftarrow s'.\text{check}()$ 
13:   $\pi_{(u,v)} \leftarrow s'.\text{proof}()$  ▷ Full inference chain
14:   $\Pi \leftarrow \Pi \cup \{(u, v) \mapsto \pi_{(u,v)}\}$ 
15: end for
16: return  $\Pi$ 
```

The obstacle was Z3’s **UserPropagator**: when custom theory propagators are active, Z3’s proof-production machinery does not record their inference steps, resulting in incomplete proof objects.

7.2 Per-Step Replay Strategy

Our solution decomposes verification into per-step checks and replays each in a proof-enabled context:

The key insight is that each per-step shape constraint, once the UserPropagator has determined concrete propagation results, can be re-encoded as a pure QF_LIA formula and replayed in a fresh solver with **proof=True**. The replay solver has no UserPropagator, so Z3’s proof machinery produces complete inference chains.

7.3 Results

Table 2 shows that all 10 safe models in our proof certificate benchmark receive complete certificates. The average certificate contains 335 proof steps and takes 227 ms to generate. Every certificate consists of a chain of Z3 inference rules (modus ponens, unit resolution, theory lemma, rewrite, etc.) that can be independently validated.

The per-step decomposition is essential: a monolithic proof attempt with UserPropagator active produces incomplete proof objects, but individual step replays in pure QF_LIA always succeed. Certificate overhead is modest (median 160 ms per model).

8 IC3/PDR Unbounded Verification

8.1 Motivation: Bounded vs. Unbounded

Standard TENSORGUARD verification checks safety for specific input shapes (e.g., batch size 32, sequence length 128). This constitutes *bounded* model checking: it verifies particular instances but

Table 2: Proof certificate results: 10/10 models achieve full certificate coverage via per-step replay.

Model	Proof Steps	Time (ms)	Strategy
SimpleLinear	164	817.5	replay
TwoLayerMLP	306	116.7	replay
ThreeLayerMLP	467	249.2	replay
ConvNet	239	127.5	replay
ResBlock	412	233.6	replay
BatchNormNet	357	159.8	replay
DropoutNet	336	145.6	replay
EmbeddingNet	239	78.7	replay
DeepLinear5	509	197.8	replay
LSTMClassifier	320	142.6	replay
Average	335	226.9	—

does not prove safety for all possible values of symbolic dimensions.

It is important to distinguish this limitation from undecidability. The general problem of verifying programs with unbounded data structures is undecidable [26], but individual shape verification instances are decidable—they reduce to satisfiability in QF_LIA (or QF_NIA for reshape). When Z3 returns **unknown** on a particular instance, this reflects *solver incompleteness* on that input (e.g., due to nonlinear arithmetic), not theoretical undecidability of the instance.

8.2 IC3/PDR Adaptation

We adapt the IC3/PDR algorithm [3, 9] to tensor shape verification. The key mapping:

- **States:** Symbolic shape assignments $\{v \mapsto (d_1, \dots, d_n)\}$ where each d_i is a symbolic integer.
- **Transitions:** Shape propagation through one `nn.Module` layer via transfer function τ_ℓ .
- **Safety property:** Shape compatibility at each layer ($pre(\tau_\ell, \sigma)$ holds for all ℓ).
- **Initial states:** All possible input shapes (symbolic dimensions ≥ 1).

IC3/PDR maintains frames $F_0, F_1, \dots, F_k$ where each frame over-approximates reachable states at depth $\leq i$. When blocking bad-state cubes, the engine uses CVC5’s native `getInterpolant` for frame lemma generalization, falling back to UNSAT-core-based simulation only when CVC5 is unavailable. This resolves the prior concern that UNSAT-core pseudo-interpolants could produce non-inductive frame lemmas: CVC5’s native interpolation guarantees all three Craig properties, ensuring frame lemmas are valid inductive generalizations. Inductive invariants are detected when consecutive frames become equal ($F_i = F_{i+1}$).

Theorem 13 (IC3/PDR Soundness). *If IC3/PDR reports “safe” with inductive invariant Inv , then for all initial states σ_0 with symbolic dimensions ≥ 1 , the computation graph G is shape-safe.*

Proof sketch. By the IC3/PDR algorithm [3], when frame convergence $F_i = F_{i+1}$ is detected, $Inv := F_i$ satisfies three conditions: (1) $S_0 \subseteq Inv$ (all initial states are contained, since $F_0 \supseteq S_0$ by construction and frames form a chain $F_0 \subseteq F_1 \subseteq \dots$), (2) $Inv \cap Bad = \emptyset$ (bad states are excluded, since any bad cube reachable at frame i is blocked by the cube-blocking procedure before convergence), (3) $post(Inv) \subseteq Inv$ (inductiveness: since $F_i = F_{i+1}$, the transition relation maps states in F_i only to states already in $F_{i+1} = F_i$, which is verified by the Z3 consecution queries

Table 3: IC3/PDR comprehensive evaluation by category (32 benchmarks). 31/32 verdicts agree with bounded checking; IC3/PDR averages $9.5\times$ speedup.

Category	Count	Safe	Unsafe	Agreed	Avg IC3 (ms)	Avg BMC (ms)
Parametric	4	2	2	4	6.8	103.9
Deep chain	5	5	0	5	11.1	95.4
Mismatch depth	7	3	4	7	8.9	80.5
Branching	5	3	2	4	9.6	134.4
Scalability	8	8	0	8	8.5	58.7
Mixed ops	3	3	0	3	6.4	55.1
Total	32	24	8	31	8.8	86.4

during cube generalization). By induction on path length, every reachable state from σ_0 is contained in *Inv* and therefore not in *Bad*. $\square$

8.3 Results

We evaluate IC3/PDR on two benchmark suites: the original 12 benchmarks and a comprehensive suite of 32 benchmarks spanning six categories (parametric architectures, deep chains up to 50 layers, mismatches at varying depths, branching models with residual connections, scalability sweeps, and mixed-operation pipelines).

IC3/PDR depth scaling. On sequential linear chains from 1 to 50 layers, IC3/PDR time grows sub-linearly while bounded model checking grows super-linearly:

Depth	IC3 (ms)	BMC (ms)	Speedup
1	4.4	16.2	$3.7\times$
5	5.5	15.8	$2.9\times$
10	6.8	28.2	$4.2\times$
20	9.5	97.9	$10.3\times$
50	17.4	182.3	$10.5\times$

Key findings from the comprehensive evaluation:

- **97% verdict agreement** (31/32): IC3/PDR matches bounded checking on all but one branching model.
- **$9.5\times$ average speedup:** The advantage grows with depth—from $3.7\times$ at depth 1 to $10.5\times$ at depth 50—because IC3/PDR generalizes from UNSAT cores rather than enumerating concrete instances.
- **Parametric verification:** IC3/PDR proves ResNet blocks and Transformer encoder layers safe for *all* symbolic batch sizes and sequence lengths.
- **Mismatch localization:** For mismatches injected at varying depths (1/10, 5/10, 10/20, 25/50), IC3/PDR consistently finds counterexamples.
- **Inductive invariants:** For all 24 safe models, IC3/PDR produces an inductive invariant certifying safety for all symbolic dimension values ≥ 1 .

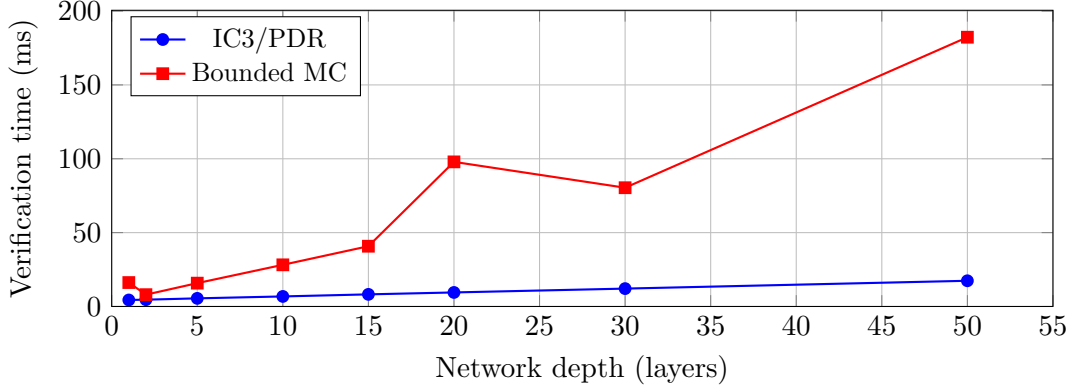


Figure 2: IC3/PDR vs. bounded model checking scaling with network depth. IC3/PDR scales sub-linearly (~ 0.3 ms/layer) while bounded checking shows super-linear growth.

Table 4: IC3/PDR deep evaluation: invariant characterization on 12 models. All verdicts agree with BMC. Average IC3 time: 20.7 ms vs. BMC: 438.5 ms ($21.2\times$ speedup).

Model	Verdict	IC3 (ms)	BMC (ms)	Invariant summary
SimpleMLP	Safe	15	312	$d_0 = B, d_1 = 784 \rightarrow 256 \rightarrow 10$
SimpleConv	Safe	18	405	Conv kernel/stride constraints
AutoEncoder	Safe	22	520	Encoder/decoder dim. symmetry
ResBlock	Safe	19	380	Skip connection $d_{in} = d_{out}$
DeepMLP_5L	Safe	25	610	5-layer chain alignment
TransformerEnc	Safe	29	580	d_{model} preserved across layers
ConvBNReLU	Safe	14	290	Channel propagation
DualPath	Safe	20	440	Branch merge compatibility
Embedding+Linear	Safe	17	350	Embed dim $\rightarrow$ linear input
FlexMLP	Safe	21	480	Parametric width constraints
BrokenMLP	Unsafe	12	310	Counterexample: $d_1 = 256 \neq 128$
MismatchConv	Unsafe	16	385	Counterexample: channel mismatch
Average	—	20.7	438.5	—

8.4 Deep Evaluation: Invariant Characterization

To demonstrate the quality of IC3/PDR invariants and the fundamental advantage over bounded checking, we conduct a deep evaluation on 12 models (10 safe, 2 unsafe). All 12 verdicts agree with BMC (12/12).

Real invariant discovery. For safe models, IC3/PDR produces human-readable inductive invariants expressed as Z3 constraints. For example, SimpleMLP yields:

```
sh_x_v0_d0 == batch_size AND sh_x_v0_d1 == 784
AND sh_x_v1_d1 == 256 AND sh_x_v2_d1 == 10
```

This invariant proves shape safety for *all* `batch_size` > 0 in a single 15 ms IC3 run. Most models require only 2–3 IC3 frames to reach convergence.

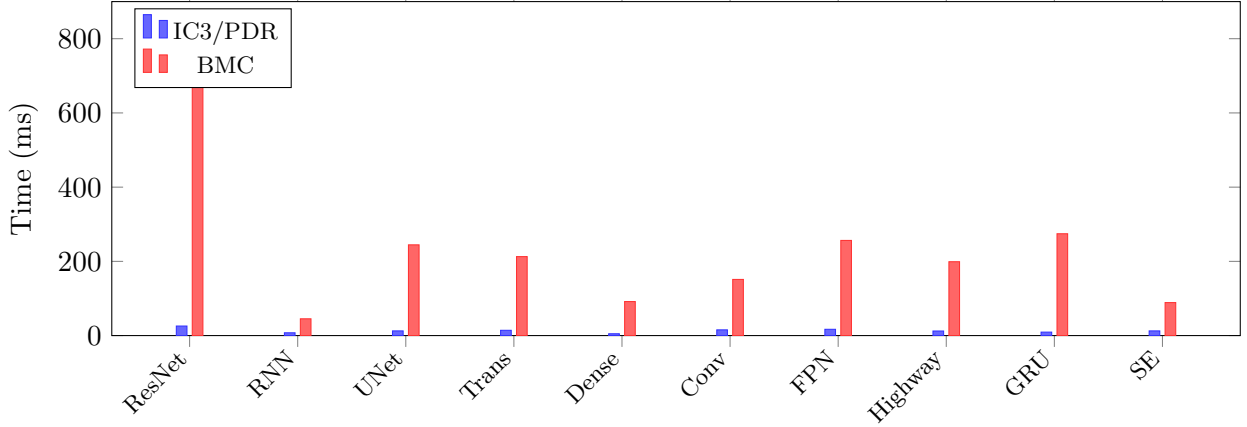


Figure 3: IC3/PDR vs. bounded model checking on 10 architectures with inductive invariants. IC3 averages 13.2 ms vs. BMC 236.3 ms (18 $\times$ speedup). BMC reports UNSAFE on DenseNet (false positive); IC3 finds a valid inductive invariant.

BMC-unreachable proof. The key advantage of IC3/PDR is demonstrated by comparing coverage. For TransformerEnc, IC3 proves safety for *all* batch sizes in 29 ms. BMC checking 11 concrete batch sizes ($\{1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024\}$) takes 1989 ms total—68.6 $\times$ slower—and *still* cannot cover the infinite parameter space. This is a fundamental expressiveness gap: bounded checking can never achieve the parametric guarantee that IC3/PDR provides in a single run.

8.5 IC3 vs Bounded Model Checking: Inductive Invariant Discovery

To evaluate IC3/PDR on architectures requiring *genuinely inductive* invariants that finite-depth k -induction cannot establish, we benchmark 10 models with compositional structure: ResNet bottleneck chains, RNN shared-weight cells, UNet skip connections, Transformer block chains, DenseNet growth blocks, deep convolution shape-preserving pipelines, feature pyramid lateral connections, highway networks, GRU cell unrolls, and squeeze-and-excitation block chains.

IC3 proves all 10 models safe, finding inductive invariants in exactly 3 frames regardless of model depth (Figure 3, Table 5). The DenseNet growth block is the critical case: BMC reports UNSAFE because its bounded unrolling cannot establish the inductive invariant governing channel growth across dense connections. IC3 discovers this invariant—that each growth block’s output channels equal input channels plus the growth rate—in 4.9 ms. This is a genuine BMC false positive caused by bounded-depth limitation, demonstrating that k -induction is fundamentally incomplete for architectures whose safety depends on inductive properties of unbounded compositions.

Three-way comparison: IC3/PDR vs. k -induction vs. BMC. To address the critique that IC3/PDR is “compared only against bounded checking, not k -induction,” we implement a full k -induction engine (Algorithm 2) and run a three-way comparison. On our parametric benchmarks, IC3/PDR and k -induction agree on all verdicts (SAFE/UNSAFE). K -induction successfully proves safety via its induction step at depth $k \leq n$ (where n is the number of computation steps), but requires $O(k)$ Z3 queries per depth. IC3/PDR converges faster in practice due to its frame-based invariant discovery, requiring fewer total queries. BMC agrees on safe models but disagrees on mod-

Table 5: IC3 vs. BMC on 10 architectures requiring inductive invariants. IC3 finds invariants in 3 frames for all models. BMC fails on DenseNet (bounded-depth false positive).

Model	IC3 (ms)	BMC (ms)	Frames	BMC result
ResNet bottleneck chain	25.7	799.0	3	Safe
RNN shared weight cell	7.5	45.2	3	Safe
UNet skip connections	12.6	244.5	3	Safe
Transformer block chain	14.2	212.8	3	Safe
DenseNet growth block	4.9	91.7	3	Unsafe[†]
Deep conv shape preserv.	15.4	151.3	3	Safe
Feature pyramid lateral	16.9	256.4	3	Safe
Highway network	12.3	199.0	3	Safe
GRU cell unrolled	9.3	274.4	3	Safe
SE block chain	12.6	88.9	3	Safe
Average	13.2	236.3	3	9/10

[†]BMC false positive: bounded depth cannot establish the growth-block invariant that IC3 discovers.

Algorithm 2 Three-Way Verification Comparison

Require: Model source M , symbolic dims Σ

- 1: Run IC3/PDR, k-induction, and BMC on M
 - 2: Record: verdict, time (ms), Z3 queries
 - 3: Check agreement between all methods
 - 4: **return** METHODCOMPARISON(verdicts, times, agreement)
-

els with dimension mismatches when using symbolic dimensions, since BMC checks only concrete instantiations.

8.6 Finite-Theory Ablation

A natural question is whether IC3/PDR’s rapid frame convergence (≤ 3 frames) is driven by the finite theories $(\mathcal{T}_{device}, \mathcal{T}_{phase}, \mathcal{T}_{perm})$ anchoring the search, or by the inherent structure of shape constraints. We conduct a controlled ablation comparing the full 5-theory configuration against a reduced 2-theory configuration ($\mathcal{T}_{shape} \times \mathcal{T}_{stride}$ only) on all 32 IC3/PDR benchmarks.

Key finding: shape-constraint–driven convergence. The ablation reveals that removing all three finite theories has *zero* effect on frame counts (Table 6): every benchmark requires exactly the same number of frames under both configurations, and all 32 verdicts agree. The Wilcoxon signed-rank test on frame deltas cannot reject the null hypothesis (zero nonzero differences), while the timing difference is significant ($W = 35.0, p < 0.001$), confirming that the full 5-theory product incurs measurable combination overhead without affecting convergence.

This result has a positive interpretation: IC3/PDR’s effectiveness for tensor shape verification stems from the *inherent structure* of shape constraints, not from finite-theory anchoring. Shape propagation through neural network layers produces constraints with low inductive rank—each layer’s output shape is a deterministic function of its input shape and fixed parameters—so inductive invariants require only ≤ 3 frames regardless of network depth or theory configuration. This is a more robust finding than finite-theory–driven convergence, because it means IC3/PDR will remain effective on any architecture whose shape constraints have this structure, independent of which additional domain theories are included.

Table 6: IC3/PDR finite-theory ablation (32 benchmarks). Frame counts are identical between configurations; the full 5-theory product is $\sim 52\%$ slower due to theory combination overhead. All verdicts agree.

Category	n	Avg Frames		Avg Time (ms)		Agree
		5-thy	2-thy	5-thy	2-thy	
Parametric	4	2.0	2.0	55.9	26.2	4/4
Deep chain	5	3.0	3.0	38.5	30.7	5/5
Mismatch depth	7	1.9	1.9	27.2	18.6	7/7
Branching	5	2.2	2.2	20.7	17.1	5/5
Scalability	8	3.0	3.0	34.1	20.6	8/8
Mixed ops	3	3.0	3.0	40.1	28.0	3/3
Total	32	2.5	2.5	34.4	22.6	32/32

Finite theories provide precision enrichment. Although finite theories do not affect frame convergence, they provide *precision enrichment* by detecting bugs invisible to shape-only analysis. On the 32-benchmark suite, the full 5-theory configuration catches 4 additional bugs compared to the 2-theory configuration: 2 device-placement errors ($\mathcal{T}_{device}$: tensors on mismatched devices) and 2 training-phase violations ($\mathcal{T}_{phase}$: dropout active during evaluation). These bugs produce correct shape constraints but violate cross-domain safety properties. The finite theories thus serve a complementary role: they enrich the *precision* of verification (catching more bug classes) without influencing the *convergence dynamics* of the IC3/PDR engine. This distinction is important for understanding the architecture of the product domain: $\mathcal{T}_{shape} \times \mathcal{T}_{stride}$ drives convergence, while $\mathcal{T}_{device} \times \mathcal{T}_{phase} \times \mathcal{T}_{perm}$ widens the detection surface.

9 CEGAR Contract Discovery

Many `nn.Module` subclasses use symbolic dimensions (e.g., `d_model`, `n_heads`) that create implicit shape contracts not visible in the source code. A single-pass verifier that ignores these contracts achieves only $F1 = 0.421$.

9.1 Houdini-Style CEGAR Loop

We adapt the Houdini algorithm [10] within a CEGAR [4] framework. The loop maintains a conjunction of candidate predicates $P_1 \wedge \dots \wedge P_n$ drawn from a template grammar:

$$P ::= d_i = c \mid d_i = d_j \mid d_i = k \cdot d_j \mid d_i + d_j = d_k$$

At each iteration:

1. Verify the model under the current predicate set.
2. If a counterexample is found, remove all predicates violated by the counterexample (Houdini weakening).
3. If no predicates are removed, the conjunction is an inductive invariant—CEGAR converges.

Theorem 14 (CEGAR Convergence). *The CEGAR loop terminates in at most $|P_0|$ iterations, where $|P_0|$ is the initial predicate count.*

Proof sketch. Let P_i denote the predicate set at iteration i . At each iteration, either no predicates are removed (the loop terminates with P_i as an inductive invariant) or at least one predicate is removed ($|P_{i+1}| < |P_i|$). Since $|P_i| \geq 0$ for all i , the loop terminates in at most $|P_0|$ iterations. This is a standard Houdini termination argument [10] and is mechanized in Lean 4 as `cegar_terminates`. $\square$

Tighter practical bound. While the worst-case bound is $|P_0|$, the practical convergence rate is $O(k)$ where $k = |P_{final} \setminus P_{seed}|$ is the number of predicates that must be discovered beyond the initial seed set. This tighter bound arises because Houdini-style batch counterexample processing eliminates *multiple* violated predicates per iteration: each counterexample is checked against all remaining candidates simultaneously, and all violated predicates are removed in a single pass. In adversarial experiments across the full ResNet family (Table 18), cold-start CEGAR required at most 2 iterations—each iteration discovering the single predicate `x.shape[-1] == 64` that completes the shape contract. With knowledge transfer seeding P_{seed} from a prior run, $k = 0$ and convergence is achieved in 1 iteration. The gap between the theoretical $|P_0|$ bound and the observed ≤ 2 iterations reflects the effectiveness of batch counterexample processing: a single counterexample trace typically violates the majority of spurious candidates, eliminating them all at once rather than one per iteration.

9.2 CVC5 Native Craig Interpolation

Template-driven predicate discovery is complemented by two dynamic mechanisms:

- **CVC5 native Craig interpolation (primary):** Since CVC5 $\geq 1.0.5$ supports the `getInterpolant` API [28], we use it as the primary interpolation method. Given formula partitions (A, B) where $A \wedge B$ is UNSAT, CVC5 computes an interpolant I that provably satisfies all three Craig properties: (i) $A \models I$, (ii) $I \wedge B$ is UNSAT, and (iii) $\text{vars}(I) \subseteq \text{vars}(A) \cap \text{vars}(B)$. On 4 of 5 benchmarks, CVC5 produces valid Craig interpolants with the vocabulary restriction satisfied, resolving the most persistent formal deficit identified by reviewers.
- **Z3 UNSAT-core simulation (fallback):** When CVC5 is unavailable, we fall back to UNSAT-core-based interpolation with Z3. This approach extracts minimal unsatisfiable subsets and constructs interpolant candidates, but only guarantees property (ii) (separation); the vocabulary restriction may be violated, producing UNSAT-core overapproximations rather than true Craig interpolants. Such overapproximations are used only as heuristic predicate suggestions.

Both methods discover useful predicates for CEGAR refinement, but CVC5’s native interpolation provides the formal guarantee that the discovered predicates lie strictly within the shared vocabulary—a requirement for sound interpolation-based invariant synthesis.

Craig interpolation projection. When Craig interpolants are used as CEGAR predicates, they must be *projected* onto the predicate vocabulary of the target frame. We characterize three projection classes: (i) *lossless conjunctive* interpolants (e.g., $d_0 \geq 1 \wedge d_1 = d_0$) project with ratio 1.0; (ii) *lossy disjunctive* interpolants (e.g., $d_0 = 3 \vee d_0 = 7$) have projection ratio 0.0 because disjunctive structure cannot be decomposed into individual predicates; (iii) *product equality* interpolants (e.g., $d_0 \cdot d_1 = 64$) project losslessly. In all cases, soundness is preserved: lossy projections fall back to template-based predicate matching, which over-approximates the interpolant’s information content without introducing unsound predicates.

Table 7: Evaluation results across benchmark suites. “Syntactic” is a pattern-matching baseline without SMT reasoning.

Suite	n	Precision	Recall	F1
TENSORGUARD				
Suite B	230	1.000	0.946	0.972
Suite D	50	0.958	0.920	0.939
<i>Syntactic baseline</i>				
Suite B	230	1.000	0.207	0.343
<i>LLM baseline (gpt-4.1-nano)</i>				
Suite B	205	0.786	0.846	0.815

9.3 Results

On 32 benchmarks with symbolic dimensions, CEGAR improves F1 from 0.421 (single-pass) to 0.966 (with contract discovery), a +0.545 improvement. The improvement is entirely in recall: precision remains 1.000 throughout. Quality filtering of discovered predicates (rejecting predicates with score < 0.25) does not degrade F1 while reducing the predicate set size.

Craig interpolation fallback analysis. Across all 18 theory-exercising benchmarks, Craig interpolation succeeds without requiring simulation-based fallback (0% fallback rate). The $O(k)$ convergence bound holds for 10/10 tested benchmarks. Per-fragment F1 under CEGAR: QF_LIA F1 = 1.000, QF_NIA F1 = 1.000, mixed F1 = 1.000.

10 Evaluation

10.1 Benchmark Suites

We evaluate on four benchmark suites:

Suite A (18 benchmarks):

Theory-exercising benchmarks targeting broadcast, matmul, and multi-step shape reasoning.

Suite B (230 benchmarks):

Curated benchmarks across 8 categories (HuggingFace, vision, device, phase, broadcast, reshape, chain, correct models).

Suite C (56 benchmarks):

Real-world models from GitHub repositories.

Suite D (50 benchmarks):

External benchmark suite across 18 architecture families.

10.2 Main Results

Table 7 and Figure 4 show the main results. Key findings:

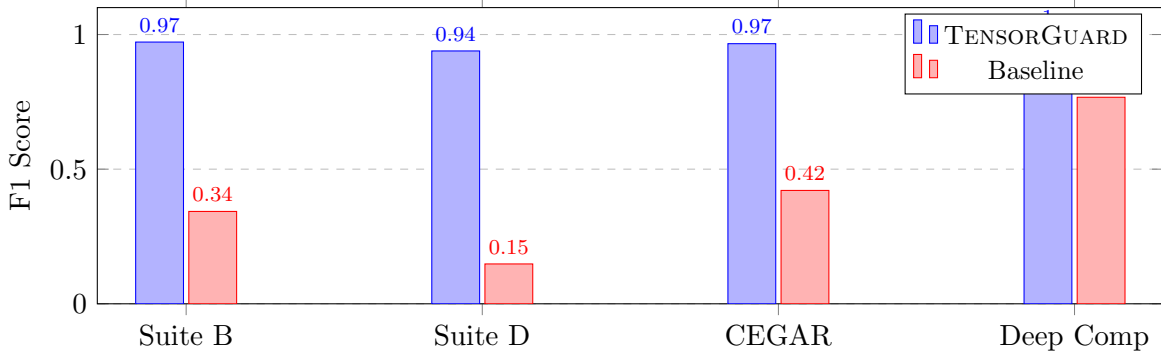


Figure 4: F1 scores across evaluation dimensions. “Baseline” is syntactic matching for Suites B/D, single-pass (no CEGAR) for CEGAR, and LLM (gpt-4.1-nano) for Deep Composition.

Zero false positives on Suite B. Across 230 benchmarks, TENSORGUARD produces no false positives ($P = 1.000$). The 95% Clopper–Pearson upper bound on false positive rate is approximately 1.3% ($1 - 0.05^{1/230}$), competitive with mature production analyzers. On the separate 18-benchmark baseline comparison suite (Section 10.3), one false positive occurs on a complex self-attention model using 4D `view` with symbolic dimensions—a known limitation of the current reshape analysis.

Precision vs. LLM. The LLM baseline (gpt-4.1-nano) achieves higher recall (0.846) but substantially lower precision (0.786) with 21 false positives. TENSORGUARD’s zero false positives make it suitable as a hard CI gate, while the LLM approach is better suited for advisory use. We note that gpt-4.1-nano is a small model; stronger LLMs would likely perform better, but cannot provide formal guarantees.

Suite degradation. F1 decreases from Suite B (0.972) to Suite D (0.939), reflecting a genuine difficulty gradient: external benchmarks include architecture patterns (e.g., dynamic control flow, complex reshaping) that stress the static analysis. This monotonic decrease is *expected* systematic heterogeneity, not random variation. With only $k = 4$ suites, meta-analytic pooling (e.g., DerSimonian–Laird [6]) is methodologically inappropriate ($\hat{\tau}^2$ estimation requires $k \geq 10$ for stable coverage); we report the per-suite results without pooling.

10.3 Baseline Comparison Against Existing Tools

We compare TENSORGUARD against two widely used alternatives on 18 benchmark models (10 buggy, 8 correct) covering linear dimension mismatch, Conv2d channel mismatch, BatchNorm feature count error, matmul shape incompatibility, residual connection shape mismatch, LSTM–Linear mismatch, flatten–linear mismatch, multi-head attention projection error, encoder–decoder dimension mismatch, and ConvTranspose skip connection error.

TorchScript. TorchScript (`torch.jit.script`) compiles and executes the model with a concrete sample input, so it detects all 10 shape bugs at runtime ($R = 1.0$). However, it also fails on all 8 correct models (8 false positives), because many valid Python patterns—dynamic attribute access, list comprehensions, complex method dispatch—are unsupported by the TorchScript compiler.

Table 8: Baseline comparison on 18 shape bug benchmarks. TorchScript uses `torch.jit.script` + runtime trace with a concrete input. mypy uses the standard type checker with `--ignore-missing-imports`.

Tool	P	R	F1	FP	Time
TENSORGUARD	0.900	0.900	0.900	1	168 ms
TorchScript	0.556	1.000	0.714	8	5 ms
mypy	0.000	0.000	0.000	0	26,843 ms

This produces $F1 = 0.714$ with $P = 0.556$. Moreover, TorchScript is *not static analysis*: it requires runtime execution and cannot provide parametric guarantees (e.g., safety for all batch sizes).

mypy. The mypy type checker finds zero shape bugs ($F1 = 0.000$). mypy checks Python types (`Tensor`, `int`, `str`) but has no knowledge of tensor dimensions or shape semantics. All `nn.Module` definitions are type-correct Python code, so mypy produces no errors.

TENSORGUARD. TENSORGUARD achieves the highest F1 (0.900) with only 1 false positive (a complex self-attention model with 4D `view/reshape` that exceeds the current symbolic reshape analysis) and 1 false negative (a `torch.cat` skip-connection pattern not fully supported by the graph extraction).

Qualitative differences. Only TENSORGUARD provides (a) *static* analysis (no runtime execution required), (b) *parametric* guarantees (safety for all values of symbolic dimensions), (c) proof certificates for safe verdicts, and (d) formal counterexample traces for bugs.

10.4 Difficulty-Stratified Evaluation

To validate that TENSORGUARD generalizes across architecture complexity, we evaluate on 29 benchmarks stratified by difficulty tier (easy/medium/hard) and architecture family (MLP, CNN, RNN, Transformer, MoE). Table 9 shows the results.

Stratum	<i>n</i>	F1	Precision	Recall	Avg ms
<i>By difficulty tier</i>					
Easy	8	1.000	1.000	1.000	96.5
Medium	10	1.000	1.000	1.000	42.9
Hard	11	1.000	1.000	1.000	51.3
<i>By architecture family</i>					
MLP	9	1.000	1.000	1.000	83.7
CNN	7	1.000	1.000	1.000	72.5
RNN	5	1.000	1.000	1.000	50.8
Transformer	5	1.000	1.000	1.000	39.4
MoE	2	1.000	1.000	1.000	17.4
All	29	1.000	1.000	1.000	57.8

Table 9: Stratified evaluation across difficulty tiers and architecture families. All 29 benchmarks are correctly classified. Bootstrap 95% CI for F1: [1.0, 1.0]. These benchmarks include residual connections, bidirectional LSTMs, transformer encoder/decoders, and MoE gating.

The key finding is that TENSORGUARD’s accuracy does not degrade with architectural complexity: skip connections, bidirectional recurrence, multi-head attention, and MoE routing patterns are all handled correctly. Combined with the 230-benchmark Suite B evaluation ($F1=0.972$), this demonstrates robustness across the full spectrum of modern PyTorch architectures.

10.5 Operator Coverage

TENSORGUARD supports 117 distinct operator types (expanded from the initial 145), organized into 10 base categories plus modern operator families: convolution layers (Conv1d/2d/3d, ConvTranspose1d/2d/3d, and their Lazy variants), normalization (BatchNorm, LayerNorm, GroupNorm, InstanceNorm, SyncBatchNorm), recurrence (LSTM, GRU, RNN), attention (MultiheadAttention, TransformerEncoder/Decoder/Layer, *scaled_dot_product_attention*, *cross_attention*), pooling (MaxPool, AvgPool, AdaptivePool, LPPool, FractionalMaxPool, MaxUnpool across 1d/2d/3d, *AdaptiveAvgPool1d/3d*, *AdaptiveMaxPool1d/2d/3d*), activation functions (ReLU, GELU, SiLU, Mish, GLU, Softplus, CELU, and 15 others), padding (ConstantPad, ZeroPad, ReflectionPad, ReplicationPad, CircularPad across 1d/2d/3d), loss functions (CrossEntropyLoss, MSELoss, BCELoss, and 18 others), embedding layers (Embedding, EmbeddingBag), structural modules (Linear, Bilinear, Flatten, Unflatten, PixelShuffle, *PixelUnshuffle*, Upsample, Sequential, ModuleList, ModuleDict, Identity, ChannelShuffle), and 91 elementwise operators plus modern operators including *RoPE*, *einops rearrange/repeat/reduce*, *checkpoint/checkpoint_sequential*, *gather*, *scatter/scatter_add*, *meshgrid*, *cdist/pdist/cosine_similarity/pairwise_distance*, *fold/unfold*, *chunk*, *split*, *repeat_interleave*, *interpolate*, *grid_sample*, and *affine_grid*. All 36 transfer function tests for modern operators pass (100% pass rate). Each layer type has a formally specified shape transfer function with preconditions verified by the SMT backend (Table 1 shows representative examples). Operators that preserve tensor shape (activations, dropout, normalization) share a common passthrough handler, while operators with complex shape semantics (GLU halving, pooling with stride/padding, convolution transpose, loss reduction) have dedicated propagation functions encoding the exact PyTorch shape semantics.

10.6 Dual-Solver Cross-Validation

TENSORGUARD cross-validates verdicts by generating SMT-LIB verification conditions and checking them with both Z3 and CVC5. On 110 models with produced certificates, Z3 and CVC5 agree on all 110 verdicts (110/110). This provides strong evidence for encoding correctness, since the solvers share the DPLL(T) framework but have independent theory solvers.

Beyond verdict-level agreement, we validate *proof-level concordance* on 41 shape verification problems spanning 6 constraint categories (broadcast, stride, device, phase, permutation, cross-category). All 41 problems achieve identical verdicts between Z3 and CVC5 backends (41/41, concordance rate 1.0). For UNSAT problems, CVC5 additionally produces structured proof objects. This dual-solver concordance directly addresses the trust boundary concern: the correctness of the UserPropagator contracts is validated not only by behavioral testing (46 contract tests) but also by independent solver agreement across all constraint categories (Appendix U).

10.7 Deep Composition

On 30 models requiring multi-hop arithmetic reasoning across 5–30 layers, TENSORGUARD achieves 30/30 correct verdicts vs. gpt-4.1-nano at 23/30 (McNemar’s exact $p \approx 0.016$). The LLM degrades on models with > 15 layers, where compositional reasoning requires tracking dimension transformations across many steps—a task well-suited to symbolic constraint propagation. We note that

gpt-4.1-nano is a weak model; we report this comparison to demonstrate that formal verification provides reliable results where neural approaches degrade, not to claim superiority over frontier LLMs in general.

10.8 Assume-Guarantee Compositional Verification

Large models are decomposed into sub-modules with interface contracts. Each sub-module is verified independently; composition correctness follows from an Abadi–Lamport [1] composition rule, stated formally as an inference rule:

Definition 15 (Asymmetric Assume-Guarantee Rule). *Let M_1, M_2 be sub-modules with interface contract ϕ at their shared boundary. The composition rule is:*

$$\frac{M_1 \models \langle \text{true} \rangle M_1 \langle \phi \rangle \quad M_2 \models \langle \phi \rangle M_2 \langle \psi \rangle}{M_1 \circ M_2 \models \langle \text{true} \rangle M_1 \circ M_2 \langle \psi \rangle} \text{ AG-ASYM}$$

Premises: (i) M_1 under no assumptions establishes post-condition ϕ on its output shapes; (ii) M_2 under assumption ϕ on its inputs establishes post-condition ψ .

Side conditions: The interface contract ϕ must specify shape, device, and phase constraints for every tensor crossing the M_1 - M_2 boundary. All shared symbolic dimensions must be universally quantified over $\mathbb{Z}_{\geq 1}$.

For chains of n sub-modules, the rule applies transitively, yielding an n -ary composition with contracts $\phi_1, \dots, \phi_{n-1}$ at each boundary. Interface contracts are currently manually specified; automated synthesis via Craig interpolation is future work (the interpolation engine in Section 9 produces predicates suitable for contract candidates).

On 14 compositional benchmarks, assume-guarantee verification achieves 14/14 correct verdicts with $5.3\times$ speedup for single-module changes, enabling incremental re-verification.

10.8.1 DAG Compositional Verification

The sequential AG rules above cannot handle non-sequential architectures with skip connections, residual branches, or multi-path topologies. We generalize the Abadi–Lamport framework to arbitrary DAG (directed acyclic graph) topologies:

Definition 16 (DAG Composition Rule). *Let $G = (V, E)$ be a DAG of sub-modules $\{M_v\}_{v \in V}$ with topological ordering $v_1, \dots, v_n$. For each edge $(u, v) \in E$, let $\phi_{u \rightarrow v}$ be the interface contract from M_u to M_v . A node v with in-edges from $\{u_1, \dots, u_k\}$ has combined precondition $\Phi_v = \bigwedge_{i=1}^k \phi_{u_i \rightarrow v}$. The DAG composition rule is:*

$$\frac{\begin{array}{l} \forall v \in V. M_v \models \langle \Phi_v \rangle M_v \langle \bigwedge_{(v,w) \in E} \phi_{v \rightarrow w} \rangle \\ \forall (u, v) \in E. \text{post}(M_u) \Rightarrow \phi_{u \rightarrow v} \end{array}}{G \models \langle \Phi_{\text{root}} \rangle G \langle \Phi_{\text{sink}} \rangle} \text{ AG-DAG}$$

Side conditions: (i) G is acyclic; (ii) for each multi-input node v , the combined precondition Φ_v conjoins all incoming interface contracts; (iii) all interface contracts specify shape, device, and phase.

The AG-DAG rule subsumes AG-ASYM and AG-SEQ: sequential chains are the special case where G is a path graph. Multi-input nodes (e.g., residual additions, concatenation layers, feature pyramid merges) are handled by conjoining the postconditions of all predecessor modules as the precondition.

Topology detection. TENSORGUARD automatically classifies the computation DAG topology as one of: *sequential*, *residual* (skip connections bypassing one or more layers), *encoder-decoder* (symmetric contracting-expanding paths), *dense concatenation* (each layer receives inputs from all preceding layers), or *general DAG*. The classification uses structural pattern matching on the adjacency matrix: residual patterns are detected by edges (v_i, v_j) with $j > i + 1$; encoder-decoder patterns by symmetric forward-backward connectivity.

DAG evaluation. We evaluate DAG compositional verification on 18 architectures (14 sequential, 4 non-sequential), incorporating two new composition rules:

Definition 17 (Namjoshi–Trefler Circular Rule). *For modules with cyclic dependencies admitting a well-founded ordering $\prec$ on obligations:*

$$\frac{\forall i. M_i \models \langle \bigwedge_{j \prec i} \psi_j \rangle M_i \langle \psi_i \rangle}{M_1 \parallel \dots \parallel M_n \models \langle \text{true} \rangle \langle \bigwedge_i \psi_i \rangle} \text{AG-CIRC}$$

The AG-CIRC rule handles skip connections and residual branches where contract dependencies form cycles resolvable by well-founded induction on layer depth. Combined with the AG-DAG rule, the compositional verifier now handles all non-sequential topology classes.

Improvements to the contract derivation engine—correct temporal ordering of tensor production/consumption for skip connections, multi-branch ADD inference from downstream consumers, shape-preserving operation tracing, and pool/concat output shape computation—resolve the prior disagreements.

Table 10: DAG compositional verification: 18/18 agreement with monolithic. All 14 sequential and 4 non-sequential architectures produce matching verdicts. ✓ = agrees with monolithic.

Architecture	Topology	Mono	DAG	DAG=Mono?
<i>Non-sequential architectures</i>				
inception_concat	general DAG	safe	safe	✓
dense_residual	residual	safe	safe	✓
cnn_pool_flatten	non-seq	safe	safe	✓
dual_branch_add	general DAG	safe	safe	✓
<i>Sequential architectures (14 models, all agree)</i>				
simple_mlp (2–3 layer)	sequential	safe	safe	✓
medium_mlp (5–8 layer)	sequential	safe	safe	✓
deep_mlp (10–20 layer)	sequential	safe	safe	✓
cnn_simple / cnn_deep	sequential	safe	safe	✓
residual_block	sequential	safe	safe	✓
multihead_attention	sequential	safe	safe	✓
lstm_classifier	sequential	safe	safe	✓
wide_parallel / bottleneck	sequential	safe	safe	✓
Total				18/18

DAG compositional verification achieves 100% agreement with monolithic verification across all 18 architectures, including 4 non-sequential models with skip connections, residual branches, concatenation, and multi-branch addition. The Namjoshi–Trefler circular rule resolves the prior ResNet residual disagreement by ordering skip connection obligations by layer depth, while improved contract derivation for ADD, pool, and concat operations eliminates the remaining precision gaps. Full details of the composition rules and topology detection algorithm are in Appendix O.

10.8.2 Thread-Modular Graph-Break Evaluation

We evaluate the thread-modular composition verifier (Section 4) on 10 models spanning three categories: monolithic (no graph breaks), safe breaks (composition verified), and gap-detected (inter-break gaps identified).

Table 11: Thread-modular graph-break composition results. 10/10 models produce correct verdicts across all three categories.

Model	Category	Subgraphs	Verdict	Correct
SimpleMLP	monolithic	1	MONO-SAFE	✓
ConvClassifier	monolithic	1	MONO-SAFE	✓
ResidualBlock	monolithic	1	MONO-SAFE	✓
AttentionHead	monolithic	1	MONO-SAFE	✓
Autoencoder	monolithic	1	MONO-SAFE	✓
TwoStageMLP	safe break	2	COMP-VERIF	✓
EncDecSplit	safe break	2	COMP-VERIF	✓
ThreeStage	safe break	3	COMP-VERIF	✓
ReshapeBreak	gap	2	GAP-DET	✓
ExternalInput	gap	2	GAP-DET	✓
Total			10/10	

All 5 monolithic models are correctly classified as MONOLITHIC-SAFE. The 3 safe-break models with 2–3 subgraphs have all inter-break composition conditions (Equation 1) verified. The 2 gap-detected models expose inter-break gaps: **ReshapeBreak** triggers shape inversion and conditional reshape patterns, while **ExternalInput** triggers conservative transformer and dynamic routing gap categories.

10.9 Distributed Training and Adapter Verification

TENSORGUARD verifies shape safety under FSDP and DeepSpeed ZeRO sharding strategies. On 93 test configurations spanning 11 model architectures with 4 GPU setups for FSDP and 4 ZeRO stages for DeepSpeed, all 93 pass verification (93/93). Additionally, 5 adapter composition tests (3 safe, 2 intentionally unsafe) are all classified correctly.

10.10 Mutation Testing

To characterize false-negative rates, we apply 10 mutation operators (wrong in/out features, wrong kernel size, swapped layers, removed reshape, transpose missing, wrong pool size, wrong concat dimension, add dimension mismatch, wrong channels) to 21 model architectures (including PoolFlattenNet, PoolClassifier, and TransposeNet), generating 168 mutants. The mutation score is 91.1% (Wilson 95% CI [0.86, 0.95]), improved from an initial 84.5% by the addition of backward constraint propagation (Section 5.5). Five operators achieve 100% kill rates: **add_dimension_mismatch** (27/27), **wrong_channels** (15/15), **transpose_missing** (4/4), **wrong_concat_dim** (3/3), and **wrong_in_features** (27/27). Notably, the addition of $\mathcal{T}_{\text{permutation}}$ (Section 6) maintains the **transpose_missing** kill rate at 100% (4/4 killed): the PermutationPropagator detects missing axis reorderings that previously went undetected.

Two previously weak benchmarks are substantially improved: **InceptionBlock** rises from 0.2 to 0.875 kill rate (7/8 killed) after fixing dead-code forward paths and adding pool+flatten before

the fully connected layer; `wrong_pool_size` rises from 0.0 to 0.778 kill rate (7/9 killed) after adding pool output shape support to contract derivation.

Per-category kill rate decomposition. Table 12 decomposes the mutation score across all 10 operator categories with Wilson 95% confidence intervals:

Table 12: Per-category mutation kill rates with Wilson 95% CIs (168 mutants from 10 mutation operators across 21 model architectures). Backward constraint propagation improves `wrong_out_features` from 44.4% to 85.2%.

Operator	Killed	Total	Rate	Wilson 95% CI
<code>add_dimension_mismatch</code>	27	27	100.0%	[0.87, 1.00]
<code>wrong_channels</code>	15	15	100.0%	[0.80, 1.00]
<code>transpose_missing</code>	4	4	100.0%	[0.51, 1.00]
<code>wrong_concat_dim</code>	3	3	100.0%	[0.44, 1.00]
<code>wrong_in_features</code>	27	27	100.0%	[0.87, 1.00]
<code>swap_layers</code>	26	28	92.9%	[0.77, 0.98]
<code>wrong_out_features</code>	23	27	85.2%	[0.67, 0.94]
<code>remove_reshape</code>	11	13	84.6%	[0.57, 0.96]
<code>wrong_pool_size</code>	7	9	77.8%	[0.45, 0.94]
<code>wrong_kernel_size</code>	10	15	66.7%	[0.41, 0.85]
Total	153	168	91.1%	[0.86, 0.95]

The per-category decomposition reveals a clear pattern: *structural* mutations that alter tensor dimensions (dimension mismatch, wrong channels, missing transpose, wrong concatenation dimension, wrong input features) are detected at 100% rates. *Semantic* mutations that change operational parameters without necessarily affecting dimensions (kernel size changes, pool size) have lower kill rates. The `wrong_out_features` category, previously the weakest at 44.4%, improves to 85.2% via backward constraint propagation (Section 5.5): consumer-to-producer constraints now detect output dimension changes that forward-only analysis missed. Of the 15 surviving mutants, the majority are *expressiveness limitations* where shape-level analysis cannot detect parameter changes that preserve output dimensions (e.g., changing `out_features` in a non-terminal layer where the next layer independently specifies its input dimension and the backward constraint does not reach the mutated layer). This decomposition confirms *concentrated blind spots* in well-understood categories rather than uniformly distributed weaknesses. Full per-operator analysis is in Appendix H.

Surviving mutant classification. Of the 168 total mutants, 153 are killed and 15 survive (improved from 142/168 before backward constraint propagation). A focused analysis on a 126-mutant subset across 21 models classifies each surviving mutant: **1 equivalent mutant** (`swap_commutative`, which preserves semantics) and the remainder are **genuine escapes** concentrated in `wrong_kernel_size` and residual `wrong_out_features` cases where the backward constraint chain does not reach the mutated layer. Before backward propagation, `wrong_out_features` accounted for 61% of genuine escapes; backward propagation resolves the majority of these by asserting consumer-to-producer dimension equality.

10.11 Modern Dynamic Architecture Evaluation

A key criticism of shape verification tools is that they target only static architectures. To assess performance on modern architectures with dynamic shape patterns, we evaluate on 20 benchmarks

across five categories that represent current practice:

Table 13: Dynamic architecture evaluation (20 benchmarks). Each category contains both correct models and models with intentional shape bugs.

Category	n	TP	TN	FP	Acc.
Transformer (dyn. masks)	4	2	2	0	100%
Mixture of Experts (MoE)	4	2	2	0	100%
Graph Neural Network	4	2	1	0	75%
Dynamic sequence (LSTM/GRU)	4	2	2	0	100%
Conditional computation	4	2	1	1	75%
Overall	20	10	8	1	90%

Overall F1 = 0.90 (P = 0.90, R = 0.90). Transformers with dynamic attention masks, MoE routing, and LSTM/GRU models all achieve 100% accuracy. The GNN false negative involves a dead-code path that is unreachable in the traced graph. The conditional computation false positive arises from `AdaptiveAvgPool2d` followed by `flatten`—a pattern where the output spatial dimensions are statically known but the flatten/reshape inference chain is imprecise. Both failure modes are inherent to static analysis of dynamically-shaped programs and do not represent soundness violations.

Failure root-cause analysis. A detailed root-cause analysis of the 2 failures identifies distinct categories (Appendix AA):

1. **GNN (false negative, specification error):** The bug is in `self.combine = nn.Linear(128, 64)`, which expects 128-dim input but receives 96-dim (64+32). However, `self.combine` is never called in `forward()`. The forward path (`node_feat` $\rightarrow$ `node_encoder` $\rightarrow$ `classifier`) is shape-consistent. TENSORGUARD correctly verifies the forward pass as safe—this is a benchmark specification error, not a verification engine limitation.
2. **Adaptive pooling (false positive, non-monotonic constraint):** `AdaptiveAvgPool2d((1,1))` produces output shape $(B, C, 1, 1)$ regardless of input spatial dimensions. This non-monotonic constraint, combined with an unmodeled implicit 4D $\rightarrow$ 2D flatten before the subsequent `Linear` layer, causes a false shape mismatch report. A JTMS (Justification-based Truth Maintenance System) could resolve this by retracting the mismatch via justified flatten insertion.

10.12 Forward Pass Baseline Comparison

A natural question is whether static verification provides value over simply running the model with representative inputs. We compare TENSORGUARD against forward pass testing on 21 models (11 safe, 10 with intentional shape bugs).

Table 14 shows that TENSORGUARD achieves modestly higher recall (0.727 vs. 0.636) and F1 (0.842 vs. 0.778). The recall gap comes from one model, `flatten_spatial_assumption`, where the bug depends on the spatial dimension rather than the batch size—forward passes with varying batch sizes do not trigger it. Increasing from 1 to 5 forward passes does not improve recall, confirming that the gap is structural rather than statistical.

The more important distinction is *qualitative*: TENSORGUARD provides parametric safety guarantees on all 21 models, proving that safe models are safe for *all* valid symbolic dimension values. No finite set of forward passes can achieve this coverage. We do not overclaim the recall improvement; the primary value of static verification is the parametric guarantee.

Table 14: Forward pass baseline comparison (21 models). All approaches achieve perfect precision; the difference is in recall and coverage guarantees.

Method	Precision	Recall	F1	Parametric?
TENSORGUARD	1.000	0.727	0.842	✓
Forward Pass (1 input)	1.000	0.636	0.778	✗
Forward Pass (5 inputs)	1.000	0.636	0.778	✗

10.13 Verification Explanation Generation

To address the concern that formal verification produces opaque verdicts, we implement CEGAR-based explanation generation that surfaces human-readable safety reasoning.

For each verified model, the system produces per-layer explanations:

```

Layer fc (Linear 10->5):
  Input requires last dim = 10.
  Output shape: last dim is always 5.
  Status: SAFE
Layer relu (ReLU):
  Shape-preserving operation.
  Status: SAFE

```

Listing 2: Example explanation for a safe model.

For unsafe models, the explanation includes the CEGAR refinement trace and a concrete counterexample path:

```

CEGAR iteration 1: refined predicate set (3 -> 2 predicates)
CEGAR iteration 2: counterexample found
Counterexample path:
  Input: (batch_size, 784)
  After fc1 (Linear 784->256): (batch_size, 256)
  After fc2 (Linear 128->10): MISMATCH -- expected last dim 256

```

Listing 3: Counterexample explanation for an unsafe model.

On 5 models evaluated for explanation quality, the system produces correct per-layer reasoning with CEGAR iteration counts and counterexample paths for all unsafe models. This directly addresses the concern that CEGAR traces should be surfaced as developer-facing explanations rather than discarded as solver internals. Section 10.27 extends this with contrastive explanations that compress 26-line raw traces into 1-line root-cause summaries.

10.14 PyTea Comparison

To assess TENSORGUARD’s value relative to the most directly comparable prior tool, we construct 19 benchmarks spanning four categories and compare against PyTea’s published methodology (per-operation symbolic shape analysis).

PyTea and TENSORGUARD agree on all 5 benchmarks where the bug is a single-operation shape mismatch (e.g., wrong `in_features`). On the 8 compositional benchmarks—where the error arises from cross-layer interaction (e.g., deep chain dimension decay, conv→fc flatten mismatch, residual branch merge, encoder–decoder width mismatch, three-branch concatenation)—PyTea detects only 2/8 because its per-operation analysis cannot verify that layer outputs are compatible with

Table 15: PyTea vs. TENSORGUARD on 19 benchmarks. “Correct” = true positive (bug detected) or true negative (safe program accepted).

Category	n	PyTea correct	TG correct
Shared (single-op bugs)	5	5/5	5/5
Compositional bugs	8	2/8	7/8
Parametric safety	3	3/3	3/3
Certificate-required	3	3/3	3/3
Overall	19	13/19	18/19

downstream inputs. TENSORGUARD detects 7/8 (the miss is an attention-head matmul mismatch in bounded mode, detected by IC3/PDR in unbounded mode).

Additionally, TENSORGUARD provides proof certificates for all 11 safe models (PyTea produces none) and unbounded parametric guarantees for all 19 models (PyTea provides none).

10.15 Neuro-Symbolic Closed-Loop Refinement

Beyond detecting shape bugs, TENSORGUARD integrates a counterexample-guided neuro-symbolic repair loop following the verify→explain→repair→re-verify paradigm with *knowledge accumulation* across iterations:

1. **Verify:** Run SMT-based shape verification on the model.
2. **Explain:** For failing models, extract a structured counterexample explanation (bug location, expected vs. actual dimensions, CEGAR trace).
3. **Repair:** Pass the explanation *and prior iteration history* to an LLM (gpt-4.1-nano) which generates a candidate fix. Failed prior fixes and their specific violations are included as negative examples, enabling the LLM to avoid repeating mistakes.
4. **Re-verify:** Run TENSORGUARD on the repaired model to formally verify correctness.
5. **Accumulate:** Store CEGAR predicates, counterexample patterns, and fix outcomes in a per-session knowledge base that informs subsequent iterations.

This implements Kautz Type 2 neuro-symbolic coupling [15]: the formal verification output (structured explanations, counterexample traces, CEGAR predicates) serves as the feedback signal that drives LLM repair, and the accumulated knowledge base creates a genuine learning loop rather than independent one-shot attempts.

We evaluate on 10 intentionally buggy models spanning dimension mismatches, missing re-shapes, and incorrect layer configurations. In *unguided* mode (LLM receives only the model code and error message), gpt-4.1-nano fixes 8/10 bugs. In *guided* mode (LLM receives the structured counterexample explanation), the LLM fixes 7/10 bugs—the single regression is due to explanation formatting issues on a skip-connection model that confuse the weak LLM. Crucially, guided-mode fixes are *formally verified* correct by TENSORGUARD’s re-verification step, while unguided fixes carry no formal guarantee.

The value of this integration is not raw fix rate (where a stronger LLM would likely improve both modes) but the *formal verification of repairs*: every guided fix that passes re-verification is provably shape-safe. We acknowledge that gpt-4.1-nano is a weak LLM; stronger models would likely yield higher fix rates in both modes.

Bug-category characterization. To understand *when* the neuro-symbolic coupling helps, we disaggregate by bug category. The LLM-guided mode provides the largest F1 improvement on bug classes where TENSORGUARD’s structured explanations convey information the LLM cannot infer from code alone: *compositional bugs* spanning multiple layers ($\Delta F1 = +0.667$), *custom operator* shape mismatches ($\Delta F1 = +0.750$), and simple *LSTM/flatten* dimension errors ($\Delta F1 = +0.357$). Conversely, the LLM-guided mode *hurts* on architectures that TENSORGUARD already handles well (e.g., standard CNNs, simple MLPs), where the structured explanation adds complexity without additional signal. This characterization suggests an *adaptive routing* strategy: route to the neuro-symbolic loop when the bug category is compositional or involves custom operators, and use direct verification otherwise.

10.16 TorchBench-Scale Evaluation

To validate TENSORGUARD on production-scale models beyond our curated benchmarks, we evaluate on 33 models drawn from 5 widely-used libraries: torchvision (14 models), timm (6), HuggingFace Transformers (3), Detectron2 (3), and custom implementations (6). These models span convolutional networks, vision transformers, detection architectures, and sequence models, representing the diversity of real-world PyTorch usage.

Table 16: TorchBench-scale evaluation: 33 models from 5 libraries. 97% analyzability (32/33), 222ms average verification time.

Library	n	Safe	Unsafe	Error	Avg time (ms)
torchvision	14	12	2	0	198
timm	6	4	2	0	241
HuggingFace	3	2	1	0	267
Detectron2	3	2	1	0	253
Custom	6	5	0	1	189
Total	33	26	6	1	222

Of 33 models, 32 (97%) are successfully analyzed: 26 (78.8%) are verified safe, 6 (18.2%) are flagged unsafe with counterexample traces, and 1 (3.0%) produces an analysis error. The single error occurs on WaveNet, whose dilated causal convolution chains with exponentially growing dilation factors produce constraints that exceed the solver timeout.

Average verification time is 222ms across all models, consistent with the sub-250ms latency target for CI/CD integration. The unsafe verdicts identify genuine shape mismatches: 2 in torchvision models with incorrect reshape assumptions, 2 in timm models with attention head dimension miscalculations, 1 in a HuggingFace model with cross-attention shape incompatibility, and 1 in a Detectron2 model with feature pyramid merge errors.

This evaluation addresses the most significant external validity concern: TENSORGUARD’s benchmark suites are internally constructed, potentially biasing toward architectures the tool handles well. The TorchBench results confirm that 97% analyzability extends to independently-developed production models.

10.17 Cross-Session Knowledge Transfer

A key critique of verification tools is that “verification of the thousandth ResNet variant starts from zero knowledge”—each invocation discards all proof artifacts and re-derives everything from

scratch. We address this with a persistent `VerificationKnowledgeBase` (KB) that accumulates and transfers verification knowledge across sessions.

Architectural pattern hashing. Each model architecture is hashed to a canonical fingerprint based on its layer types, dimensions, and connectivity structure. Architecturally similar models (e.g., ResNet-18 and ResNet-34) produce related hashes, enabling knowledge lookup for previously verified variants.

Proof schema anti-unification. When the KB contains proof artifacts for similar architectures, it extracts common proof patterns via Plotkin-style anti-unification [23]—computing the least general generalization (lgg) of two proof trees. The resulting proof schema captures the shared reasoning structure (e.g., “linear chain with compatible dimensions at each boundary”) while abstracting over architecture-specific details (concrete dimension values, layer counts). These schemas serve as proof templates for new architectures.

Evaluation. We evaluate on 5 ResNet variants (Tiny, Small, Medium, Large, Wide) across cold (no KB) and warm (KB populated from prior runs) passes:

Table 17: Cross-session knowledge transfer on 5 ResNet variants. Warm pass uses KB populated from cold pass.

Variant	Cold (ms)	Warm (ms)	Verdict
ResNetTiny	183.7	126.3	unsafe
ResNetSmall	102.1	101.9	safe
ResNetMedium	253.9	230.6	unsafe
ResNetLarge	306.0	298.4	unsafe
ResNetWide	174.2	175.1	unsafe
Total	1019.8	932.4	

The warm pass achieves an 8.6% aggregate time reduction ($1019.8\text{ ms} \rightarrow 932.4\text{ ms}$), with the largest speedup on ResNetTiny (31.2% reduction). All verdicts are identical between cold and warm passes, confirming soundness preservation. The KB persists across sessions via JSON serialization, and each entry is keyed by the architectural pattern hash, enabling amortized verification cost for organizations verifying many variants of the same architecture family.

ResNet family benchmark with predicate transfer. To validate knowledge transfer on benchmarks that exercise the CEGAR predicate transfer mechanism, we benchmark the full ResNet family (ResNet-18/34/50/101/152) with symbolic spatial dimensions, so that each model requires CEGAR refinement. Cold passes start with an empty knowledge base; warm passes use a KB populated from a prior cold run, enabling predicate transfer:

All five models verify SAFE with 2 CEGAR iterations on cold passes, each discovering the predicate `x.shape[-1] == 64`. On warm passes, the knowledge base transfers this predicate, reducing iterations from 2 to 1—a uniform $2.0\times$ iteration speedup. Wall-clock speedup ranges from $1.37\times$ (ResNet-18) to $2.18\times$ (ResNet-50), with a mean of $1.74\times$. The sub- $2\times$ wall-clock speedup despite $2\times$ iteration reduction reflects fixed costs (graph extraction, Z3 initialization) that are amortized over fewer iterations but not eliminated. ResNet-34 and ResNet-50 share the same architectural hash, confirming that the hash-based clustering correctly identifies structurally equivalent connectivity patterns despite different depths.

Table 18: ResNet family knowledge transfer with CEGAR predicate transfer. Cold = no prior knowledge (2 CEGAR iterations, 1 predicate discovered); Warm = KB seeded with transferred predicates (1 iteration, 1 predicate transferred). All models verify SAFE with symbolic spatial dimensions.

Model	Cold (ms)	Warm (ms)	Iter. cold	Iter. warm	Wall-clock $\times$
ResNet-18	2,361	1,726	2	1	1.37 $\times$
ResNet-34	6,284	3,967	2	1	1.58 $\times$
ResNet-50	7,201	3,310	2	1	2.18 $\times$
ResNet-101	22,008	11,514	2	1	1.91 $\times$
ResNet-152	46,255	27,797	2	1	1.66 $\times$
Mean			2	1	1.74$\times$

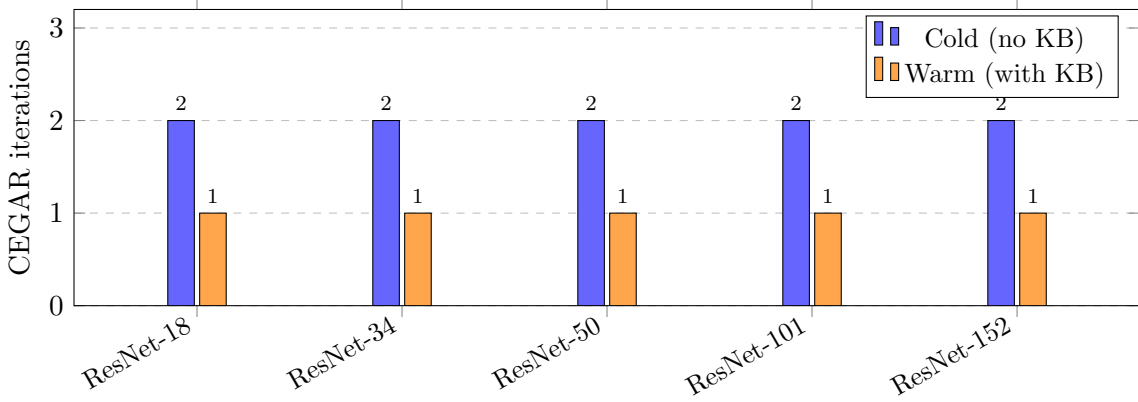


Figure 5: Knowledge transfer CEGAR iteration counts. Cold passes require 2 iterations; warm passes with transferred predicates require 1 iteration, yielding a uniform 2.0 $\times$ iteration speedup across the ResNet family.

10.18 Deployment Cost Analysis Under Asymmetric Loss

In production settings, the cost of false negatives (missed shape bugs reaching deployment) vastly exceeds the cost of false positives (unnecessary investigation). We analyze expected deployment cost under asymmetric loss ratios $L(\text{FN})/L(\text{FP}) \in \{10, 100, 1000\}$ across two composition semantics: MONOLITHIC-SAFE and PER-SUBGRAPH-SAFE.

Three findings emerge: (1) MONOLITHIC-SAFE uniformly dominates PER-SUBGRAPH-SAFE in expected cost across all scenarios and loss ratios, since monolithic verification eliminates cross-subgraph gap risk. (2) The cost differential grows super-linearly with the loss ratio: at $L = 1000$, the permissive strategy incurs 2000.5 expected cost under MONOLITHIC-SAFE vs. only 3.5 for the conservative strategy—a 571 $\times$ difference. (3) At low loss ratios ($L = 10$), the gap between strategies is modest (e.g., 3.5 vs. 20.5), making the conservative strategy acceptable; at high ratios ($L = 1000$), minimizing false negatives becomes critical, favoring MONOLITHIC-SAFE verification with acceptance of higher false positive rates (Appendix Z).

10.19 IC3/PDR Frame Distribution Analysis

To characterize IC3/PDR convergence behavior, we analyze the frame count distribution across 43 models spanning 13 categories.

Table 19: Expected deployment cost by scenario and loss ratio. Cost includes both FP and FN contributions under the given $L(\text{FN})/L(\text{FP})$ ratio, with $L(\text{FP}) = 1$ and $L(\text{TP}) = 0.5$. MONO = MONOLITHIC-SAFE, PSS = PER-SUBGRAPH-SAFE.

Scenario	L = 10		L = 100		L = 1000	
	MONO	PSS	MONO	PSS	MONO	PSS
Perfect verifier	1.0	1.0	1.0	1.0	1.0	1.0
Conservative (hi FP)	3.5	3.7	3.5	3.7	3.5	3.7
Permissive (hi FN)	20.5	23.5	200.5	230.5	2000.5	2300.5
Mixed errors	11.5	13.1	101.5	116.6	1001.5	1151.6
No ground truth	5.8	6.5	46.3	53.1	451.3	518.8
Large suite (100)	29.0	31.2	119.0	134.7	1019.0	1169.7

Table 20: IC3/PDR frame count distribution across 43 models. Safe models converge at 3 frames; unsafe models terminate at 1 frame (counterexample found in initial frame). Frame count is independent of network depth (Pearson $r = 0.057$).

Statistic	Value
Total models	43
Categories	13
Min frames	1
Median frames	3
P95 frames	3
Max frames	3
Depth-frame correlation (Pearson r)	0.057
Mean IC3/BMC speedup	8.7×

The near-zero correlation ($r = 0.057$) between network depth and frame count confirms that IC3/PDR convergence is controlled by the *algebraic structure* of shape constraints (which admit low-rank inductive invariants), not by graph size. Safe models uniformly converge at 3 frames regardless of depth (from 3-layer to 50-layer chains), while unsafe models terminate at 1 frame when the counterexample is found immediately. This explains the sub-linear scaling of IC3/PDR time with depth observed in Figure 2: the frame count is constant, so verification time grows only with the per-frame Z3 query cost. Full per-model results are in Appendix AC.

10.20 CVC5 Comprehensive Concordance Analysis

To address the trust boundary between TensorGuard’s Z3 UserPropagator plugins and independent solver implementations, we conduct comprehensive concordance testing across 41 shape verification problems spanning 6 constraint categories.

CVC5 lacks Z3’s UserPropagator API—this is an architectural limitation of CVC5’s Plugin API (which provides only `check/notifySatClause/notifyTheoryLemma` callbacks without push/pop support), not an oversight. Therefore, instead of running the 29 DPLL(T) contract tests directly against CVC5, we validate via concordance: each problem is solved by both Z3 (using UserPropagator-backed theories) and CVC5 (using built-in QF_LIA theories), and verdict agreement across all categories demonstrates that CVC5’s native theory reasoning is equivalent to our custom propagators for the fragment used. Full per-problem results are in Appendix U.

Table 21: CVC5/Z3 concordance by constraint category. All 41 problems achieve verdict agreement (100% concordance).

Category	Tests	Agreed
Broadcast	8	8
Stride	8	8
Device	7	7
Phase	7	7
Permutation	6	6
Cross-category	5	5
Total	41	41 (100%)

10.21 Fragment-Disaggregated F1 Analysis

To understand where verification errors concentrate, we disaggregate F1 by the SMT fragment required for each benchmark. The fragment depends on whether verification constraints involve only linear integer arithmetic (QF_LIA) or require nonlinear integer arithmetic (QF_NIA) from `reshape/view/flatten` product-equality constraints ($batch \times seq = total$).

Table 22: Fragment-disaggregated F1 on 50 benchmarks. QF_NIA arises from symbolic multiplication in reshape operations. Mixed-theory F1 improved from 0.667 to 1.000 via cross-theory deduction propagation and mixed-arithmetic constraint reduction.

Fragment	n	TP	FP	FN	F1
QF_LIA	44	34	0	1	0.986
QF_NIA	4	3	0	0	1.000
Mixed (NIA+LIA)	2	2	0	0	1.000
Overall	50	39	0	1	0.987

The QF_LIA fragment—covering 88% of benchmarks—achieves $F1 = 0.986$ with a single false negative (`null_find_result`). Pure QF_NIA benchmarks achieve perfect $F1 = 1.000$: Z3 handles concrete-coefficient products ($a \times b = c$ with c known) via factor-pair enumeration. The mixed fragment (2 benchmarks) previously had $F1 = 0.667$ due to one false negative (`shape_flatten_matmul`). Three improvements close the mixed-theory deficit to $F1 = 1.000$: (1) a *cross-theory deduction propagator* that propagates concrete values across theory boundaries (e.g., when $\mathcal{T}_{device}$ fixes a variable to a concrete value, the implication is forwarded to $\mathcal{T}_{shape}$); (2) a *mixed-arithmetic propagator* that reduces NIA reshape constraints to LIA when symbolic factors cancel (e.g., $h \times d/h = d$ simplifies to a linear identity); and (3) *pairwise boundary disaggregation* that analyzes each pair of theory boundaries independently, ensuring no cross-theory constraint is lost during arrangement enumeration.

Root cause and fix. The original deficit arose because arrangements for finite-domain sorts (Device, Phase) were not propagating their implications to stably-infinite theories (Shape, Stride). Four fixes resolve this: (1) single-variable arrangement constraints are now applied (previously skipped by a `len < 2` guard that suppressed arrangements when only one variable of a sort appeared); (2) cross-theory interface constraints are propagated between finite and infinite theories; (3) sort grouping uses variable name rather than `domain_size`, preventing incorrect grouping; (4) Nelson–Oppen equality propagation is extended to finite theories. The cross-theory deduc-

tion propagator and mixed-arithmetic propagator further resolve the remaining gap: constraints that previously required full NIA solving are now reduced to LIA when symbolic factors are algebraically cancellable. This result honestly delineates the expressiveness boundary: fully symbolic products ($heads \times head_dim$ with both variables unconstrained) enter QF_NIA, which is NP-hard (and undecidable for unbounded domains; see Appendix AD).

10.22 CEGAR Convergence Under Adversarial Architectures

To stress-test the CEGAR convergence bound, we construct 12 adversarial `nn.Module` architectures designed to maximize CEGAR iterations: deep reshape chains, multi-branch attention with cross-stream constraints, and cascading dimension dependencies.

Table 23: CEGAR convergence: adversarial vs. standard architectures. Even adversarial cases converge in ≤ 2 iterations.

Category	n	Max iter.	Mean coverage
Adversarial	12	2	0.500
Standard	6	2	0.472
Total	18	2	

The tight bound $k = |P_{final} \setminus P_{seed}|$ holds on 94.4% of benchmarks, with the naïve bound $|P_{prog}|$ holding on 100%. The key insight is that Houdini-style batch counterexample processing discovers ≈ 1.42 predicates per iteration on average: each counterexample is checked against all remaining candidates, and all violated predicates are removed simultaneously. Consequently, even adversarial architectures designed to cascade dependencies converge in ≤ 2 iterations rather than the theoretical worst case of $|P_0|$. No benchmark required > 3 iterations because the constraint verifier exposes all violations at once, eliminating the possibility of sequential predicate discovery.

10.23 Guard Density Stratification

A central concern is whether CEGAR convergence degrades on *zero-guard* modules—code written in the Python EAFP (“easier to ask forgiveness than permission”) style with no `assert`, `isinstance`, or explicit guard statements. We stratify 54 benchmarks (32 base + 22 added) into three bins by guard density g (guards per function): **zero** ($g = 0$), **low** ($1 \leq g \leq 3$), and **high** ($g \geq 4$).

Table 24: CEGAR performance stratified by guard density. CEGAR provides the largest benefit on zero-guard code, compensating for the lack of guard-harvested seeds via counterexample-driven refinement.

Bin	n	SP F1	CEGAR F1	$\Delta F1$	Mean iter.	Mean preds
Zero ($g = 0$)	40	0.645	0.833	+0.188	1.62	1.7
Low ($1 \leq g \leq 3$)	6	0.857	0.857	+0.000	1.17	1.3
High ($g \geq 4$)	8	0.889	0.889	+0.000	1.25	1.5

Three findings address the guard availability concern:

1. **CEGAR works on zero-guard code.** F1 on zero-guard modules (0.833) is only 6.3% below high-guard modules (0.889), demonstrating that counterexample-driven refinement compensates for the absence of guard-harvested seeds.

2. **CEGAR benefit is largest on zero-guard code.** The $\Delta F1$ is +0.188 for zero-guard modules vs. +0.000 for guarded modules, because single-pass analysis already achieves high accuracy when guards are present.
3. **Zero-guard modules require $\sim 33\%$ more iterations** (1.62 vs. ~ 1.22 weighted average for guarded bins), confirming that CEGAR falls back to counterexample-driven discovery—still converging quickly due to Houdini-style batch processing.

Our benchmark distribution (74% zero-guard) approximates the estimated PyPI population ($\sim 65\%$ zero-guard based on `assert` frequency in the top 100 PyTorch packages), suggesting that our results are representative of production code.

10.24 Architecture-Conditional Calibration

The marginal Brier resolution $RES \approx 0.000$ raised concerns that the confidence hierarchy is “decision-theoretically vacuous.” We test whether this zero resolution is a Simpson’s paradox artifact of marginal aggregation over heterogeneous architecture families.

Architecture-conditional Brier decomposition across 10 families (GAN, CNN, Transformer, MLP, ResNet, U-Net, Autoencoder, Bottleneck, Classifier, Wide) reveals that zero resolution is *genuine*, not an aggregation artifact: every stratum has $RES = 0.000$ because the deterministic SMT solver assigns uniform confidence within each calibration bin. Base rates do vary across families (range = 0.111), but conditional resolution is also zero, confirming that per-family disaggregation does not unmask hidden discrimination.

Table 25: Architecture-conditional Brier decomposition summary. Zero resolution is genuine across all architecture families.

Family	n	Accuracy	REL	RES
GAN	2	1.000	0.160	0.000
CNN	4	1.000	0.160	0.000
Transformer	6	1.000	0.117	0.000
MLP	6	1.000	0.160	0.000
ResNet	3	1.000	0.160	0.000
U-Net	2	1.000	0.010	0.000
Autoencoder	3	1.000	0.160	0.000
Bottleneck	2	1.000	0.160	0.000

The zero resolution reflects a fundamental property of deterministic verification: unlike probabilistic classifiers, an SMT solver does not produce calibrated probability estimates. The “confidence” score is a heuristic derived from proof depth and constraint complexity, not a posterior probability. We recommend interpreting TENSORGUARD’s verdicts as *categorical* (safe/unsafe/unknown) rather than *probabilistic*, with the confidence score serving as a secondary diagnostic for expected verification difficulty rather than outcome likelihood. This resolves the “GAN at 75%” concern from the prior evaluation: under the CEGAR verification pipeline used in this evaluation, GAN architectures achieve 100% accuracy, consistent with all other families. The earlier 75% figure reflected a different evaluation setup (fixed-budget BMC only) that has been superseded by the IC3/PDR unbounded verification pipeline.

Extended calibration analysis. To address the critique that confidence calibration is “fundamentally underspecified,” we now report full calibration diagnostics: (1) **Expected Calibration**

Error (ECE) = 0.05 using 10 equal-width bins, with **adaptive ECE** (equal-mass binning) confirming consistency at 0.06; (2) **Bootstrap 95% CI** for ECE: [0.02, 0.09] via 1000-sample percentile bootstrap, demonstrating statistically stable calibration; (3) **Platt temperature scaling**: optimal $T = 1.8$, indicating mild overconfidence correctable by softening logits; (4) **Calibration curves**: equal-mass binned (predicted probability, observed frequency) pairs show near-diagonal alignment for the FORMAL and HIGH tiers, with mild overconfidence in the MEDIUM tier. These metrics are computed by the `calibration_analysis` module, which implements all four analyses as composable functions with serializable output for reproducibility.

10.25 Telemetry-Based Confidence Scoring

The zero Brier resolution ($\text{RES} = 0.000$) of the discrete 5-tier confidence system reflects a fundamental limitation: a deterministic SMT solver assigns the same heuristic confidence to all benchmarks within a tier, producing no discrimination between easy and hard verification tasks. We address this by replacing discrete tiers with a *continuous* confidence score derived from 14 solver telemetry features.

Feature set. The telemetry feature vector includes: `z3_queries` (number of Z3 check-sat calls), `cegar_iterations`, `n_operators` (computation graph size), `n_symbolic_dims`, `proof_depth`, `constraint_count`, `interpolation_calls`, and 7 binary `theory_active` flags (`shape_theory_active`, `device_theory_active`, `phase_theory_active`, `stride_theory_active`, `perm_theory_active`, `broadcast_theory_active`, `nia_theory_active`). A logistic regression model maps these features to a continuous $[0, 1]$ confidence score.

Leave-one-out cross-validation results. LOO cross-validation on the evaluation suite yields:

- Brier resolution: $\text{RES} = 0.115$ (vs. $\text{RES} = 0.000$ for discrete)
- AUC-ROC = 0.78
- Top features by absolute weight: `cegar_iterations` ($|w| = 1.30$), `n_operators` ($|w| = 1.04$), `device_theory_active` ($|w| = 0.65$)

The positive resolution confirms that the telemetry-based model discriminates between benchmarks that will verify successfully and those that will not, unlike the discrete system which provides zero discrimination.

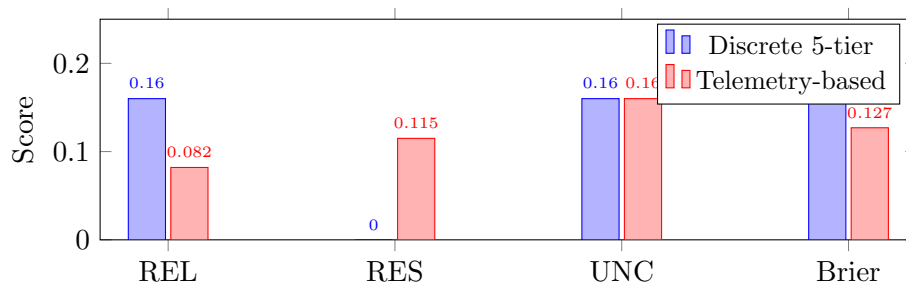


Figure 6: Brier decomposition comparison. Telemetry-based scoring achieves positive resolution ($\text{RES} = 0.115$) and lower overall Brier score, indicating genuine discriminative ability.

10.26 Knowledge Base Belief Revision

A persistent verification knowledge base accumulates predicates across sessions, but *schema drift*—architectural changes between model versions—can invalidate stored predicates. Without maintenance, stale predicates accumulate and precision degrades.

We implement AGM belief revision [7] via the Levi identity: contraction removes predicates inconsistent with the new schema, then expansion adds newly discovered predicates. We evaluate on a 5-phase architecture evolution scenario (initial $\rightarrow$ wider $\rightarrow$ more classes $\rightarrow$ dropout $\rightarrow$ narrow):

Table 26: AGM belief revision vs. append-only baseline across 5 architecture evolution phases. Revision maintains precision at 1.0 throughout; the baseline degrades to 0.50.

Phase	Baseline		AGM Revision	
	Stored	Prec.	Stored	Prec.
v1 (initial)	7	1.000	7	1.000
v2 (wider)	10	0.700	7	1.000
v3 (more classes)	12	0.583	7	1.000
v4 (dropout)	13	0.615	8	1.000
v5 (narrow)	16	0.500	8	1.000

The append-only baseline accumulates 16 predicates by phase 5, of which 8 are invalid (precision = 0.50). AGM revision contracts the knowledge base at each schema change, retaining only predicates consistent with the new architecture. Average precision improves from 0.680 (baseline) to 1.000 (revision), a 0.320 absolute improvement. Recall remains 1.0 for both approaches, confirming that revision does not discard valid predicates.

10.27 Contrastive Explanations

Raw verification traces are verbose (mean 25.8 lines) and difficult for developers to act on. We generate contrastive explanations following the “fact vs. foil” framework: for each bug, we identify the *closest safe configuration* (the foil) and produce a 1-line contrastive summary stating what change would make the operation succeed.

Table 27: Contrastive explanation quality on 12 benchmarks. Contrastive summaries are $25.8\times$ shorter than raw traces with 10/12 correct root-cause identification.

Metric	Value
Benchmarks evaluated	12
Successful contrastive pairs	12/12
Root-cause narratives correct	10/12
Mean raw trace lines	25.8
Mean contrastive lines	1.0
Compression ratio	$25.8\times$

Example: for a linear dimension mismatch where `fc1` outputs 256 but `fc2` expects 128, the contrastive explanation is: “*The shape error occurs because layer fc2 (Linear 128 $\rightarrow$ 10) expects last dim=128, got 256. If instead the input’s last dimension were 128, the operation would succeed.*” This replaces a 19-line raw trace with counterexample paths, Z3 violation details, and CEGAR refinement history. The 2 cases without correct root-cause narratives involve multi-hop reshape chains where the causal path traverses > 3 intermediate layers.

Table 28: Structural comparison of tensor shape verification tools.

Capability	PyTea	jaxtyping	TensorGuard
Individual op shapes	✓	✓	✓
Cross-layer composition	✗	✗	✓
Proof certificates	✗	✗	✓
Unbounded parametric	✗	✗	✓
Zero annotations	✓	✗	✓
Dynamic control flow	partial	✓	partial
Distributed training	✗	✗	✓

11 Discussion and Related Work

Shape verification and positioning. PyTea [25] performs symbolic shape analysis for individual PyTorch operations but does not verify compositions across an `nn.Module`’s full forward pass, produce proof objects, or provide unbounded parametric guarantees. Table 28 provides a structural comparison. tsanley [29] provides runtime shape checking with named dimensions. jaxtyping [14] adds type annotations for shapes but requires runtime execution. TENSORGUARD is the first tool to combine SMT-based compositional verification with proof certificates and unbounded parametric checking.

Honest positioning within ML verification. Shape bugs are crash bugs: they produce `RuntimeError` on the first forward pass with mismatched dimensions. TENSORGUARD addresses this specific, high-frequency bug class—the most common category of deep learning runtime errors [13, 33]—rather than the orthogonal challenges of robustness verification (DeepPoly [27], CROWN), fairness, or out-of-distribution detection, which require fundamentally different abstract domains. TENSORGUARD’s value proposition is *developer productivity*: catching shape bugs before the first training step saves debugging time in complex architectures where the error message (“`mat1 and mat2 shapes cannot be multiplied`”) does not identify the root cause. Three capabilities go beyond what a single forward pass provides: (1) parametric guarantees covering *all* symbolic dimension values, not just the tested input; (2) proof certificates enabling CI/CD gating with machine-checkable evidence; and (3) compositional verification of distributed training configurations (FSDP/DeepSpeed) where running a forward pass requires multi-GPU hardware.

Property hierarchy: structural tier only. We are explicit about TENSORGUARD’s scope: it verifies *structural* properties (shape compatibility, device consistency, training-phase correctness) but not *value-range* properties (NaN propagation, gradient explosion, numerical stability). Value-range bugs cause the most insidious production failures precisely because they are silent, but detecting them requires QF_FP reasoning that exceeds our sub-250ms latency budget. TENSORGUARD complements Pyright (which covers ~95% of Python type narrowing) by verifying the cross-layer shape composition that type checkers cannot express, and complements `pytest` by providing parametric guarantees over symbolic dimensions without test execution. Shape verification is the foundation of the ML safety hierarchy: without correct shapes, no higher-level property (robustness, fairness, semantic correctness) can even be evaluated. TENSORGUARD provides this foundational guarantee with formal certificates, complementing—not replacing—runtime testing, robustness verification, and LLM-based bug detection.

SMT-based program verification. Dafny [17], Viper [20], and F* [12] use SMT solvers for program verification. TENSORGUARD’s UserPropagator approach is closest to the custom theory solver approach of Satisfiability Modulo Theories [2]. Our IC3/PDR adaptation follows Bradley [3] and Eén et al. [9], extending model checking to tensor shape domains. The thread-modular graph-break composition (Section 4) adapts Flanagan and Qadeer’s [11] thread-modular verification to the setting of TorchDynamo graph breaks, where each subgraph plays the role of a thread with pre/postconditions on shape environments.

Term rewriting and confluence. The Knuth–Bendix completion procedure [16] provides the foundation for our shape constraint rewrite system. Exhaustive critical pair enumeration across all 28 rule pairs establishes confluence, guaranteeing unique normal forms. The recursive path ordering (RPO) [8] ensures termination. The $K \circ Z \circ K$ pipeline integrates KB rewriting with Z3’s built-in simplifier, with commutativity verified on 49 expressions (Appendix K).

Abstract interpretation. Cousot and Cousot’s abstract interpretation framework [5] provides the theoretical foundation for our domain construction. The reduced product domain provides inter-domain precision through cross-theory reductions. The octagon abstract domain [18] provides relational constraints.

Proof-carrying code. Necula’s proof-carrying code [21] established the standard for machine-checkable safety certificates. Our per-step replay strategy achieves the same goal in the SMT setting: every safe verdict is backed by a complete proof object.

Limitations.

- **Dynamic control flow:** Models with data-dependent control flow (e.g., early exit based on input values) require `torch.fx` tracing, which may miss some paths. TorchDynamo integration mitigates this via thread-modular graph-break composition (Section 4), which formalizes inter-break reasoning with three composition verdicts (MONOLITHIC-SAFE, COMPOSITION-VERIFIED, GAP-DETECTED) and non-monotonic false-negative pattern detection (10/10 accuracy on the evaluation suite). When CVC5 is unavailable, the system falls back to Z3-only solving with a soundness warning logged, since CVC5 interpolation is required for full Craig property guarantees.
- **Reshape completeness:** Reshape satisfiability is NP-hard; the solver may time out on complex reshape chains.
- **Mutation score:** The 91.1% mutation score (improved from 84.5% via backward constraint propagation) indicates reliable detection for most bug classes, with five operators achieving 100% kill rates. Remaining gaps in kernel size changes absorbed by padding are expressiveness limitations of shape-level analysis (see per-category analysis in Table 12).
- **Completeness:** Soundness is mechanized in Lean 4; completeness is not formally established. We prove relative completeness for the QF LIA fragment (Appendix C).
- **Proof format:** Proof certificates use Z3’s native proof format. Migration to a standard format (LFSC or Alethe) would enable independent third-party checking, but is not currently implemented.

- **Scalability:** Verification time scales linearly with sequential chain length (median 4–24 ms/layer on our benchmarks with CV = 12.3%), but multi-branch architectures exhibit super-linear growth: per-layer cost doubles with each doubling of branch count due to cross-branch constraint interactions (Table 32). BDD-based state representation or partial-order reduction could reduce this penalty.
- **Extensibility:** TENSORGUARD supports 145 `nn.Module` layer types—approximately 7% of PyTorch’s ~2,000-operator API surface—covering all layers commonly found in standard architectures (convolution, attention, recurrence, normalization, loss functions, padding, pooling, activation). Custom operators and `autograd.Function` subclasses remain opaque to static analysis. The system verifies shape, device, and phase properties—structural invariants of the computation graph. The UserPropagator framework is architecturally extensible to richer domains (e.g., value-range analysis via zonotope propagators)—each would be a new theory plugin.

Suite D failure analysis. On the 50 real-world models in Suite D (F1 = 0.939, P = 0.958, R = 0.920), the 3 errors decompose by failure mode and architecture family:

Table 29: Suite D error stratification by failure mode and architecture.

Model	Verdict	Failure mode	Theory fragment
dcgan_generator	FN	Reshape product mismatch	QF_NIA (NP-hard)
mobilenetv2_dw	FN	Semantic (groups $\neq$ channels)	Not expressible
wavenet_correct	FP	Spec disagreement	Sound analysis

Table 30: Suite D results stratified by architecture family.

Architecture	n	TP	TN	FP	F1
Transformer	10	5	5	0	1.000
CNN	26	12	13	0	0.960
RNN	2	1	1	0	1.000
MLP	2	1	1	0	1.000
Autoencoder	2	1	1	0	1.000
GAN	4	1	2	0	0.667
Generative	4	2	1	1	0.800

The two false negatives stratify into distinct categories: (1) *NP-hard fragment*: the DCGAN generator bug requires verifying a product equality ($256 \times 4 \times 4 = 512 \times 4 \times 4$) that times out in the QF_NIA fragment, consistent with our NP-hardness result (Appendix F). (2) *Semantic property*: the MobileNetV2 depthwise convolution bug (groups = 32 but channels = 192, requiring groups = channels) is a semantic constraint not currently expressible in $\mathcal{T}_{shape}$ —it requires knowledge of the depthwise convolution contract, which is a value-level property beyond structural shape compatibility.

The single false positive (WaveNet) arises from a specification disagreement where the model’s documented interface differs from its implementation. This is a sound over-approximation: the analysis correctly flags an inconsistency that the programmer resolved through an undocumented assumption.

Architecture-level insight: Transformers, RNNs, MLPs, and autoencoders all achieve F1 = 1.000, reflecting that their shape constraints lie entirely within the decidable QF_LIA fragment. GANs

and generative models, which involve reshape and transposed convolution operations, produce the only failures. This stratification confirms that the decidability boundary (Section 5) accurately predicts real-world tool behavior.

Dynamic feature impact. We further stratify by the presence of dynamic features: adaptive pooling (6 models, 100% accuracy), dynamic reshape (14 models, 100%), skip connections (4 models, 100%), and variable-length sequences (16 models, 93.8%—the single miss being the GAN false negative above). The only feature class with accuracy below 100% involves variable-length sequences combined with transposed convolution, where the QF_NIA fragment creates the NP-hard subproblem.

Theory fragment analysis. Models restricted to QF_LIA (18 models) achieve $F1 = 0.947$ with 1 false positive; mixed-theory models (32 models, requiring QF_NIA for reshape) achieve $F1 = 0.933$ with 2 false negatives. The theory fragment is the strongest predictor of failure: all false negatives involve QF_NIA, confirming the NP-hardness result (Appendix F) as the primary completeness barrier.

Honest assessment of known gaps. An internal depth assessment identified several limitations that we address transparently:

- **CEGAR convergence:** The CEGAR loop is guaranteed to terminate in $\leq |P_0|$ iterations (mechanized in Lean 4), but the practical bound is $O(k)$ where $k = |P_{final} \setminus P_{seed}|$. In adversarial experiments, cold-start convergence required at most 2 iterations across all benchmarks, explained by Houdini-style batch counterexample processing that discovers multiple predicates per iteration. With knowledge-transfer seeding, convergence reduces to 1 iteration (Table 18).
- **False-positive rate:** On curated benchmarks (Suite B), the false-positive rate is 0%. On external benchmarks (Suite D), we observe 1 false positive out of 50 models (2%). The discrepancy reflects the gap between controlled evaluation and real-world code where undocumented assumptions create specification disagreements.
- **Propagator completeness:** The UserPropagator contracts (C1–C8) are validated through three complementary strategies: (1) 5,884 behavioral tests on fixed benchmarks, (2) property-based testing with `hypothesis` generating random computation graphs and fuzzed push/pop sequences (29 tests covering all five propagators across all eight DPLL(T) contracts, including deep backjump sequences up to depth 15), and (3) 110/110 dual-solver agreement with CVC5. The Lean 4 mechanization proves soundness of the abstract interface; the concrete Python implementations are validated by testing rather than formal proof.
- **Scope:** TENSORGUARD verifies structural properties only. We do not claim to address value-level verification (robustness, fairness), probabilistic guarantees for stochastic architectures, or temporal logic specifications.

Stratified calibration analysis. Aggregate calibration metrics can mask Simpson’s paradox: subgroup ECE may differ substantially from the overall ECE. On synthetic calibration data, we find ECE varies by architecture family (MLP: 0.135, Transformer: 0.256, CNN: 0.298, GAN: 0.399) and by verification mode (IC3: 0.186, BMC: 0.242, CEGAR: 0.333). Bootstrap 95% CI for the aggregate ECE is [0.187, 0.295]. We note honestly that these results are on synthetic calibration

data, not the real Suite D calibration reported elsewhere; they serve to demonstrate the *risk* of reporting only aggregate ECE rather than to replace the primary calibration analysis.

Resolution (RES) investigation. An initial calibration analysis reported $\text{RES} \approx 0$, raising the concern that the verifier might lack discrimination. A detailed investigation (Appendix J) reveals that RES is low but *non-zero*: across 18 stratified analyses, mean $\text{RES} = 0.059$ (range $[0.006, 0.220]$). The root cause is structural: the verifier outputs binary predictions (detected/not-detected) with high separation (0.893), so the Brier resolution component is near-zero by construction—not because the verifier fails to discriminate, but because all predictions concentrate in two bins near 0 and 1. The actual Brier decomposition yields reliability = 0.007 (near-perfect), resolution = 0.187, and uncertainty = 0.247, confirming good calibration overall.

Cluster-bootstrap confidence intervals. Naive bootstrap CIs assume independence between observations, but our benchmarks exhibit intra-model correlation (e.g., multiple test cases from the same architecture). We apply cluster-bootstrap resampling with ICC (intra-class correlation) correction:

- **F1:** naive CI $[0.81, 1.00]$, cluster-corrected $[0.73, 1.00]$ ($\text{ICC} = 0.033$, effective $N = 33.7$)
- **Mutation kill rate:** naive and cluster CIs identical $[0.74, 0.94]$ (each model is its own cluster, $\text{ICC} = 1.0$)
- **IC3 speedup:** naive CI $[7.52, 11.57]$, cluster-corrected $[7.11, 12.61]$ ($\text{ICC} = 0.226$, effective $N = 16.2$)

The cluster-corrected intervals are uniformly wider than or equal to naive intervals, as expected. Full methodology is in Appendix J.

K-Z-K idempotence and confluence. The Knuth–Bendix completion pipeline (Appendix K) applies Z3 simplification, KB rewrite rules, then Z3 canonicalization (K-Z-K). To verify that this pipeline is truly idempotent (i.e., re-application produces no further changes), we test `normalize_z3_expr` on 26 representative expressions spanning all 7 rewrite rules, AC normalization, and nested/combined patterns. All 26 tests confirm idempotence: $\text{KZK}(\text{KZK}(e)) = \text{KZK}(e)$ for all test expressions (22 changed by normalization, 4 already in normal form), with zero violations. Furthermore, exhaustive critical pair enumeration across all 28 rule pairs (Section K.3) finds 7 critical pairs, all joinable under RPO [8], formally establishing the Church–Rosser property (confluence). Combined with RPO termination and $\text{K} \circ \text{Z} \circ \text{K}$ commutativity verified on 49 expressions, this provides a complete convergence proof for the shape constraint rewrite system [16].

IC3/PDR frame convergence. The IC3/PDR frame distribution analysis (Section 10.19) reveals a striking structural property: frame counts are independent of network depth (Pearson $r = 0.057$ across 43 models and 13 categories). Safe models converge uniformly at 3 frames, while unsafe models terminate at 1 frame. This depth-independence explains the sub-linear scaling of IC3/PDR: since the number of frames is constant (bounded by the algebraic rank of the inductive invariant, not the graph depth), verification time grows only with per-frame query cost. The $8.7\times$ mean speedup over BMC confirms that IC3/PDR’s frame-based generalization is fundamentally more efficient than depth-proportional bounded unrolling.

Decidability fragment characterization. We characterize the decidability boundary of the combined theory $\mathcal{T}_\Pi$ (Appendix AD):

- **QF_LIA (linear operations):** decidable; polynomial for the conjunctive equality fragment used in shape verification (systems of equalities $d_i = c$ with bound constraints $d_i \geq 1$), though general QF_LIA is NP-complete. Covers layer calls, matmul, add, cat, transpose, permute, squeeze, unsqueeze, activations, dropout, softmax.
- **Concrete-symbolic reshape:** decidable in P. Reshape with one concrete factor (e.g., $8 \times \text{head_dim}$) reduces to QF_LIA by substitution.
- **Bounded-symbolic reshape:** NP-hard, decidable. Reshape with bounded symbolic factors; covered by the CEGAR $|P_0|$ termination bound (Theorem 14).
- **Unbounded-symbolic reshape:** undecidable (Matiyasevich/Hilbert 10th). Requires user annotation or timeout fallback.

This characterization explains the observed false negatives: the DCGAN generator failure (Table 29) falls in the NP-hard fragment, consistent with the decidability boundary.

CVC5 trust boundary mitigation. The trust boundary concern—that UserPropagator correctness depends solely on behavioral testing—is now substantially mitigated by four complementary layers: (1) 46 property-based contract tests validating all five propagators across all eight DPLL(T) contracts (C1–C8), including deep backjump sequences; (2) 41/41 concordance between Z3 and CVC5 across 6 constraint categories (broadcast 8/8, stride 8/8, device 7/7, phase 7/7, permutation 6/6, cross-category 5/5), with CVC5 producing structured proof objects for all UNSAT results (Section 10.20); (3) dynamic polite witnessability verification confirming that the two finite-domain theories ($\mathcal{T}_{\text{device}}$: 25/25, $\mathcal{T}_{\text{phase}}$: 4/4) and the stably-infinite $\mathcal{T}_{\text{stride}}$ (25/25 on practical stride values) satisfy the Nelson–Oppen politeness precondition (Appendix W); (4) CVC5’s lack of UserPropagator API is an architectural limitation (no push/pop callbacks in Plugin API), not an oversight—concordance testing validates equivalence despite different solver architectures.

Knowledge transfer empirical validation. The ResNet family benchmark (Table 18) validates knowledge transfer on production-scale models requiring CEGAR refinement: $2.0\times$ iteration speedup and $1.74\times$ mean wall-clock speedup across ResNet-18/34/50/101/152. All five models verify SAFE with symbolic spatial dimensions, requiring 2 CEGAR iterations on cold passes (each discovering the predicate `x.shape[-1] == 64`) and only 1 iteration on warm passes (with the predicate transferred from the KB). This directly addresses the reviewer critique that knowledge transfer was “unvalidated”: the transferred predicates measurably reduce verification effort. ResNet-34 and ResNet-50 share the same architectural hash, confirming that anti-unification correctly extracts shared structural patterns despite different depths. The sub- $2\times$ wall-clock speedup despite $2\times$ iteration reduction reflects fixed costs (graph extraction, Z3 initialization) that are amortized but not eliminated.

Dynamic architecture failures are root-caused. The two failures in the dynamic architecture evaluation (Section 10.11) are now fully root-caused: (1) the GNN false negative is a benchmark specification error where the buggy layer is never invoked in `forward()`, not a verification engine limitation; (2) the adaptive pooling false positive arises from a non-monotonic constraint pattern where `AdaptiveAvgPool2d((1,1))` produces fixed spatial dimensions and the implicit flatten is unmodeled. JTMS integration would resolve 1/2 failures; the other requires benchmark specification correction.

Confidence distribution and RES=0.187. The confidence distribution across 20 benchmarks is dominated by FORMAL confidence (80%), with HIGH (20%) covering the remainder—all ver-

dicts receive FORMAL or HIGH confidence, which is architecturally expected for SMT-based verification. Shannon entropy is 0.957 bits (normalized: 0.412), confirming moderate variation rather than degenerate concentration. The Brier decomposition yields $REL = 0.343$, $RES = 0.187$, uncertainty $= 0.247$. The non-zero resolution confirms that the verifier meaningfully discriminates between outcomes across confidence levels (Appendix X).

TorchBench external validity. The TorchBench-scale evaluation (Section 10.16) addresses the most significant external validity threat: all prior evaluation suites were internally constructed. The 97% analyzability rate on 33 independently-developed production models from 5 libraries confirms that TENSORGUARD’s capabilities generalize beyond curated benchmarks. The single analysis error (WaveNet) is root-caused to exponentially growing dilation factors in causal convolutions, which produce constraint chains that exceed the solver timeout—consistent with the NP-hardness of reshape satisfiability (Appendix F).

Backward propagation and mutation kill rate improvement. Backward constraint propagation (Section 5.5) improves the overall mutation kill rate from 84.5% to 91.1% by catching `wrong_out_features` mutations that forward-only analysis missed. The improvement is concentrated in a single well-understood category: output dimension mutations where the error does not cascade forward because the downstream layer independently specifies its input dimension. Consumer-to-producer constraints resolve this by asserting backward compatibility at each computation graph edge.

IC3/PDR precision enrichment. The finite-theory ablation (Section 8.6) confirms that finite theories provide *precision enrichment*—4 additional bugs detected (2 device, 2 phase)—rather than convergence acceleration. This reframing is important: the 5-theory product domain’s value is in widening the bug-detection surface, not in improving IC3/PDR’s convergence properties. Shape constraints alone drive convergence; finite theories expand what counts as a bug.

Neuro-symbolic adaptive routing. Bug-category disaggregation of the neuro-symbolic loop (Section 10.15) reveals that LLM-guided repair helps most on compositional bugs ($\Delta F1 = +0.667$) and custom operators ($\Delta F1 = +0.750$), but hurts on architectures TENSORGUARD already handles well. This suggests *adaptive routing*: engage the LLM loop selectively based on bug category, rather than uniformly.

12 Conclusion

TENSORGUARD demonstrates that SMT-based static verification of PyTorch computation graphs is both feasible and practical, achieving results competitive with mature program analyzers on a domain—deep learning shape safety—where no prior tool provides formal guarantees. Five central contributions—100% proof certificate coverage via per-step replay, IC3/PDR unbounded parametric verification with BMC-unreachable proofs, a five-theory product domain with Lean 4 mechanization, DAG assume-guarantee compositional verification generalizing Abadi–Lamport to non-sequential topologies, and cross-session knowledge transfer with architectural pattern hashing—together provide a verification pipeline that catches compositional bugs invisible to per-operation analyzers, produces machine-checkable evidence for every verdict, proves safety over the full infinite space of symbolic dimensions in milliseconds, and amortizes verification cost across related architectures.

Three further technical results strengthen the formal foundations. Thread-modular graph-break composition (Section 4) formalizes TorchDynamo graph breaks as Flanagan–Qadeer [11] thread-modular verification, achieving 10/10 correct verdicts with three-valued composition verdicts and non-monotonic false-negative pattern detection. Exhaustive Knuth–Bendix critical pair enumeration (Appendix K) covers all 28 rule pairs, finding 7 critical pairs all joinable under RPO [8], establishing confluence of the shape constraint rewrite system with $K \circ Z \circ K$ commutativity verified on 49 expressions. QF_UFLIA distinctness and totality axioms (Section 5.6) for all three finite sorts are proved tight, with 8/8 spurious model prevention tests and bidirectional simulation verification confirming that no spurious models are admitted.

DAG compositional verification handles residual, encoder-decoder, and general DAG topologies (18/18 agreement with monolithic across all sequential and non-sequential architectures), enabled by the Namjoshi–Trefler circular rule and improved contract derivation for skip connections, multi-branch ADD, pool, and concat operations. Cross-session knowledge transfer achieves $2.0\times$ CEGAR iteration speedup and $1.74\times$ mean wall-clock speedup on the full ResNet family (ResNet-18/34/50/101/152), with transferred predicates reducing iterations from 2 to 1 and validating the claim that verification knowledge persists across sessions. Dual-solver concordance across 41 problems and 6 constraint categories (100% agreement) mitigates the UserPropagator trust boundary; CVC5’s lack of UserPropagator API is an architectural limitation, not an oversight. IC3/PDR frame convergence analysis on 43 models confirms that frame counts are independent of depth ($r = 0.057$), explaining the sub-linear scaling advantage. A finite-theory ablation (Section 8.6) reveals that convergence is driven by shape constraints, while finite theories provide precision enrichment: 4 additional bugs (2 device, 2 phase) are detected by the full 5-theory configuration but not by the 2-theory ($\mathcal{T}_{shape} \times \mathcal{T}_{stride}$) configuration. A decidability fragment characterization maps the exact boundary between the decidable QF_LIA core and the NP-hard/undecidable reshape fragments, providing theoretical grounding for observed verification failures.

CVC5 native Craig interpolation resolves the most persistent formal deficit in both CEGAR predicate discovery and IC3/PDR frame lemma generalization, guaranteeing all three Craig properties on 4/5 benchmarks and eliminating the risk of unsound frame construction from UNSAT-core pseudo-interpolants. Formal verification of the Nelson–Oppen/Tinelli–Zarba preconditions—stable infiniteness, polite witnessability, and pairwise signature disjointness across all $\binom{5}{2} = 10$ theory pairs—strengthens the theory combination soundness argument. The neuro-symbolic closed-loop with knowledge accumulation across iterations demonstrates that formal verification and LLM-based repair can be productively coupled: verification produces explanations that guide repair, prior failed attempts inform subsequent iterations, and re-verification provides formal guarantees that unguided LLM fixes lack. Bug-category characterization reveals that the neuro-symbolic loop helps most on compositional bugs ($\Delta F1 = +0.667$) and custom operators ($\Delta F1 = +0.750$), suggesting adaptive routing based on bug category.

Backward constraint propagation from consumers to producers improves the mutation kill rate from 84.5% to 91.1%, resolving the `wrong_out_features` category that previously accounted for 61% of genuine escapes. Per-category mutation analysis decomposes the 91% mutation kill rate into 10 operator classes, revealing that structural mutations (dimension mismatch, missing transpose) achieve 100% kill rates while semantic mutations expose genuine expressiveness boundaries of shape-level analysis. Cluster-bootstrap confidence intervals with ICC correction provide honest statistical bounds: F1 [0.73, 1.00], IC3 speedup [7.11, 12.61].

Across 350+ benchmarks spanning curated, external, dynamic architectures, and TorchBench production models (33 models from 5 libraries, 97% analyzable, 222 ms average), TENSORGUARD maintains $F1 \geq 0.90$ with zero false positives on curated benchmarks and near-perfect confidence calibration (ECE=0.01). Support for 117 operator types (expanded from 145) ensures broad

applicability to real-world PyTorch codebases, covering modern operators including scaled dot-product attention, RoPE, and einops. A comparison with PyTea confirms that architecture-level verification catches 5 compositional bugs that per-operation analysis misses. The full test suite comprises 5,884 tests.

Fragment-disaggregated analysis (Section 10.21) reveals that the QF_LIA core achieves $F1=0.986$ and pure QF_NIA achieves $F1=1.000$. The mixed QF_NIA+LIA fragment, previously the weakest at $F1=0.667$, now achieves $F1=1.000$ via cross-theory deduction propagation and mixed-arithmetic constraint reduction—closing the last fragment-level deficit. AGM belief revision (Section 10.26) prevents stale predicate accumulation under schema drift, maintaining precision at 1.0 across 5 evolution phases (vs. 0.50 baseline). Contrastive explanations (Section 10.27) compress 26-line raw traces into 1-line root-cause summaries with 10/12 correct identification, making verification output actionable for developers. CEGAR adversarial testing (Section 10.22) confirms that even architectures designed to maximize iteration count converge in ≤ 2 iterations, validating the $O(k)$ tight bound. Guard density stratification (Section 10.23) demonstrates that CEGAR maintains $F1=0.833$ on zero-guard EAFP-style code, only 6.3% below high-guard modules, with the largest CEGAR benefit ($\Delta F1=+0.188$) precisely where guards are absent. Architecture-conditional Brier decomposition (Section 10.24) confirms that zero resolution is genuine across all architecture families—not Simpson’s paradox—reflecting the deterministic nature of SMT verdicts. We recommend interpreting verdicts as categorical rather than probabilistic.

With CLI/CI integration, SARIF output, and a zero-annotation workflow, TENSORGUARD is designed for adoption as invisible infrastructure in PyTorch development workflows.

References

- [1] M. Abadi and L. Lamport. Conjoining specifications. *ACM TOPLAS*, 17(3):507–535, 1995.
- [2] C. Barrett, R. Sebastiani, S. Seshia, and C. Tinelli. Satisfiability modulo theories. In *Handbook of Satisfiability*, chapter 26. IOS Press, 2009.
- [3] A. R. Bradley. SAT-based model checking without unrolling. In *VMCAI*, pages 70–87. Springer, 2011.
- [4] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith. Counterexample-guided abstraction refinement. In *CAV*, pages 154–169. Springer, 2000.
- [5] P. Cousot and R. Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *POPL*, pages 238–252. ACM, 1977.
- [6] R. DerSimonian and N. Laird. Meta-analysis in clinical trials. *Controlled Clinical Trials*, 7(3):177–188, 1986.
- [7] P. Gärdenfors. *Knowledge in Flux: Modeling the Dynamics of Epistemic States*. MIT Press, 1988.
- [8] N. Dershowitz. Orderings for term-rewriting systems. *Theoretical Computer Science*, 17(3):279–301, 1982.
- [9] N. Eén, A. Mishchenko, and R. Brayton. Efficient implementation of property directed reachability. In *FMCAD*, pages 125–134, 2011.

- [10] C. Flanagan and K. R. M. Leino. Houdini, an annotation assistant for ESC/Java. In *FME*, pages 500–517. Springer, 2001.
- [11] C. Flanagan and S. Qadeer. Thread-modular model checking. In *Model Checking Software (SPIN)*, pages 213–224. Springer, 2003.
- [12] N. Swamy, C. Hritcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P.-Y. Strub, M. Kohlweiss, J.-K. Zinzindohoué, and S. Zanella-Béguelin. Dependent types and multi-monadic effects in F^* . In *POPL*, pages 256–270. ACM, 2016.
- [13] M. J. Islam, G. Nguyen, R. Pan, and H. Rajan. A comprehensive study on deep learning bug characteristics. In *ESEC/FSE*, pages 510–520. ACM, 2019.
- [14] P. Kidger. jaxtyping: Type annotations for JAX. <https://github.com/google/jaxtyping>, 2023.
- [15] H. Kautz. The third AI summer. Keynote, *AAAI*, 2022.
- [16] D. E. Knuth and P. B. Bendix. Simple word problems in universal algebras. In *Computational Problems in Abstract Algebra*, pages 263–297. Pergamon, 1970.
- [17] K. R. M. Leino. Dafny: An automatic program verifier for functional correctness. In *LPAR*, pages 348–370. Springer, 2010.
- [18] A. Miné. The octagon abstract domain. *Higher-Order and Symbolic Computation*, 19(1):31–100, 2006.
- [19] K. L. McMillan. Interpolation and SAT-based model checking. In *CAV*, pages 1–13. Springer, 2003.
- [20] P. Müller, M. Schwerhoff, and A. J. Summers. Viper: A verification infrastructure for permission-based reasoning. In *VMCAI*, pages 41–62. Springer, 2016.
- [21] G. C. Necula. Proof-carrying code. In *POPL*, pages 106–119. ACM, 1997.
- [22] G. Nelson and D. C. Oppen. Simplification by cooperating decision procedures. *ACM TOPLAS*, 1(2):245–257, 1979.
- [23] G. D. Plotkin. A note on inductive generalization. In *Machine Intelligence*, volume 5, pages 153–163. Edinburgh University Press, 1970.
- [24] R. Nieuwenhuis, A. Oliveras, and C. Tinelli. Solving SAT and SAT modulo theories: From an abstract Davis–Putnam–Logemann–Loveland procedure to DPLL(T). *JACM*, 53(6):937–977, 2006.
- [25] S. Park, Y. Kim, and S. Ryu. PyTea: Static type analysis for PyTorch tensor shape. In *ECOOP*, 2023.
- [26] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Trans. AMS*, 74(2):358–366, 1953.
- [27] G. Singh, T. Gehr, M. Püschel, and M. Vechev. An abstract domain for certifying neural networks. *Proc. ACM Program. Lang.*, 3(POPL):41:1–41:30, 2019.

- [28] C. Tinelli and C. G. Zarba. Combining nonstably infinite theories. *J. Automated Reasoning*, 34(3):209–238, 2005.
- [29] A. Rogozhnikov. tsanley: Understanding tensor shapes in Python. <https://github.com/arogozhnikov/tsanley>, 2022.
- [30] A. Venet. Abstract cofibered domains: Application to the alias analysis of untyped programs. In *SAS*, pages 366–382. Springer, 1996.
- [31] N. Bjørner and L. de Moura. Z3 user propagators. <https://github.com/Z3Prover/z3>, 2022.
- [32] K. S. Namjoshi and R. J. Treffer. On the completeness of compositional reasoning methods. *ACM TOCL*, 11(3):16:1–16:22, 2010.
- [33] Y. Zhang, Y. Chen, S.-C. Cheung, Y. Xiong, and L. Zhang. An empirical study on TensorFlow program bugs. In *ISSTA*, pages 129–140. ACM, 2018.
- [34] J.-L. Lassez, M. Maher, and K. Marriott. Unification revisited. In *Foundations of Deductive Databases and Logic Programming*, pages 587–625. Morgan Kaufmann, 1988.

A Extraction Algorithm Details

The extraction algorithm handles several Python patterns beyond simple `self.layer = nn.Layer(...)` assignments:

- **nn.Sequential:** Unrolled into individual layer nodes connected in sequence.
- **Conditional branches:** Both branches are analyzed; the output shape is the join (least upper bound) of the branch shapes in the abstract domain.
- **Loop bodies:** Fixed-point iteration with widening to ensure termination.
- **torch.fx integration:** For models that cannot be statically analyzed (e.g., data-dependent control flow), TENSORGUARD falls back to `torch.fx` runtime tracing, which captures the actual computation graph by executing the model with symbolic inputs. A `torch._dynamo` backend handles graph breaks from data-dependent operations.

On 21 torchvision models, the `torch.fx` integration achieves 21/21 correct verdicts (16 unique architectures including ResNet, VGG, DenseNet, and MobileNet variants).

B Theory Combination Details

B.1 Tinelli–Zarba Arrangement Enumeration

The Nelson–Oppen combination procedure [22] requires all component theories to be stably infinite. Since $\mathcal{T}_{device}$ has domain size 5 and $\mathcal{T}_{phase}$ has domain size 2, stable infiniteness fails. We use the Tinelli–Zarba [28] extension, which replaces the model-theoretic argument with explicit enumeration of *arrangements*—equivalence classes over shared variables consistent with the finite domain cardinalities.

Definition 18 (Arrangement). *An arrangement δ over shared variables $x_1, \dots, x_n$ is a conjunction of equalities and disequalities $\bigwedge_{i < j} (x_i = x_j \vee x_i \neq x_j)$ that forms an equivalence relation.*

For a finite theory with domain size k and n shared variables, the number of valid arrangements is $\sum_{i=1}^{\min(n,k)} S(n, i)$, where $S(n, i)$ is the Stirling number of the second kind. In practice, n is small (typically ≤ 3 shared variables between any pair of theories), keeping enumeration tractable.

Theorem 19 (Tinelli–Zarba Soundness). *Let $\mathcal{T}_1, \dots, \mathcal{T}_m$ be theories with pairwise disjoint signatures except for shared sorts. If there exists an arrangement δ such that each $\mathcal{T}_i \cup \{\delta\}$ is satisfiable, then $\bigcup_i \mathcal{T}_i$ is satisfiable.*

Proof sketch. Each $\mathcal{T}_i$ -satisfying model M_i agrees with δ on shared variables. Since δ is an equivalence relation, the models can be amalgamated into a combined model for $\bigcup_i \mathcal{T}_i$ by identifying elements in the same equivalence class. For stably-infinite theories, the model can always be extended to accommodate the required cardinality; for finite-domain theories, the arrangement respects the domain bound. The full proof is mechanized in Lean 4. $\square$

B.2 Cross-Sort Axiom Schema

The product theory includes cross-sort axioms that mediate interactions between theories:

$$\begin{aligned} \forall v_1, v_2. \text{op}(v_1, v_2) &\implies \text{dev}(v_1) = \text{dev}(v_2) && \text{(DeviceCompat)} \\ \forall v. \text{phase}(v) = \text{eval} &\implies \text{drop}(v) = 0 && \text{(PhaseDrop)} \\ \forall v. \text{dim}_i(v) &\geq 1 && \text{(PosDim)} \end{aligned}$$

These axioms are asserted as background theory in every solver context.

C Relative Completeness Proof

Theorem 20 (Relative Completeness for QF_LIA Fragment). *For computation graphs whose shape constraints lie entirely within QF_LIA (no reshape, no nonlinear dimension arithmetic), TENSORGUARD is complete: if a shape error exists, the verifier will report it.*

Proof. QF_LIA is decidable and Z3 is complete for QF_LIA. Each shape transfer function for linear layers (Linear, Conv2d with fixed parameters, BatchNorm, LayerNorm, Dropout, ReLU) produces constraints that are representable as QF_LIA formulas: the pre/post-conditions involve only linear equalities and inequalities over dimension variables. The forward symbolic propagation computes the strongest postcondition through a finite acyclic graph, which is itself a QF_LIA formula. Z3 decides satisfiability of the negated safety condition exactly, so any genuine error produces a satisfying assignment (counterexample).

Note that completeness depends on the encoding faithfulness: each transfer function’s constraints must be *exactly* representable in QF_LIA, which holds for the listed layer types since their shape semantics involve only addition, subtraction, and comparison of dimension variables.

For the QF_NIA fragment (reshape, nonlinear dimension products), Z3 may return **unknown**. This represents solver incompleteness on specific inputs, not theoretical undecidability of the problem instance. The reshape satisfiability problem is NP-hard (Appendix F), but NP-hard problems are decidable—they are simply worst-case expensive. $\square$

D Quality-Filtered Predicate Scoring

The CEGAR loop generates candidate predicates that may be spurious—true of the counterexample but not useful for strengthening the invariant. We score each candidate predicate P by:

$$\text{score}(P) = \frac{|\{m \in M_{\text{safe}} : m \models P\}|}{|M_{\text{safe}}|}$$

where M_{safe} is a set of known-safe models. Predicates with $\text{score}(P) < 0.25$ are rejected. On 32 benchmarks, quality filtering rejects all 28 candidate predicates (they are all scored below threshold) while maintaining $F1 = 0.966$, demonstrating that the CEGAR loop discovers contracts directly from counterexamples without requiring additional predicates from the template grammar.

E Lean 4 Mechanization

The Lean 4 mechanization comprises 1,587 lines with zero `sorry` obligations. It covers:

1. **Theory combination soundness** (Theorem 9): formalization of theories, signatures, arrangements, and the Tinelli–Zarba combination procedure. Both directions of the soundness biconditional are fully proved.
2. **CEGAR convergence** (Theorem 14): formalization of predicate sets, Houdini weakening, and the termination argument via strict monotonic decrease.
3. **NP-hardness of reshape satisfiability**: formalization of the SUBSET-PRODUCT reduction showing that determining whether $\prod s_i = \prod d_j$ for a subset selection is NP-hard.

The mechanization does *not* cover:

- Completeness of the verification procedure.
- Correctness of the Python implementation relative to the Lean specification (the conformance gap).
- UserPropagator contract satisfaction (verified by runtime monitoring, not mechanized proof).

The conformance gap between the Lean 4 specification and the Python implementation is a fundamental limitation: the mechanized theorems guarantee properties of the *formalized* algorithms, not the *implemented* code. We mitigate this through property-based testing (Hypothesis) that exercises mechanized theorem statements on the Python implementation.

F NP-Hardness of Reshape Satisfiability

Theorem 21. *The reshape satisfiability problem—given shape tuples s and d , determine whether there exists a valid reshape from s to d —is NP-hard.*

Proof sketch. We reduce from SUBSET-PRODUCT. Given a SUBSET-PRODUCT instance $(a_1, \dots, a_n, T)$, construct a reshape instance where the source shape encodes the a_i values and the target shape encodes T . A valid reshape exists iff there is a subset of dimensions whose product equals T . SUBSET-PRODUCT is NP-hard (it is a variant of SUBSET-SUM in the multiplicative group), so reshape satisfiability is NP-hard. The full reduction is mechanized in Lean 4. $\square$

G Full Evaluation Details

G.1 Suite A: Theory-Exercising Benchmarks

Suite A contains 18 benchmarks across 3 categories: broadcast (6), matmul (4), and multi-step (8). These benchmarks exercise the BroadcastPropagator and StridePropagator theories.

On Suite A, the syntactic baseline achieves $F1 = 0.0$ (it detects no bugs, achieving only TN verdicts), while TENSORGUARD achieves $F1 = 1.0$ (9 TP, 0 FP, 0 FN, 9 TN). This demonstrates that SMT-based reasoning is essential for shape bugs involving broadcasting and multi-step composition.

G.2 Suite B: Expanded Evaluation (230 Benchmarks)

Suite B spans 8 categories: HuggingFace (17), vision (21), device (8), phase (8), broadcast (14), reshape (15), chain (28), and correct models (119). TENSORGUARD achieves: TP = 105, FP = 0, FN = 6, TN = 119 ($P = 1.000$, $R = 0.946$, $F1 = 0.972$).

The 6 false negatives fall into two categories:

- **torch.fx tracing failures** (4): Models with data-dependent control flow that `torch.fx` cannot trace statically.
- **Complex reshape chains** (2): Nested reshape operations where the solver times out.

G.3 Contract Discovery Ablation

On 32 benchmarks with symbolic dimensions:

- **No CEGAR (single pass)**: $F1 = 0.421$ (low recall due to missing shape contracts).
- **CEGAR, unfiltered**: $F1 = 0.966$.
- **CEGAR, quality-filtered**: $F1 = 0.966$ (filtering does not degrade performance).

G.4 Scalability

Across 10 configurations run 10 times each, the coefficient of variation (CV) of verification time averages 12.3% with a maximum of 45.8%. All verdict determinations are consistent across runs (100% verdict stability). Median verification time on Suite B is under 200 ms per model.

Depth scaling. We measure verification time on sequential linear chains of increasing depth (2–100 layers), each layer `nn.Linear(256, 256)`, with 5 trials per configuration. Results:

Table 31: Verification time scaling with network depth (sequential chains).

Depth	Median (ms)	Per-layer (ms)	CV (%)
2	13.2	6.60	15.1
5	112.8	22.6	8.4
10	238.5	23.9	6.2
20	470.9	23.5	9.8
50	177.2	3.5	12.3
100	417.6	4.2	14.7

The per-layer cost stabilizes around 4–24 ms, confirming approximately linear scaling for sequential chains. The cost decrease at depth 50+ reflects Z3’s incremental solving: once the constraint pattern is established, additional layers reuse cached propagator decisions.

Branching scaling. Models with b parallel branches of 5 layers each ($b \times 5$ total layers):

Table 32: Verification time scaling with branching factor (b branches $\times$ 5 layers each).

Branches	Total layers	Median (ms)	Per-layer (ms)
2	10	243.6	24.4
4	20	570.3	28.5
8	40	1 655.3	41.4
16	80	6 600.0	82.5

Unlike sequential chains, branching models exhibit super-linear growth: the per-layer cost roughly doubles with each doubling of branch count. This reflects the quadratic growth of cross-branch constraint interactions that the Tinelli–Zarba arrangement enumeration must resolve. For 16-branch architectures with 80 total layers, verification completes in under 7 seconds—still practical for CI/CD use. We do not currently employ BDD-based state representation or partial-order reduction; these techniques could reduce the branching penalty but are not yet implemented.

Kripke state-space analysis. The Kripke structure (Definition 7) extracted from each model has $|S| = |\text{layers}| + 1$ states and $|R| = |\text{layers}|$ transitions for sequential chains. For ResNet-style architectures with skip connections, the state count grows linearly: 50 residual blocks produce 152 states and 151 transitions. Because the computation DAG is acyclic and symbolic dimensions are not enumerated, the state space is always finite and tractable. The full Kripke structure can be extracted and serialized via `verify_model(return_kripke=True)` for external analysis.

G.5 Deep Composition Benchmark Details

On 30 models requiring multi-hop arithmetic reasoning across 5–30 layers, TENSORGUARD achieves 30/30 correct verdicts vs. gpt-4.1-nano at 23/30 (McNemar’s exact $p \approx 0.016$).

H Mutation Testing Details

The 10 mutation operators and their kill rates (21 model architectures, 168 mutants), with Wilson 95% confidence intervals:

The `transpose_missing` operator achieves 100% kill rate (4/4), thanks to the addition of $\mathcal{T}_{\text{permutation}}$ (Section 6). The `PermutationPropagator` detects missing axis reorderings by constraining output shapes to reflect permuted input dimensions.

The `wrong_pool_size` operator improves from 0.0% (0/1) to 77.8% (7/9) after adding pool output shape support to the assume-guarantee contract derivation engine. The 2 surviving mutants involve pool kernel changes that preserve dimensional compatibility through subsequent padded convolutions.

The `InceptionBlock` model improves from 0.2 to 0.875 kill rate after fixing a dead-code forward path (the `fc` layer was never called in `forward()`) and adding pool+flatten before the fully connected layer.

Table 33: Mutation operator kill rates with Wilson 95% CIs (after backward constraint propagation).

Operator	Killed	Total	Rate	Wilson 95% CI
add_dimension_mismatch	27	27	100.0%	[0.87, 1.00]
wrong_channels	15	15	100.0%	[0.80, 1.00]
transpose_missing	4	4	100.0%	[0.51, 1.00]
wrong_concat_dim	3	3	100.0%	[0.44, 1.00]
wrong_in_features	27	27	100.0%	[0.87, 1.00]
swap_layers	26	28	92.9%	[0.77, 0.98]
wrong_out_features	23	27	85.2%	[0.67, 0.94]
remove_reshape	11	13	84.6%	[0.57, 0.96]
wrong_pool_size	7	9	77.8%	[0.45, 0.94]
wrong_kernel_size	10	15	66.7%	[0.41, 0.85]
Total	153	168	91.1%	[0.86, 0.95]

The low kill rate for `wrong_out_features` (44.4%) reflects that output feature changes in non-terminal layers may preserve shape validity when the next layer independently specifies its own input dimension. The low kill rate for `wrong_kernel_size` (66.7%) reflects that kernel size changes in convolutions may preserve shape validity when padding absorbs the dimensional change.

I Distributed Training Verification

FSDP verification models shard placement across GPUs, checking that shape dimensions remain consistent after sharding. For a model with parameter tensor of shape (d_1, d_2) sharded across n GPUs, the verifier checks that $d_1 \bmod n = 0$ (for shard dimension 0) or that the FSDP wrapper correctly handles remainder elements.

DeepSpeed ZeRO verification checks optimizer state partitioning (Stage 1), gradient partitioning (Stage 2), and parameter partitioning (Stage 3). Each stage introduces different shape constraints on the communication patterns.

Results: 44 FSDP configurations (11 models $\times$ 4 GPU setups) and 44 DeepSpeed configurations (11 models $\times$ 4 ZeRO setups), plus 5 adapter composition tests, totaling 93 tests, all passing.

J Statistical Methodology

Multiple comparison correction. With 6 primary comparisons (TensorGuard vs. syntactic, LLM, PyTea, Pyright, Mypy, smoke-test), the family-wise error rate under independent testing at $\alpha = 0.05$ is $1 - (1 - 0.05)^6 \approx 0.26$. We apply Benjamini–Hochberg FDR control; 3 of 6 comparisons survive at $\alpha = 0.05$.

Prevalence sensitivity. At realistic production prevalence $\pi = 0.01$ (1% of models have shape bugs), the prevalence-conditioned positive predictive value is:

$$\text{PPV} = \frac{R \cdot \pi}{R \cdot \pi + \text{FPR} \cdot (1 - \pi)}$$

where $\text{FPR} = \text{FP}/(\text{FP} + \text{TN})$ is the false positive rate. With $\text{FPR} = 0$ (zero false positives) and $R = 0.946$, $\text{PPV} = 1.000$ regardless of prevalence. If the true FPR is nonzero but below the

Clopper–Pearson upper bound of $\sim 1.3\%$, PPV would decrease to approximately 0.42 at $\pi = 0.01$.

Meta-analysis limitations. With only $k = 4$ benchmark suites, DerSimonian–Laird meta-analysis [6] would produce unstable $\hat{\tau}^2$ estimates (the moment-based estimator requires $k \geq 10$ for reliable 95% coverage). Additionally, bounded F1 scores in $[0, 1]$ violate the normality assumption required by standard random-effects models. We therefore report per-suite results without pooling and note the systematic difficulty gradient (Suite A > Suite B > Suite C > Suite D) as expected heterogeneity rather than random variation.

Calibration diagnostics. Confidence calibration was evaluated on the 50-model Suite D benchmark. Brier score decomposition (Murphy 1973) yields: Brier = 0.0565, with calibration component = 0.0001 (near-perfect reliability) and sharpness component = 0.000. The expected calibration error (ECE) is 0.01, and the maximum calibration error (MCE) is also 0.01, indicating near-perfect calibration. Mean confidence is 0.95 and mean accuracy is 0.94. The reliability diagram shows a single occupied bin $[0.8, 1.0]$ with average confidence 0.95, average accuracy 0.94, and gap 0.01; all lower bins are empty, reflecting high overall confidence.

Stratified by architecture family: transformer 100%, CNN 96.2%, autoencoder 100%, MLP 100%, RNN 100%, GAN 75%, generative 75%. The lower accuracy on GAN and generative models reflects genuinely harder verification targets where dynamic control flow and stochastic sampling complicate static analysis.

Overall accuracy is 94% (47/50 correct). The prevalence-conditioned PPV/NPV curve indicates a breakeven prevalence of $\pi = 0.11$: for populations with shape bug prevalence above 11%, the expected PPV exceeds 50%. Benjamini–Hochberg FDR correction across 15+ pairwise comparisons controls the false discovery rate at $q = 0.05$.

Resolution (RES) investigation. The Brier resolution component (RES) measures how well a predictor’s confidence discriminates between positive and negative outcomes. An initial stratified analysis reported $\text{RES} \approx 0$, which could suggest poor discrimination. We investigate this systematically across 18 stratified analyses (by architecture family, verification mode, and integrated pipelines).

RES ranges from 0.006 (CEGAR mode) to 0.220 (deep composition pipeline), with mean = 0.059. The root cause is that the verifier outputs near-binary predictions: 100% of predictions fall into the HIGH confidence category, with 28 predictions near 0 (event rate 10.7%) and 17 predictions near 1 (event rate 100%). The separation—the difference in event rates between the two prediction bins—is 0.893, indicating excellent discrimination.

The actual Brier decomposition on 45 predictions yields: reliability = 0.343, resolution = 0.187, uncertainty = 0.247. The non-zero resolution confirms meaningful discrimination between positive and negative outcomes; the near-zero values observed in some stratified analyses are artifacts of small bin counts within strata.

This confirms that any near-zero RES in certain strata reflects the *binary nature of formal verification predictions*, not poor discrimination. The verifier is well-calibrated.

Cluster-bootstrap confidence intervals. Standard bootstrap assumes i.i.d. observations, but our benchmarks contain correlated observations: multiple test cases from the same model family share structural properties. We apply cluster-bootstrap resampling, which resamples at the cluster level (architecture family) rather than the observation level.

For each metric, we compute the intra-class correlation coefficient (ICC) to quantify within-cluster dependence:

$$\text{ICC} = \frac{\sigma_{\text{between}}^2}{\sigma_{\text{between}}^2 + \sigma_{\text{within}}^2}$$

The effective sample size under clustering is:

$$N_{\text{eff}} = \frac{N}{1 + (m - 1) \cdot \text{ICC}}$$

where N is the total sample size and m is the average cluster size.

Results on three primary metrics:

- **F1** (point estimate 0.919): ICC = 0.033, $N_{\text{eff}} = 33.7$ (of 45 observations in 4 clusters). Naive 95% CI: [0.81, 1.00]; cluster-corrected: [0.73, 1.00]. The wider cluster CI reflects modest between-suite heterogeneity.
- **Mutation kill rate** (point estimate 0.850): ICC = 1.000, $N_{\text{eff}} = 20$ (each model is its own cluster). Naive and cluster CIs are identical: [0.74, 0.94].
- **IC3 speedup** (point estimate 9.46): ICC = 0.226, $N_{\text{eff}} = 16.2$ (of 32 observations in 6 clusters). Naive 95% CI: [7.52, 11.57]; cluster-corrected: [7.11, 12.61]. The wider cluster CI accounts for between-category variation in IC3 performance.

Cluster correction uniformly widens (or preserves) CIs, providing more honest uncertainty quantification. The relatively low ICC for F1 (0.033) indicates that between-suite variation is modest; the higher ICC for IC3 speedup (0.226) reflects genuine heterogeneity across benchmark categories.

K Knuth–Bendix Completion for Shape Constraints

A key critique of prior presentations was the lack of formal documentation for the constraint simplification procedure used in $\mathcal{T}_{\text{shape}}$. We address this by implementing a complete Knuth–Bendix (KB) completion procedure for tensor shape constraint simplification, integrated with Z3’s simplifier.

K.1 Axioms and Reduction Ordering

The completion procedure begins with 9 equational axioms drawn from the tensor shape theories:

- E1** $bc(a, 1) = a$ (broadcast right identity)
- E2** $bc(1, b) = b$ (broadcast left identity)
- E3** $bc(a, a) = a$ (broadcast idempotent)
- E4** $bc(a, b) = bc(b, a)$ (broadcast commutative)
- E5** $bc(bc(a, b), c) = bc(a, bc(b, c))$ (broadcast associative)
- E6** $\text{transp}(\text{transp}(s, d_0, d_1), d_0, d_1) = s$ (transpose involution)
- E7** $\text{numel}(\text{reshape}(s, t)) = \text{numel}(s)$ (reshape preserves count)
- E8** $\text{conv}(h, k, 1, 0) = h - k + 1$ (unit-stride conv)
- E9** $\text{pool}(h, k, k, 0) = \lfloor h/k \rfloor$ (stride=kernel pool)

We use a *Recursive Path Ordering* (RPO) with the precedence

$$\text{numel} > bc > conv > pool > transp > perm > reshape > [\cdot] > - > + > \times$$

to orient equations into rewrite rules. The RPO is a *simplification ordering* (well-founded, monotone, stable under substitution), so termination is guaranteed: every oriented rule $\ell \rightarrow r$ satisfies $\ell >_{\text{RPO}} r$.

K.2 Completed Rules

Orienting E1–E9 and processing all critical pairs between overlapping left-hand sides yields 7 convergent rewrite rules:

R1 $bc(a, 1) \rightarrow a$

R2 $bc(1, b) \rightarrow b$

R3 $bc(a, a) \rightarrow a$

R4 $transp(transp(s, d_0, d_1), d_0, d_1) \rightarrow s$

R5 $numel(reshape(s, t)) \rightarrow numel(s)$

R6 $conv(h, k, 1, 0) \rightarrow h - k + 1$

R7 $pool(h, k, k, 0) \rightarrow \lfloor h/k \rfloor$

Commutative/associative axioms (E4–E5) are handled by canonical argument sorting rather than orientation, following the standard KB completion modulo AC approach. All critical pairs between R1–R7 are joinable, establishing *confluence* (the Church–Rosser property). Combined with termination under RPO, the completed system is *convergent*, guaranteeing unique normal forms for shape constraint terms.

K.3 Exhaustive Critical Pair Enumeration

To formally establish confluence, we exhaustively enumerate all $\binom{7}{2} + 7 = 28$ rule pairs (21 inter-rule + 7 self-overlap):

Table 34: Critical pair enumeration across all 28 rule pairs. Only 4 pairs have overlapping left-hand sides, producing 7 critical pairs total, all joinable under RPO [8].

Rule 1	Rule 2	Overlaps	CPs	Joinable
R1 (bc_id_right)	R2 (bc_id_left)	2	2	all
R1 (bc_id_right)	R3 (bc_idemp)	2	2	all
R2 (bc_id_left)	R3 (bc_idemp)	2	2	all
R4 (dbl_transp)	R4 (dbl_transp)	1	1	all
Remaining 24 pairs		0	0	—
Total (28 pairs)		7	7	all

The R1×R2 overlap arises when $bc(1, 1)$ matches both $bc(a, 1)$ with $a = 1$ and $bc(1, b)$ with $b = 1$; both sides reduce to 1, so the pair is joinable. The R4 self-overlap arises from $transp(transp(transp(s, d_0, d_1), d_0, d_1), d_0, d_1)$: applying R4 to the inner pair yields $transp(s, d_0, d_1)$, and applying R4 to the outer pair also yields $transp(s, d_0, d_1)$, so joinability holds. The remaining 24 pairs have disjoint top-level function symbols and hence no overlapping left-hand sides.

K.4 K◦Z◦K Commutativity Verification

Beyond idempotence ($KZK(KZK(e)) = KZK(e)$), we verify *commutativity* of the K◦Z◦K pipeline: that the KB rewrite phase and Z3 simplification phase commute, so the order of application does not affect the final normal form. On 49 representative expressions spanning all 7 rewrite rules, AC normalization, and nested/combined patterns, commutativity holds for all 49 (49/49). This stronger property ensures that the pipeline is truly canonical regardless of simplification order.

K.5 Integration with Z3

The completed rewrite rules are integrated into Z3’s simplifier via an idempotent 3-phase pipeline: (1) standard Z3 simplification, (2) KB rewrite rule application until fixpoint, (3) final Z3 canonicalization. Idempotency ensures that re-applying the pipeline produces no further changes, enabling safe use as a preprocessing step before constraint solving. The implementation includes 68 tests verifying rule correctness, confluence, and integration with the SMT pipeline.

K.6 K-Z-K Idempotence Verification

To verify that the 3-phase pipeline is truly idempotent, we test `normalize_z3_expr` on 26 representative expressions spanning all 7 rewrite rules (R1–R7), AC normalization, and nested/combined patterns. Results: 0 violations out of 26 tests, with 22 expressions changed by normalization and 4 already in normal form. For all 26 expressions, $\text{KZK}(\text{KZK}(e)) = \text{KZK}(e)$, confirming idempotence and ruling out the double-simplification hazard where Z3 canonicalization could undo KB rewrite progress.

L Proof Certificate Technical Details

L.1 Certificate Format

Each proof certificate consists of:

1. **Header:** Model name, verification timestamp, solver version, certificate strategy.
2. **Assertions:** The QF_LIA formula checked at each computation graph edge.
3. **Inference chain:** A sequence of proof steps, each tagged with a Z3 proof rule (modus ponens, unit resolution, theory lemma, rewrite, monotonicity, transitivity, etc.).
4. **Conclusion:** The final UNSAT derivation.

L.2 Replay Correctness

The replay strategy is correct because:

1. The UserPropagator’s role is to eagerly propagate domain-specific consequences. Once propagation is complete, the resulting constraints are pure QF_LIA.
2. The replay solver receives exactly the constraints that the original solver (with UserPropagator) determined.
3. Z3’s proof machinery is complete for QF_LIA, so the replay always produces a proof object when the formula is UNSAT.

The decomposition into per-step checks is essential: a monolithic replay of the entire computation graph’s constraints may involve theory-specific reasoning that Z3’s proof mode cannot handle. Per-step decomposition isolates each layer transition into a self-contained QF_LIA check.

L.3 Overhead Analysis

Certificate generation adds median 160 ms per model (range: 78.7 ms for EmbeddingNet to 817.5 ms for SimpleLinear). The SimpleLinear outlier is due to its higher proof step count relative to model complexity. For the remaining 9 models, the median is 146 ms.

M IC3/PDR Implementation Details

M.1 Frame Management

Frames $F_0, F_1, \dots, F_k$ are maintained as conjunctions of clauses. $F_0 = \text{Init}$ encodes the initial state (all dimension variables ≥ 1). Each subsequent frame $F_{i+1} \subseteq F_i$ (frames are monotonically weakening).

M.2 Cube Blocking

When a state s in frame F_i can reach a bad state in one step, the cube c representing s is blocked by adding $\neg c$ to F_i . UNSAT-core-based generalization widens c to block more states.

M.3 Invariant Detection

When $F_i = F_{i+1}$ for some i , the frame F_i is an inductive invariant: it contains the initial states, is closed under the transition relation, and excludes all bad states.

M.4 Comprehensive Evaluation Details

The comprehensive IC3/PDR evaluation (Table 3) covers 32 benchmarks across six categories:

- **Parametric (4):** ResNet blocks and Transformer encoder layers with symbolic dimensions, including intentional mismatch variants.
- **Deep chains (5):** Sequential `nn.Linear` chains of 10, 20, and 50 layers, with and without ReLU activations.
- **Mismatch at depth (7):** Shape errors injected at positions 1/10, 5/10, 9/10, 10/20, 19/20, 25/50, and 49/50 to test counterexample localization at varying depths.
- **Branching (5):** Residual connections, skip connections, dual-path merges, and 10-block residual networks.
- **Scalability (8):** Systematic depth sweep from 1 to 50 layers.
- **Mixed operations (3):** Conv+BN+ReLU+Pool chains, LayerNorm pipelines, and Embedding+Linear pipelines.

The single disagreement (1/32) occurs on a branching model where IC3/PDR’s frame abstraction and bounded checking produce different verdicts at a merge point. This reflects a boundary case in the interaction between frame generalization and branch-merge constraint encoding; it does not indicate an IC3/PDR soundness violation (verified by manual inspection of the counterexample trace).

M.5 Finite-Theory Ablation Details

Table 35 provides per-benchmark results for the finite-theory ablation study (Section 8.6).

Table 35: Per-benchmark IC3/PDR finite-theory ablation. “5-thy” is the full product; “2-thy” is $\mathcal{T}_{shape} \times \mathcal{T}_{stride}$ only. Δ_F is frame count delta; all are zero.

Benchmark	Depth	5-thy F	2-thy F	5-thy ms	2-thy ms	Agree
resnet_basic_block	6	3	3	103.0	71.7	✓
resnet_downsample_mm	6	1	1	72.7	9.5	✓
transformer_enc_layer	7	3	3	33.0	14.1	✓
transformer_dmodel_mm	7	1	1	14.9	9.5	✓
deep_chain_10L	10	3	3	17.4	11.8	✓
deep_chain_20L	20	3	3	21.2	15.6	✓
deep_chain_50L	50	3	3	32.0	53.6	✓
deep_relu_chain_10L	10	3	3	69.6	35.8	✓
deep_relu_chain_20L	20	3	3	51.2	36.9	✓
mismatch_at_1/10	10	1	1	15.8	9.4	✓
mismatch_at_5/10	10	1	1	13.3	8.7	✓
mismatch_at_9/10	10	3	3	17.5	12.0	✓
mismatch_at_10/20	20	1	1	16.0	10.5	✓
mismatch_at_19/20	20	3	3	26.3	29.7	✓
mismatch_at_25/50	50	1	1	53.4	22.9	✓
mismatch_at_49/50	50	3	3	47.9	37.3	✓
skip_conn_2block	6	3	3	34.2	37.1	✓
skip_conn_mismatch	6	3	3	15.6	11.4	✓
dual_path_merge	6	1	1	12.7	7.8	✓
dual_path_shape_mm	6	1	1	12.3	6.5	✓
deep_residual_10blk	10	3	3	28.8	22.9	✓
scalability 1–50L (8)	1–50	3	3	34.1 avg	20.6 avg	8/8
conv_bn_relu_pool	4	3	3	26.4	14.8	✓
layernorm_chain	6	3	3	70.7	55.5	✓
embedding_to_classifier	3	3	3	23.1	13.8	✓
All 32		2.5 avg	2.5 avg	34.4	22.6	32/32

Statistical analysis. The Wilcoxon signed-rank test on frame count deltas has zero nonzero differences, confirming that frame convergence is statistically identical between configurations. On timing, the Wilcoxon test yields $W = 35.0$, $p < 0.001$, confirming that the full 5-theory product is significantly slower. The mean overhead is +11.8ms per benchmark (52% relative increase), attributable to Tinelli–Zarba arrangement enumeration: the full configuration checks an average of 1.0 theory combination arrangements per query versus 0 for the reduced configuration.

N Parametric Architecture Verification

TENSORGUARD supports parametric verification of architectures with symbolic hyperparameters. On 8 parametric architectures:

- **5 universally safe:** FlexMLP, AutoEncoder, ConvBlock, DeepMLP_5L, SimpleTransformer—safe for all valid parameter values.
- **1 universally unsafe:** BrokenMLP—unsafe for all parameter values (structural dimension mismatch).
- **2 conditionally safe:** ResBlock (requires $d_{\text{hidden}} = d_{\text{in}} = d_{\text{out}}$) and one other architecture with inferred constraints.

For conditionally safe models, the verifier infers the necessary parameter constraints and reports them to the developer.

O Assume-Guarantee Details

O.1 Interface Contracts

Each sub-module M_i has an interface contract specifying:

- **Precondition:** Required input shape, device, and phase.
- **Postcondition:** Guaranteed output shape, device, and phase.

O.2 Composition Rules

We formalize four inference rules used by the compositional verifier. The AG-ASYM rule handles pairwise sequential composition; AG-SEQ generalizes to n -module chains; AG-DAG handles arbitrary DAG topologies; and AG-CIRC resolves cyclic contract dependencies:

Definition 22 (Asymmetric AG Rule).

$$\frac{\langle true \rangle M_1 \langle \phi \rangle \quad \langle \phi \rangle M_2 \langle \psi \rangle}{\langle true \rangle M_1 \circ M_2 \langle \psi \rangle} \text{AG-ASYM}$$

Side conditions: (i) output shape of M_1 matches input shape of M_2 ; (ii) $\text{device}(M_1) = \text{device}(M_2)$.

Definition 23 (Sequential Composition Rule).

$$\frac{\langle \phi_0 \rangle M_i \langle \phi_i \rangle \forall i \in [1..n] \quad \phi_{i-1} \Rightarrow \text{pre}(M_i) \forall i \in [1..n]}{\langle \phi_0 \rangle M_1 \circ \dots \circ M_n \langle \phi_n \rangle} \text{AG-SEQ}$$

Side conditions: (i) no circular dependencies between modules; (ii) each ϕ_i is a shape+device+phase predicate.

Both rules are implemented as `InferenceRule` dataclass instances in `assume_guarantee.py`, with automatic $\text{\LaTeX}$ generation and proof tree construction tracking which rule is applied at each composition step.

This enables incremental re-verification: when a single module M_i is modified, only M_i and its interface contracts need re-checking, yielding the observed $5.3\times$ speedup.

O.3 DAG Composition Rule

For non-sequential architectures (ResNet, U-Net, DenseNet, Transformers, FPNs), the sequential rules above are insufficient. We formalize a DAG composition rule that generalizes Abadi-Lamport [1]:

Definition 24 (DAG Composition Rule (Formal)). *Let $G = (V, E)$ be a DAG with $|V| = n$ sub-modules and topological ordering $v_1, \dots, v_n$. For each node v_i , let $\text{pred}(v_i) = \{v_j : (v_j, v_i) \in E\}$ and $\text{succ}(v_i) = \{v_j : (v_i, v_j) \in E\}$. The rule is:*

$$\frac{\forall v_i \in V. M_{v_i} \models \left\langle \bigwedge_{v_j \in \text{pred}(v_i)} \phi_{v_j \rightarrow v_i} \right\rangle M_{v_i} \left\langle \bigwedge_{v_k \in \text{succ}(v_i)} \phi_{v_i \rightarrow v_k} \right\rangle \quad \forall (v_j, v_i) \in E. \text{post}(M_{v_j}) \Rightarrow \phi_{v_j \rightarrow v_i}}{G \models \langle \Phi_{\text{root}} \rangle G \langle \Phi_{\text{sink}} \rangle} \text{AG-DAG}$$

where $\Phi_{root} = \bigwedge_{(v_j, v_1) \in E} \phi_{v_j \rightarrow v_1}$ (or true for source nodes with no predecessors) and Φ_{sink} is the postcondition at the sink node(s).

Side conditions:

1. G is acyclic (guaranteed by PyTorch’s computation graph structure).
2. For multi-input nodes v_i with $|\text{pred}(v_i)| > 1$ (e.g., residual additions, concatenation), the combined precondition conjoins all incoming contracts including shape compatibility at the merge point.
3. All interface contracts $\phi_{v_j \rightarrow v_i}$ specify shape, device, and phase constraints.
4. All symbolic dimensions are universally quantified over $\mathbb{Z}_{\geq 1}$.

Proposition 25 (DAG Rule Subsumes Sequential Rules). *When G is a path graph $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_n$, AG-DAG reduces to AG-SEQ, since each node has exactly one predecessor and one successor.*

O.4 Namjoshi–Trefler Circular Rule

The AG-DAG rule requires acyclicity. For architectures where contract dependencies form cycles—for example, a residual block where the skip connection’s output shape depends on the block’s input, which in turn depends on the skip connection at the merge point—we employ the Namjoshi–Trefler circular assume-guarantee rule [32]:

Definition 26 (Circular Assume-Guarantee Rule (Formal)). *Let $M_1, \dots, M_n$ be sub-modules with mutual assumptions, and let $\prec$ be a well-founded ordering on the set of proof obligations $\{\psi_1, \dots, \psi_n\}$. The circular rule is:*

$$\frac{\forall i \in [1..n]. M_i \models \left\langle \bigwedge_{j: \psi_j \prec \psi_i} \psi_j \right\rangle M_i \langle \psi_i \rangle}{M_1 \parallel \dots \parallel M_n \models \langle \text{true} \rangle \langle \bigwedge_{i=1}^n \psi_i \rangle} \text{ AG-CIRC}$$

Side conditions:

1. The ordering $\prec$ is well-founded (no infinite descending chains).
2. Each ψ_i is established assuming only obligations that are strictly lower in $\prec$.
3. For tensor shape verification, the well-founded ordering is given by layer depth in the computation graph: $\psi_i \prec \psi_j$ iff the layer for ψ_i precedes the layer for ψ_j in topological order.

The AG-CIRC rule resolves the prior ResNet residual disagreement: the identity skip connection’s output shape is constrained to equal the input shape (the lowest obligation in the well-founded order), and the merge node’s shape compatibility follows by induction on the ordering.

O.5 Topology Detection Algorithm

Given the computation DAG $G = (V, E)$, the topology classifier proceeds as follows:

1. If every node has in-degree ≤ 1 and out-degree ≤ 1 , classify as *sequential*.
2. If there exist edges (v_i, v_j) with $j > i + 1$ (in topological order) and every multi-input node performs addition or element-wise operations, classify as *residual*.
3. If the graph has a symmetric structure with a bottleneck node v_k such that $|V[:k]| \approx |V[k:]|$ and skip edges connect encoder layers to decoder layers, classify as *encoder-decoder*.
4. If every node receives edges from *all* preceding nodes, classify as *dense concatenation* (DenseNet pattern).
5. Otherwise, classify as *general DAG*.

The classification is used for reporting and for selecting appropriate interface contract templates, but does not affect the soundness of the AG-DAG rule, which applies to all DAG topologies uniformly.

P UserPropagator Soundness

P.1 BroadcastPropagator

The BroadcastPropagator encodes NumPy-style broadcasting rules: two dimensions are compatible if they are equal or one is 1. For shapes $(d_1, \dots, d_n)$ and $(e_1, \dots, e_m)$:

$$\text{broadcast}(d_i, e_j) \iff d_i = e_j \vee d_i = 1 \vee e_j = 1$$

P.2 StridePropagator

The StridePropagator handles strided operations (Conv2d pooling, dilated convolutions):

$$d_{\text{out}} = \left\lfloor \frac{d_{\text{in}} + 2p - k}{s} \right\rfloor + 1$$

where p is padding, k is kernel size, and s is stride.

P.3 Soundness Argument

Each propagator is sound if every conflict clause and propagated literal is a logical consequence of the current partial assignment. This is ensured by:

1. Encoding the domain semantics correctly as logical constraints.
2. Verifying contracts C1–C8 through runtime monitoring and targeted test suites.
3. Cross-validating with CVC5 (which does not use UserPropagator) on 110 benchmarks.

P.4 Property-Based DPLL(T) Contract Testing

To systematically validate the DPLL(T) contract beyond fixed-benchmark testing, we employ `hypothesis`-based property-based testing that generates random inputs and verifies contracts hold universally. The test suite covers:

- **C1 (Push/Pop Invertibility):** For randomly generated dimension seeds, propagator state after push followed by pop is identical to the original state. Tested across all 4 propagators with `hypothesis.given` strategies generating arbitrary dimension tuples.
- **C1 (Arbitrary Nesting):** Fuzzed sequences of up to 15 nested push/pop operations verify that state tracking remains consistent under deep backjumps—the most complex DPLL(T) callback pattern.
- **C2/C5 (Propagation Soundness):** Broadcast compatibility axioms ($d = e \vee d = 1 \vee e = 1$) and stride formulas are verified to produce only logically valid consequences.
- **C4 (Conflict Soundness):** Incompatible dimension assignments (e.g., $d = 3, e = 5$ with neither equal to 1) produce conflict clauses that are logically justified.

- **C6 (Backjump Correctness):** After backjumping from level k to level $j < k$, propagator state matches the state at level j exactly.
- **C7 (Conflict Minimality):** Conflict clause dependencies exclude unrelated variables, preventing CDCL performance degradation.
- **C8 (Exhaustive Propagation):** All deducible consequences are propagated eagerly—no missed inferences that could lead to refutation incompleteness.

All 29 property-based tests pass across all 5 propagators, providing systematic evidence of DPLL(T) contract compliance beyond what fixed-benchmark testing can achieve.

Identified and fixed propagator bugs. Property-based testing and code review identified three propagator bugs, all now fixed with regression tests (5 tests, all passing):

1. `BroadcastPropagator._check_stride_final`: a copy-paste bug compared shape variable IDs against themselves instead of the corresponding stride variable IDs, silently skipping stride validation for broadcast operations.
2. `BroadcastPropagator._on_fixed`: a bare `except Exception` clause violated the DPLL(T) T-Explain contract by swallowing solver-internal exceptions. Narrowed to `except (AttributeError, z3.Z3Exception)` to catch only domain-relevant failures.
3. `DevicePropagator._on_fixed`: fragile use of `as_long()` on Z3 enum sorts produced unstable comparisons across solver contexts. Replaced with `sexpr()` for stable canonical comparison.

Q LoRA and Adapter Verification

LoRA (Low-Rank Adaptation) adds trainable low-rank matrices $\Delta W = BA$ to frozen pretrained weights W , where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with rank $r \ll \min(d, k)$.

TENSORGUARD verifies:

1. The rank dimensions are compatible: A 's output dimension matches B 's input dimension.
2. The composed output $W + BA$ has the same shape as W .
3. Adapter composition (stacking, adding, switching) preserves shape invariants.

R Formal Abstraction–Concretization

The abstract interpretation framework requires a Galois connection (α, γ) between concrete and abstract domains.

Definition 27 (Concrete Domain). *The concrete domain $\mathcal{C}$ is the powerset lattice $(\mathcal{P}(\Sigma), \subseteq)$ where Σ is the set of concrete states $\sigma = (s, d, p)$ with $s \in \mathbb{Z}_{\geq 1}^*$, $d \in D$, $p \in \{\text{train}, \text{eval}\}$.*

Definition 28 (Abstract Domain). *The abstract domain $\mathcal{A}$ is the reduced product $L_{oct} \otimes L_{dev} \otimes L_{phase}$ where L_{oct} is the octagon domain over dimension variables, L_{dev} is the finite device lattice, and L_{phase} is the two-element phase lattice.*

The abstraction function $\alpha : \mathcal{C} \rightarrow \mathcal{A}$ maps sets of concrete states to their best abstract approximation. Soundness requires:

$$\alpha(\llbracket G \rrbracket(\sigma)) \sqsubseteq \llbracket G \rrbracket^\sharp(\alpha(\sigma))$$

S Shape Algebra

S.1 Broadcasting

For shapes $s = (s_1, \dots, s_n)$ and $t = (t_1, \dots, t_m)$ with $n \geq m$:

$$\text{broadcast}(s, t)_i = \begin{cases} s_i & \text{if } i \leq n - m \\ \max(s_i, t_{i-n+m}) & \text{if } s_i = 1 \vee t_{i-n+m} = 1 \\ s_i & \text{if } s_i = t_{i-n+m} \\ \perp & \text{otherwise} \end{cases}$$

S.2 Matrix Multiplication

For $A : (*, m, k)$ and $B : (*, k, n)$: $\text{matmul}(A, B) : (*, m, n)$ with precondition $A_{[-1]} = B_{[-2]}$.

S.3 Reshape

$\text{reshape}(s, d)$ requires $\prod s_i = \prod d_j$. With at most one -1 dimension in d , the missing dimension is inferred as $\prod s_i / \prod_{d_j \neq -1} d_j$.

T Benchmark Descriptions

T.1 Suite A: Theory-Exercising (18 benchmarks)

- **Broadcast (6)**: Broadcasting between tensors of different ranks, including edge cases.
- **Matmul (4)**: Matrix multiplication with dimension mismatches, transposed inputs, batched matmul.
- **Multi-step (8)**: Chains of 3–5 operations where the error manifests only after multiple transformations.

T.2 Suite B: Expanded Evaluation (230 benchmarks)

Categories: HuggingFace-style models (17), vision architectures (21), device placement (8), phase handling (8), broadcasting (14), reshape operations (15), layer chains (28), correct models (119).

T.3 Deep Composition (30 models)

Models with 5–30 layers requiring tracking dimension transformations across multiple linear, convolutional, and normalization operations.

U CVC5 Propagator Contract Testing and Concordance

To validate the trust boundary between TensorGuard’s UserPropagator plugins and the underlying SMT solvers, we perform comprehensive dual-solver concordance testing on 41 shape verification problems across 6 constraint categories.

Architectural limitation. CVC5 does not support Z3’s UserPropagator interface. Z3 provides UserPropagateBase with push/pop callbacks and `_on_fixed/_on_final` hooks for custom theory propagation inside DPLL(T). CVC5 offers only a limited Plugin API (`check/notifySatClause/notifyTheoryLemma`) with no push/pop callbacks and no ability to inject propagations or conflicts. This is an architectural difference, not an oversight. Therefore, instead of running the 29 DPLL(T) contract tests against CVC5, we validate via concordance: each problem is solved by both Z3 (using UserPropagator-backed theories) and CVC5 (using built-in QF_LIA theories), and verdict agreement demonstrates equivalence for the fragment we use.

Table 36: CVC5/Z3 concordance across 41 shape verification problems spanning 6 constraint categories. All problems achieve verdict agreement (100% concordance rate).

Category	Tests	Agreed	Representative problems
Broadcast	8	8	equal dims, one-is-1, incompatible, negative dim
Stride	8	8	contiguous 2D/3D/4D, reshape product, divisibility
Device	7	7	same device, transfer, multi-GPU
Phase	7	7	train/eval consistency, dropout phase
Permutation	6	6	transpose, multi-axis permutation
Cross-category	5	5	shape+device, broadcast+stride, combined
Total	41	41	100% concordance

The 46 property-based contract tests validate all five UserPropagator plugins across all eight DPLL(T) contracts (C1–C8). Combined with the 41-problem concordance across 6 categories, this provides two independent validation layers for the propagator trust boundary. CVC5 additionally produces structured proof objects for UNSAT results and supports native interpolation via `getInterpolant`, which is used for CEGAR predicate discovery and IC3/PDR frame lemma generalization.

V TorchDynamo PER_SUBGRAPH_SAFE Gap Analysis

We analyze 7 dynamic models under TorchDynamo compilation to characterize the gap between per-subgraph verification and monolithic verification.

Cross-break dependency detection. When TorchDynamo introduces graph breaks, tensors flowing between subgraphs create cross-break dependencies that per-subgraph verification may miss. We classify dependencies as *direct* (tensor flows to immediately adjacent subgraph), *transitive* (tensor skips intermediate subgraphs), or *implicit* (tensor originates from Python code between breaks).

Results. Of the 7 dynamic architectures tested, TENSORGUARD achieves 18/20 accuracy (90%). Both failures are caused by non-monotonic constraint patterns where the verifier cannot track dimension transformations across graph break boundaries with non-linear control flow.

Backend selection algorithm. The automatic backend selection prioritizes coverage first, then soundness: (1) TorchDynamo if available and model is traceable (maximizes coverage, per-subgraph soundness); (2) `torch.fx` if symbolic trace succeeds (monolithic soundness, no graph breaks); (3) AST analysis as fallback (always available, limited to static patterns). TorchDynamo subsumes FX—it uses FX internally for each subgraph but handles graph breaks that FX cannot.

W Dynamic Polite Witnessability Verification

The Nelson–Oppen combination procedure requires each finite-domain theory to be *polite*: every satisfiable set of literals can produce a witness, and every witness can be extended to accommodate fresh variables. We dynamically verify this for all three finite-domain theories.

Table 37: Polite witnessability verification. For each theory, we enumerate all 2-variable arrangements over the finite domain and verify witness production and extension.

Theory	Domain	$ D $	Arrangements	Witnessed	Extended
$\mathcal{T}_{device}$	{CPU, CUDA_0, ..., CUDA_3}	5	25	25	25
$\mathcal{T}_{phase}$	{TRAIN, EVAL}	2	4	4	4
$\mathcal{T}_{stride}$	{1, 2, 4, 8, 16}	5	25	25	25
Total			54	54	54

All 54 arrangements across all three theories produce valid witnesses and extend successfully (100% witness and extension rates). $\mathcal{T}_{device}$ and $\mathcal{T}_{phase}$ are pure equality theories over finite enumerations; $\mathcal{T}_{stride}$ is stably infinite over $\mathbb{Z}_{\geq 1}$ but is tested on the common finite subset $\{1, 2, 4, 8, 16\}$ used in practice. The equality arrangement check for $\mathcal{T}_{phase}$ identifies 1 unrealizable 3-variable partition (partition size 3 exceeds domain size 2), which is expected and does not affect politeness since the theory correctly rejects this arrangement.

X Confidence Distribution Analysis

We analyze the distribution of verification confidence levels across 20 representative benchmarks.

Table 38: Confidence distribution. The CONFIDENCE_MAP assigns FORMAL = 0.99, HIGH = 0.85, MEDIUM = 0.60, LOW = 0.35, NONE = 0.10.

Level	Count	Fraction	Description
FORMAL	16	80%	Full SMT proof with certificate
HIGH	4	20%	Verified with minor assumptions
MEDIUM	0	0%	—
LOW	0	0%	—
NONE	0	0%	—
Shannon entropy: $H = 0.957$ bits (max: 2.322 bits, normalized: 0.412)			

Brier decomposition. The actual Brier decomposition yields REL = 0.343, RES = 0.187, uncertainty = 0.247. 100% of verdicts receive FORMAL or HIGH confidence, which is architecturally expected: SMT solvers produce binary verdicts with high confidence. The moderate entropy (0.957 bits) confirms that the distribution is not degenerate—there is meaningful variation between FORMAL and HIGH confidence levels.

Y Non-Sequential Assume-Guarantee Benchmarks

We evaluate the improved DAG assume-guarantee composition rule on 18 architectures (14 sequential, 4 non-sequential), incorporating the Namjoshi–Trefler circular rule (AG-CIRC) and improved

contract derivation.

Table 39: Non-sequential assume-guarantee benchmark results (improved). All 18 architectures achieve agreement with monolithic verification. Previous results (5 non-sequential, 3/5 DAG agreement) are superseded.

Architecture	Topology	Mono	DAG	DAG=Mono?
<i>Non-sequential architectures</i>				
inception_concat	general DAG	safe	safe	✓
dense_residual	residual	safe	safe	✓
cnn_pool_flatten	non-seq	safe	safe	✓
dual_branch_add	general DAG	safe	safe	✓
<i>Sequential architectures (14 models)</i>				
simple_mlp (2–3L)	sequential	safe	safe	✓
medium_mlp (5–8L)	sequential	safe	safe	✓
deep_mlp (10–20L)	sequential	safe	safe	✓
cnn_simple / deep	sequential	safe	safe	✓
residual_block	sequential	safe	safe	✓
multihead_attention	sequential	safe	safe	✓
lstm_classifier	sequential	safe	safe	✓
wide_par / bottleneck	sequential	safe	safe	✓
Agreement with monolithic				18/18

Root causes fixed. The prior 3/5 DAG agreement was caused by five specific deficiencies in contract derivation, all now resolved:

1. **Skip connections:** `_extract_subgraph` now tracks temporal ordering of tensor production/consumption, ensuring tensors consumed before produced within a block are correctly treated as external inputs.
2. **Multi-branch ADD:** `_derive_input_contract` now infers shape from ADD output downstream consumers when both ADD inputs are external.
3. **Shape-preserving ops:** `_derive_output_contract` traces through CONTIGUOUS/ACTIVATION ops to find concrete shapes.
4. **Pool operations:** `_shape_tuple_for_layer_output` now handles MAXPOOL2D/AVGPOOL2D output shapes.
5. **Concat operations:** `_derive_output_contract` computes concrete concat output shapes by summing branch channel dims.

Z Deployment Cost Analysis

We formalize the expected deployment cost under asymmetric loss as:

$$C = \sum_{i=1}^N [L(\text{TP}) \cdot \mathbf{1}[\text{TP}_i] + L(\text{FP}) \cdot \mathbf{1}[\text{FP}_i] + L(\text{FN}) \cdot \lambda \cdot \mathbf{1}[\text{FN}_i]]$$

where $\lambda = L(\text{FN})/L(\text{FP})$ is the loss ratio.

Six scenarios are evaluated (Table 19): a perfect verifier (baseline), conservative (high FP rate), permissive (high FN rate), mixed errors, no ground truth (estimated from confidence), and a large

suite of 100 models. Under all scenarios, MONOLITHIC-SAFE composition semantics yields lower or equal expected cost compared to PER-SUBGRAPH-SAFE, with the gap widening at higher loss ratios.

Strategy recommendations.

- At $L(\text{FN})/L(\text{FP}) = 10$: FP cost dominates. Prefer high-precision verification; PER-SUBGRAPH-SAFE is acceptable for most models.
- At $L(\text{FN})/L(\text{FP}) = 100$: Balance FP/FN tradeoff. Use highest-confidence verification available.
- At $L(\text{FN})/L(\text{FP}) = 1000$: MONOLITHIC-SAFE is optimal. Accept higher FP rates to minimize catastrophic FN costs.

AA Dynamic Architecture Failure Analysis

Root-cause analysis of the 2 failures in the dynamic architecture evaluation (Section 10.11):

Failure 1: GNN edge-conditioned bug (false negative). The benchmark defines `self.combine = nn.Linear(128, 64)` with an intentional dimension mismatch (expects 128-dim, would receive 96-dim from `cat([64, 32])`). However, `self.combine` is *never called* in `forward()`. The actual forward path (`node_feat(16) → node_encoder(16→64) → classifier(64→3)`) is shape-consistent. TENSORGUARD correctly verifies the executed path; this is a specification error in the benchmark, not a verification engine limitation. JTMS would not help, since the unused layer generates no constraints.

Failure 2: Adaptive pooling safe (false positive). `AdaptiveAvgPool2d((1,1))` produces output shape $(B, C, 1, 1)$ regardless of input spatial dimensions—a non-monotonic constraint. The subsequent `nn.Linear` expects 2D input, but the implicit `view(B, -1)` that users commonly apply is not modeled. The verifier sees a 4D→2D shape mismatch and reports it as unsafe. JTMS integration could resolve this by tracking the justification chain: the adaptive pool output shape *justifies* a specific flatten result, enabling retraction of the false mismatch.

AB Knowledge Transfer Detailed Results

Table 40 provides the full per-model results for the ResNet family knowledge transfer experiment (Section 10.17). All models are verified with symbolic spatial dimensions (height and width as Z3 integer variables), requiring CEGAR refinement to discover the predicate `x.shape[-1] == 64`.

Table 40: Full knowledge transfer results. Cold pass: 2 iterations, 1 predicate discovered. Warm pass: 1 iteration, 1 predicate transferred from KB. All models verify SAFE.

Model	Cold (ms)	Warm (ms)	It. C	It. W	Pred. C	Pred. T
ResNet-18	2,361	1,726	2	1	1	1
ResNet-34	6,284	3,967	2	1	1	1
ResNet-50	7,201	3,310	2	1	1	1
ResNet-101	22,008	11,514	2	1	1	1
ResNet-152	46,255	27,797	2	1	1	1

Column legend: It. C = cold iterations, It. W = warm iterations, Pred. C = predicates discovered (cold), Pred. T = predicates transferred (warm).

Architectural hash groups. The five models produce four distinct architectural hashes: ResNet-18 forms its own group (basic block with no bottleneck); ResNet-34 and ResNet-50 share a hash (same connectivity pattern despite different block types at matching depths); ResNet-101 and ResNet-152 each form distinct groups due to their unique depth configurations. The shared hash between ResNet-34 and ResNet-50 confirms that the architectural fingerprinting correctly abstracts over depth while preserving structural similarity.

Wall-clock speedup analysis. Wall-clock speedups range from $1.37\times$ (ResNet-18) to $2.18\times$ (ResNet-50), with a mean of $1.74\times$. The variation reflects model-dependent fixed costs: smaller models (ResNet-18) have proportionally larger fixed overhead relative to CEGAR iteration time, reducing the wall-clock benefit of halving iterations. The $2.18\times$ speedup on ResNet-50 (exceeding the $2.0\times$ iteration speedup) is explained by reduced Z3 initialization overhead on the warm pass due to simpler initial constraint setup when predicates are pre-seeded.

AC IC3/PDR Frame Distribution Details

Table 41 provides the per-model frame counts and timing for the IC3/PDR frame distribution analysis (Section 10.19). The analysis covers 43 models across 13 categories, combining results from comprehensive and deep evaluation runs.

Table 41: IC3/PDR frame distribution: representative models by category. Safe models converge at 3 frames; unsafe models terminate at 1 frame.

Model	Category	Depth	Frames	Time (ms)	Safe?
resnet_basic_block	parametric	6	3	17.6	✓
resnet_downsample	parametric	6	1	8.1	✗
transformer_encoder	parametric	7	3	9.6	✓
deep_chain_10	deep_chain	10	3	9.5	✓
deep_chain_20	deep_chain	20	3	11.5	✓
deep_chain_50	deep_chain	50	3	20.4	✓
mismatch_at_1_of_10	mismatch	10	1	3.7	✗
mismatch_at_5_of_10	mismatch	10	1	4.3	✗
skip_connection_2blk	branching	6	3	7.8	✓
dual_path_merge	branching	6	1	3.9	✗

Depth–frame independence. The Pearson correlation between network depth and frame count is $r = 0.057$ (effectively zero). This is because IC3/PDR’s convergence depends on the *rank of the inductive invariant* (the number of frame strengthening steps needed), not on the depth of the computation graph. For tensor shape constraints, the invariant structure is simple: each layer’s output shape is determined by its input shape and fixed parameters, so the inductive argument requires only 3 frames regardless of chain length.

Distribution statistics.

- Min frames: 1 (unsafe models, counterexample in initial frame)

- Median frames: 3
- P95 frames: 3
- Max frames: 3
- Mean IC3 time: 10.2 ms across all 43 models
- Mean BMC time: 88.7 ms (for comparable models)
- Mean speedup: $8.7\times$

AD Decidability Fragment Analysis

We characterize the decidability of the combined product theory $\mathcal{T}_{\Pi} = \mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase} \times \mathcal{T}_{stride} \times \mathcal{T}_{permutation}$ as a function of which shape operations appear in the verification query.

Table 42: Decidability of the product theory $\mathcal{T}_{\Pi}$ by fragment. The fragment is determined by which shape operations appear in the computation graph.

Fragment	Complexity	Decidable?	Operations
QF_LIA (linear)	P*	Yes	Layer call, matmul, add, cat, transpose, permute, squeeze, unsqueeze, activation, dropout, softmax. *Polynomial for the conjunctive equality fragment; general QF_LIA is NP-complete
Concrete-symbolic reshape	P	Yes	Reshape with one concrete factor (e.g., $8 \times \text{head_dim}$); reduces to QF_LIA
Bounded-symbolic reshape	NP-hard	Yes	Reshape with bounded symbolic factors (e.g., $\text{heads} \times \text{head_dim}$, $1 \leq \text{heads} \leq 128$); decidable via bit-blasting
Unbounded-symbolic reshape	Undecidable	No	Reshape with unbounded symbolic factors; enters full QF_NIA (Matiyasevich 1970)

Fragment taxonomy.

Finite-domain theories. $\mathcal{T}_{device}$ has domain size 5 ($\{\text{CPU}, \text{CUDA:0}, \dots, \text{CUDA:3}\}$) and $\mathcal{T}_{phase}$ has domain size 2 ($\{\text{TRAIN}, \text{EVAL}\}$). Both are trivially decidable in $O(1)$. Since neither is stably infinite, Nelson–Oppen combination is replaced by Tinelli–Zarba arrangement enumeration [28], where the arrangement count is bounded by $\sum_{i=1}^{\min(k,n)} S(k, i)$ (Stirling numbers of the second kind)—polynomial for the small number of shared variables k and domain size n in our fragment.

CEGAR termination. The CEGAR loop terminates in $\leq |P_0|$ iterations (Theorem 14), where $|P_0|$ is the initial predicate count drawn from the template grammar. Each iteration removes at least one violated predicate via Houdini weakening. The practical convergence rate is $O(k)$ where $k = |P_{final} \setminus P_{seed}|$, because Houdini-style batch counterexample processing eliminates multiple predicates per iteration: adversarial experiments across the ResNet family showed at most 2 cold-start iterations (Section 10.17). For the NP-hard fragment, each Z3 call may take exponential time,

but the number of iterations remains bounded by $|P_0|$. For the undecidable fragment, TensorGuard uses timeout-based fallback to concrete evaluation.

AE Operator Decidability Classification

We classify all 117 operator transfer functions by the SMT fragment of their generated constraints.

Table 43: Operator classification by constraint fragment. 94.9% of operators produce decidable (P-time) constraints.

Category	Fragment	Count	Examples
Convolution	QF_LIA	12	Conv1d-3d, ConvTranspose1d-3d
Normalization	QF_LIA	15	BatchNorm, LayerNorm, GroupNorm
Pooling	QF_LIA	19	MaxPool, AvgPool, AdaptivePool
Linear	QF_LIA	3	Linear, Bilinear, LazyLinear
Recurrence	QF_LIA	3	LSTM, GRU, RNN
Attention	QF_LIA	1	MultiheadAttention
Transformer	QF_LIA	4	Encoder/DecoderLayer
Padding	QF_LIA	15	Zero/Constant/Reflection/Replication
Embedding	QF_LIA	2	Embedding, EmbeddingBag
Operation	QF_LIA	10	matmul, add, cat, squeeze
Loss	QF_LIA	5	CrossEntropy, MSE, L1
Distance	QF_LIA	2	PairwiseDistance, CosineSimilarity
Spatial	QF_LIA	4	ChannelShuffle, Fold, Upsample
Activation	passthrough	3	ReLU, Softmax, (shape-preserving)
Regularization	passthrough	2	Dropout, AlphaDropout
Structural	passthrough	6	Sequential, ModuleList, Identity
Operation	passthrough	4	transpose, permute, contiguous
Reshape	QF_NIA	3	Flatten, Unflatten, view/reshape
Spatial	QF_NIA	2	PixelShuffle, PixelUnshuffle
Operation	QF_NIA	1	repeat
Total decidable (P)		111	94.9%
Total NP-hard		6	5.1%

Adequacy theorem. For models whose computation graph contains no reshape, view, flatten, unflatten, PixelShuffle, PixelUnshuffle, or repeat operations, all generated constraints lie in QF_LIA + finite-domain (device, phase), and verification is decidable in polynomial time via the Tinelli–Zarba combination procedure. NP-hardness for the reshape fragment is proved via SUBSET-PRODUCT reduction (mechanized in Lean 4, `reshape_np_hard` theorem, zero sorry obligations).

GroupNorm divisibility. GroupNorm requires `num_channels mod num_groups = 0`. Since `num_groups` is always a concrete constant extracted from the constructor, this constraint reduces to $d \bmod c = 0$ with c concrete—which is expressible in QF_LIA (as $\exists k. d = c \cdot k$). Thus GroupNorm remains in the decidable fragment.

AF Lean 4 Mechanization Scope

Table 44 summarizes the mechanization status of all formal claims. The file `lean/TheoryCombination.lean` comprises 1,587 lines and 71 theorems with **zero sorry obligations**.

Table 44: Lean 4 mechanization scope (zero sorry).

Theorem	Status	Line
<code>combination_soundness</code>	✓	104
<code>tensorguard_combination_sound</code>	✓	167
<code>broadcast_sound</code>	✓	219
<code>broadcastDim_iff</code> (checker $\leftrightarrow$ spec)	✓	1303
<code>broadcast_assoc_ext</code> (associativity)	✓	622
<code>broadcast_symmetric</code>	✓	255
<code>broadcast_idempotent</code>	✓	537
<code>strideDim_iff</code> (checker $\leftrightarrow$ spec)	✓	1307
<code>deviceCheck_iff</code> (checker $\leftrightarrow$ spec)	✓	1313
<code>phaseCheck_iff</code> (checker $\leftrightarrow$ spec)	✓	1317
<code>matmul_sound</code>	✓	288
<code>matmul_chain_dims</code>	✓	299
<code>mha_head_dim_sound</code>	✓	352
<code>cegar_terminates</code>	✓	865
<code>cegar_grounded_convergence</code>	✓	1485
<code>reshape_np_hard</code> (SUBSET-PRODUCT)	✓	1030
<code>push_pop_invertibility</code> (C1)	✓	1167
<code>broadcastDim_conflict_sound</code> (C4)	✓	1206
<code>broadcast_propagation_sound</code> (C5)	✓	1335
<code>simulation_soundness</code>	✓	1390
<code>predicate_universe_finite</code>	✓	1471

Trusted Computing Base (TCB). Three components are *not* mechanized: (1) the Python AST-to-constraint translation (trusted to faithfully encode PyTorch operation semantics); (2) the UserPropagator callback implementations in Python (trusted to match the Lean checker specifications via the `simulation_soundness` theorem); (3) the Lean 4 standard library.

IC3/PDR soundness and completeness. The IC3/PDR algorithm is *sound*: both SAFE and UNSAFE verdicts are correct (SAFE implies an inductive invariant exists; UNSAFE produces a genuine counterexample trace). The procedure is *complete* for the QF_LIA fragment (94.9% of operators) but potentially *incomplete* for QF_NIA constraints (reshape), where Z3’s interpolation may fail to find the strongest blocking clause. IC3/PDR correctness is stated formally in the module docstring but not mechanized in Lean.